

GROUP ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT127-3-2-PFDA

Programming for Data Analysis

Intake Code **: APU2F2411IT(DA)**

Module Code **: CT127-3-2-PFDA**

Hand Out Date **: 6 January 2025**

Hand In Date **: 16 February 2025**

Weightage **: 50%**

Lecturer **: Dr. Kulothunkan A/L Palasundram**

ASSIGNMENT: GROUP REPORT

Student Name	Student ID
Faiyad Mahabub Tasin (Leader)	TP077983
Emmanuel Naranendhya Tyatan	TP077873
Lau Wei Wen	TP079439
Yuke Liew Nyuk Kee	TP079074

Table of Contents

1.0 Introduction (Yuke)..... 4
1.1 Data Description (Wei Wen)..... 4
1.2 Assumptions (Faiyad & Emmanuel)..... 5
1.3 Hypothesis & Objective..... 5
Hypothesis..... 5
Objectives..... 5
2.0 Data Preparation (Faiyad & Emmanuel)..... 6
2.1 Data Import..... 6
2.2 Data Cleaning / Pre - Processing..... 8
2.2.1 Dependent Variable:..... 8
● Loss..... 8
2.2.2 Independent Variables..... 18
● Country..... 18
● Date..... 25
● Web Server..... 26
● OS..... 28
3.0 Data Analysis..... 31
3.1 Objective 1: To investigate the relationship between country and loss. (Faiyad Mahabub Tasin | TP077983 | CS-DA)..... 31
3.1.1 Analysis 1-1: What are the highest and least countries that experienced most of the losses?..... 32
3.1.2 Analysis 1-2: Does total revenue loss for all countries correlate with the frequency of several attacks?..... 35
3.1.3 Analysis 1-3: Do certain countries show a consistent pattern of extreme Loss values, are there any underlying issues that result in these extreme losses?..... 37
3.1.4 Analysis 1-4: Do other factors affect extreme loss values?..... 40
Summary Findings..... 42
Extra Feature..... 43
3.2 Objective 2: To examine the relationship between operating systems and losses. (Emmanuel Naranendhya Tyatan | TP078574 | CS-DA)..... 44
3.2.1 Analysis 2-1: Which operating systems are associated with highest losses?..... 44
3.2.2 Analysis 2-2: Has OS vulnerability changed over time?..... 46
3.2.3 Analysis 2-3: Is there a relationship between OS popularity and loss severity?..... 49
3.2.4 Analysis 2-4: How does OS Patch Level Affect Loss?..... 51
Summary Findings..... 53
3.3 Objective 3: The find the correlation between downtime and loss (Lau Wei Wen| TP079439 | CS-DA)..... 54

3.3.1 Analysis 3-1: What patterns exist between downtime duration and financial losses?. 56
3.3.2 Analysis 3-2: How has the downtime-loss relationship changed over time?..... 58
3.3.3 Analysis 3-3: How does downtime affect overall business loss?..... 61
3.3.4 Analysis 3-4: How has the financial impact of downtime evolved over time?..... 62
Summary Findings..... 64
3.4 Objective 4: To determine if web server versions affect financial losses from attacks (Yuke Liew Nyuk Kee | TP079074 | CS-DA)..... 65
3.4.1 Analysis 4-1: Which web servers experience the highest losses?..... 66
3.4.2 Analysis 4-2: Have web server vulnerabilities changed over time?..... 68
3.4.3 Analysis 4-3: Is there a correlation between web server popularity and financial loss?..... 70
3.4.4 Analysis 4-4: Do specific web servers exhibit seasonal attack patterns?..... 72
Summary Findings..... 75
4.0 Hypothesis Testing Using GLM Model..... 76
5.0 Conclusion..... 78
6.0 Workload Matrix..... 79
7.0 References..... 80

1.0 Introduction (Yuke)

Data analytics plays a crucial role in understanding cybersecurity trends. This project, Web Hacking Trends, leverages data analysis techniques to examine cyberattacks over the past 15 years. The dataset includes key attributes such as attack dates, notifiers, IP addresses, and web server details, allowing for an in-depth investigation of hacking patterns and security vulnerabilities.

Our primary objective is to identify trends in web-based attacks, gather valuable information, and provide strategies for improving cybersecurity defenses. Data exploration, transformation, and visualization are key analytical methods that use R programming to provide automation and reliability. The research focuses on finding anomalies, dealing with missing values, and identifying relationships using statistical analysis, graphs, and tables. Advanced analytical approaches may be used to enhance interpretation and increase the accuracy of results. By detecting important hacking trends, this project is aimed to help cybersecurity experts make data-driven decisions. The information gathered will help improve security measures and build proactive defensive systems. Furthermore, the study provides the basis for future research, assisting companies in staying ahead of new threats in a rapidly changing digital environment.

1.1 Data Description (Wei Wen)

The dataset provides detailed information about website defacement incidents, including key attributes related to the attack and its financial impact. It records the Date of the defacement and whether the incident was Notify by a person or group. The URL and IP address of the compromised server are included, along with the Country where the affected server is located. Technical details such as the operating system (OS), Web server version, and character Encoding used in the defacement message are provided. Additionally, the dataset captures the language (Lang) of the defaced website. Financial impact is measured through the Ransom amount paid (in thousands), Downtime (in days) caused by the attack, and revenue Loss incurred due to the incident. This dataset enables an in-depth analysis of web security vulnerabilities and their economic consequences.

1.2 Assumptions (Faiyad & Emmanuel)

- **OS:**

  In the data, “Unknown” records are kept as valid OS data points. It is said “Unknown” due to several reasons such as system blocking by security control, or intentional masking of OS information by attackers (Stephanie, 2019; Rapid7, n.d.)

- **Country:**

  The “Unknown” present in the “Country” is considered to be one country as a whole, since even after the imputation of many countries there were still many “Unknowns” that couldn't be imputed.

1.3 Hypothesis & Objective

Hypothesis

- Null Hypothesis (H_0) -> There is no significant combined relationship between the independent variables of Month, Year, Season, OS, Webserver, Country, Downtime, and the dependent variable of Loss.
- Alternative Hypothesis (H_1) -> A significant combined relationship is seen between independent variables of Date, Month, Year, Season, OS, Webserver, Country, Downtime, and the dependent variable of Loss.

The hypothesis checks whether over time, system configurations, and geographical location collectively influence financial loss and determines if these factors significantly impact the extent of financial damage.

Objectives

- **First Objective**: To investigate the relationship between country and loss.
- **Second Objective**: To examine the relationship between operating systems and losses.
- **Third Objective**: To find the correlation between downtime and loss
- **Fourth Objective**: To determine if web server versions affect financial losses from attacks

2.0 Data Preparation (Faiyad & Emmanuel)

Data preparation is collecting, integrating, structuring, and arranging raw data for analysis, ensuring accuracy for machine learning algorithms (GeeksforGeeks, 2024).

2.1 Data Import

- At first, we import the hacking data into R studio.

```
# Importing The Dataset

```{r Importing the Dataset, echo=TRUE}
setwd("C:/Users/LENOVO/OneDrive - Asia Pacific University/Class Material/Semester 3/Programming for Data Analysis/Assignment")
original_hacking_dataset <- read.csv("hackingData.csv")
```
```

Figure 1.0: Importing the Hacking Dataset

- We run the code to check whether the data has been imported successfully or not.

```
```{r}
raw_data_frame <- data.frame(original_hacking_dataset)

str(raw_data_frame)
```
```

```
'data.frame':   212093 obs. of  12 variables:
 $ Date     : chr  "02/01/1998" "03/01/1998" "04/01/1998" "04/01/1998" ...
 $ Notify   : chr  "Team CodeZero" "Feliz" "Optiklenz(LOU)" "Team CodeZero" ...
 $ URL      : chr  "http://www.janet-jackson.com" "http://cariari.ucr.ac.cr" "http://marin.k12.ca.us"
"http://www.dm.af.mil" ...
 $ IP       : chr  "" "" "" "" ...
 $ Country  : chr  "UNKNOWN" "COSTARICA" "UNITED STATES" "AFGHANISTAN" ...
 $ OS       : chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ WebServer: chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ Encoding : chr  "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Lang     : chr  "NULL" "NULL" "Helvetica" "NULL" ...
 $ Ransom   : int  NA NA 2034 NA NA 2316 195 NA 2877 1070 ...
 $ DownTime : int  18 31 35 58 59 34 21 37 28 21 ...
 $ Loss     : num  678 1988 12204 NA NA ...
```

Figure 1.1: Check the Data Structure Using str() Function.

```
```{r summary of the whole data, echo=TRUE}
summary(raw_data_frame)
```
```

```
     Date              Notify              URL                IP              Country             OS
 WebServer           Encoding
  Length:212093      Length:212093      Length:212093      Length:212093      Length:212093      Length:212093
Length:212093      Length:212093
  Class :character   Class :character   Class :character   Class :character   Class :character   Class :character
Class :character   Class :character
  Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character
Mode  :character   Mode  :character


     Lang              Ransom         DownTime         Loss
  Length:212093      Min.   : 100    Min.   : 7.00   Min.   :   30
  Class :character   1st Qu.: 817    1st Qu.:20.00   1st Qu.:  730
  Mode  :character   Median :1543    Median :32.00   Median : 1625
                     Mean   :1546    Mean   :31.89   Mean   : 2771
                     3rd Qu.:2272    3rd Qu.:43.00   3rd Qu.: 3085
                     Max.   :3000    Max.   :60.00   Max.   :18000
                     NA's   :160000                  NA's   :33767
```

Figure 1.2: Check the Data Summary Using summary() Function.

	Date	Notify	URL	IP	Country	OS	WebServer	Encoding	Lang	Ransom	DownTime	Loss
1	02/01/1998	Team CodeZero	http://www.janet-jackson.com		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	18	678.0
2	03/01/1998	Feliz	http://cariari.ucr.ac.cr		COSTARICA	Unknown	Unknown	NULL	NULL	NA	31	1988.0
3	04/01/1998	Optiklenz(LOU)	http://marin.k12.ca.us		UNITED STATES	Unknown	Unknown	iso-8859-1	Helvetica	2034	35	12204.0
4	04/01/1998	Team CodeZero	http://www.dm.af.mil		AFGHANISTAN	Unknown	Unknown	utf-8	NULL	NA	58	NA
5	04/01/1998	Team CodeZero	http://www.bolling.af.mil		AFGHANISTAN	Unknown	Unknown	windows-1252	Times New Roman	NA	59	NA
6	04/01/1998	foxy man	http://hope.hinet.net		UNKNOWN	Unknown	Unknown	utf-8	NULL	2316	34	13896.0
7	05/01/1998	Team CodeZero	http://www.nic.ad		ANDORRA	Unknown	Unknown	utf-8	NULL	195	21	390.0
8	05/01/1998	Zyklon	http://www.tamu-commerce.edu			Unknown		NULL	NULL	NA	37	1895.0
9	05/01/1998	net_	http://pr0n.prizon.net		UNKNOWN	Unknown	Unknown	utf-8	NULL	2877	28	11508.0
10	07/01/1998	D.A.M.M.	http://www.unicef.org		UNKNOWN	Unknown	Unknown	windows-1252	NULL	1070	21	2140.0
11	08/01/1998	Optiklenz(LOU)	http://www.wvlc.wvnet.edu		UNKNOWN	Unknown	Unknown	utf-8	NULL	2760	33	16560.0
12	09/01/1998	ViRtU3	http://www.trsystems.com			Unknown		NULL	NULL	NA	17	443.0
13	10/01/1998	Magica de Bin	http://www.webbnet.com		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	43	3509.0
14	11/01/1998	Optiklenz	http://www.easysoftware.com		UNKNOWN	Unknown	Unknown	utf-8	Helvetica	2406	15	1684.2
15	14/01/1998	Magica de Bin	http://www.itp.berkeley.edu		UNKNOWN	Unknown	Unknown	utf-8	NULL	902	39	5412.0
16	15/01/1998	Duncan Silver of L.O.U	http://www.emergent.com		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	50	2296.0
17	17/01/1998	I-h4k4	http://www.thebuzz.com		UNKNOWN	Unknown	Unknown	NULL	NULL	1898	34	11388.0
18	18/01/1998	Zyklon	http://www.kvtc.mtcs.tec.me.us			Unknown		NULL	NULL	NA	55	NA
19	19/01/1998	Older Generation	http://www.njcool.net		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	13	97.0
20	19/01/1998	SqUiRl & TdAwG	http://www.kalnet.com		UNKNOWN	Unknown	Unknown	utf-8	NULL	1809	40	10854.0
21	20/01/1998	Zyklon	http://ceo-www.jrc.it			Unknown		NULL	NULL	NA	38	2290.0
22	22/01/1998	The No)d Crew	http://www.legislate.com		UNKNOWN	Unknown	Unknown	iso-8859-1	NULL	NA	56	NA
23	25/01/1998	Giftgas	http://www.kleiber.com		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	19	412.0
24	25/01/1998	ToxicEdge	http://www.chem.umu.se		SWEDEN	Unknown	Unknown	utf-8	NULL	NA	59	NA
25	26/01/1998	Optiklenz	http://www.hartnet.org		UNKNOWN	Unknown	Unknown	utf-8	Helvetica	NA	50	2467.0
26	28/01/1998	Giftgas	http://www.datapark.net		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	7	NA
27	29/01/1998	SpiritWalker-SisterMoon-Sn1per	http://www.resopal.nl		NETHERLANDS	Unknown	Unknown	utf-8	NULL	1559	24	3118.0
28	29/01/1998	SpiritWalker	http://www.connectos.com		UNKNOWN	Unknown	Unknown	utf-8	NULL	NA	42	3472.0

Figure 1.3: Data Viewing

We can conclude that the data has been imported successfully in R studio and we are good to proceed for the next step.

2.2 Data Cleaning / Pre - Processing

Data cleaning involves detecting, correcting, or eliminating errors, inconsistencies, and inaccuracies present in the data (GeeksforGeeks, 2024b). These are then cleaned for carrying out meaningful analysis and making informed decision-making (GeeksforGeeks, 2024b).

2.2.1 Dependent Variable:

Before cleaning Loss let's understand the data.

- Loss

```
```{r checking for missing values, echo=TRUE}
missing_values <- colSums(is.na(raw_data_frame))
missing_values
```
```

```
     Date    Notify       URL        IP   Country        OS WebServer  Encoding      Lang    Ransom  DownTime      Loss 
        0         0         0         0         0         0         0         0         0    160000         0     33767 
```

Figure 2.0: Checking Missing Values For All Columns

Figure 2.1 shows missing values whereas Loss shows about 33767 missing values.

```
```{r summary statistics for Loss Column, echo=TRUE}
summary(raw_data_frame$Loss)
```
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.    NA's 
     30     730    1625    2771    3085   18000   33767 
```

Figure 2.1: Understand the Statistic Summary of Loss Data

Figure 2.2 shows the description of the statistics of Loss to understand what is happening to loss data.

```
```{r}
skew_p1 <- ggplot(raw_data_frame, aes(x = Loss)) +
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue", color = "black") +
  geom_density(color = "orange", linewidth = 1) +
  labs(title = "Distribution of Loss",
       subtitle = paste("Skewness:", round(skewness(raw_data_frame$Loss, na.rm = TRUE), 3)),
       x = "Loss",
       y = "Density") +
  theme_minimal()
print(skew_p1)
```
```

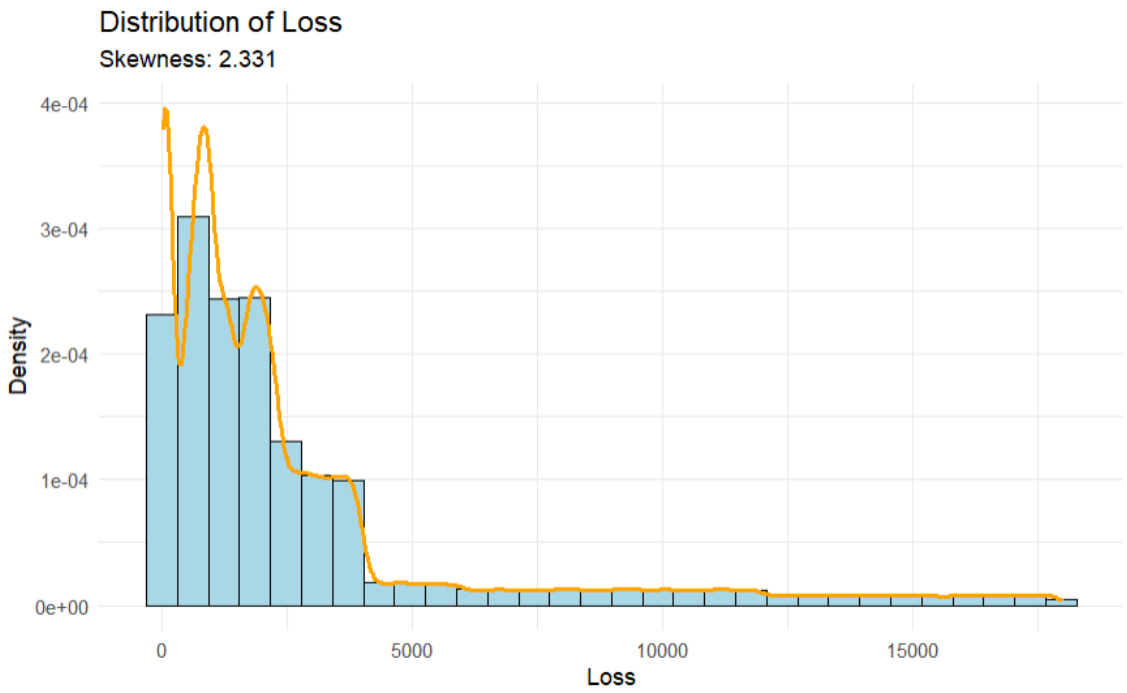


Figure 2.3: Checking the Distribution of Loss

We also run some visualizations to see the distribution of loss. The loss column is mostly rightly skewed where it has a very large skewness value of 2.331.

```r
```{r}
Q1 <- quantile(raw_data_frame$Loss, 0.25, na.rm = TRUE)
Q3 <- quantile(raw_data_frame$Loss, 0.75, na.rm = TRUE)
IQR_Value <- Q3 - Q1
IQR_outliers_before <- sum(raw_data_frame$Loss < (Q1 - 1.5 * IQR_Value) | raw_data_frame$Loss > (Q3 + 1.5 * IQR_Value), na.rm = TRUE)

cat("The Number of outliers for Loss (IQR method):", IQR_outliers_before, "\n")
```
```

```
The Number of outliers for Loss (IQR method): 20282
```

Figure 2.4: Checking the Number of Outliers For Loss

We can find loss outliers using IQR testing. It has a lot of outliers which is around 20,282.

```{r}
ggplot(raw_data_frame, aes(y = Loss)) +
  geom_boxplot() +
  labs(title = "Distribution of Loss")
theme_minimal()
```

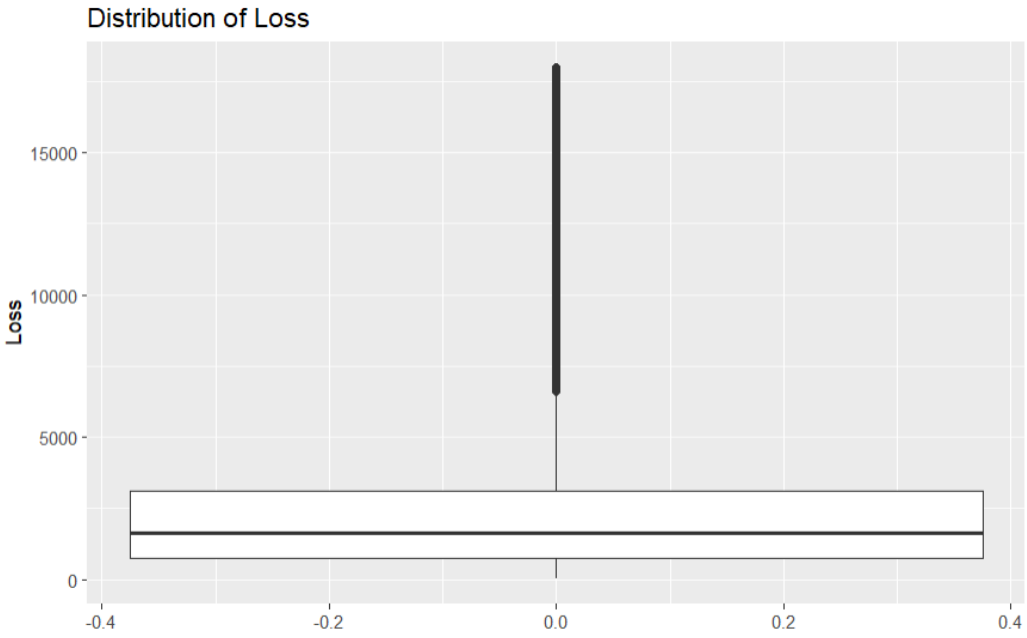


Figure 2.5: The Distribution of Loss using Boxplot

Figure 2.5 uses a boxplot to visualize the outliers present for loss.

```{r}
missing_Loss_percentage <- (sum(is.na(original_hacking_dataset$Loss)) / length(original_hacking_dataset$Loss)) * 100

cat("Missing Percentage: ", round(missing_Loss_percentage), "%")
```

```
Missing Percentage:  16 %
```

Figure 2.6: Checking Missing Value Percentage

We see that 16% of the data is missing from the loss column. So we need to impute our data.

To verify further whether we have to impute the loss column or not, we test for MCAR testing

```{r MCAR Test, echo=TRUE}
mcar_test(raw_data_frame)
```

A tibble: 1 × 4

statistic <dbl>	df <dbl>	p.value <dbl>	missing.patterns <int>
149947.9	32	0	4

1 row

Figure 2.7: MCAR Test

where the p-value is less significant as it is less than 0.5, then we fail to reject the null hypothesis which is that the missing data for the Loss column should be removed as p-value> 0.5. After test execution, the data is not MCAR positive as p-value < 0.5 so data should not be removed. The data should be imputed instead, using different methods.

We are using Random Forest as our best imputation method, to verify our methodology, we assume that there is no correlation with other variables present in the dataset with the loss column, and there is a weak/strong correlation between Date, Downtime, and Ransom.

First, we convert the Date into proper date format since it was in character format.

```{r}
raw_data_frame$Date <- as.Date(raw_data_frame$Date, format="%d/%m/%Y")
str(raw_data_frame)
```

```
'data.frame':   212093 obs. of  12 variables:
 $ Date     : Date, format: "1998-01-02" "1998-01-03" ...
 $ Notify   : chr  "Team CodeZero" "Feliz" "Optiklenz(LOU)" "Team CodeZero" ...
 $ URL      : chr  "http://www.janet-jackson.com" "http://cariari.ucr.ac.cr"
"http://marin.k12.ca.us" "http://www.dm.af.mil" ...
 $ IP       : chr  "" "" "" "" ...
 $ Country  : chr  "UNKNOWN" "COSTARICA" "UNITED STATES" "AFGHANISTAN" ...
 $ OS       : chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ WebServer: chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ Encoding : chr  "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Lang     : chr  "NULL" "NULL" "Helvetica" "NULL" ...
 $ Ransom   : int  NA NA 2034 NA NA 2316 195 NA 2877 1070 ...
 $ DownTime : int  18 31 35 58 59 34 21 37 28 21 ...
 $ Loss     : num  678 1988 12204 NA NA ...
```

Figure 2.8: Change Date format from character to Date

Second, we change the Date into numeric form and insert a new column in the dataset as Date_numeric for carrying out Spearman rank correlation testing.

```{r}
raw_data_frame$Date_numeric <- as.numeric(raw_data_frame$Date)
str(raw_data_frame)
```

```
'data.frame':   212093 obs. of  13 variables:
 $ Date        : Date, format: "1998-01-02" "1998-01-03" ...
 $ Notify      : chr  "Team CodeZero" "Feliz" "Optiklenz(LOU)" "Team CodeZero"
...
 $ URL         : chr  "http://www.janet-jackson.com" "http://cariari.ucr.ac.cr"
"http://marin.k12.ca.us" "http://www.dm.af.mil" ...
 $ IP          : chr  "" "" "" "" ...
 $ Country     : chr  "UNKNOWN" "COSTARICA" "UNITED STATES" "AFGHANISTAN" ...
 $ OS          : chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ WebServer   : chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ Encoding    : chr  "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Lang        : chr  "NULL" "NULL" "Helvetica" "NULL" ...
 $ Ransom      : int  NA NA 2034 NA NA 2316 195 NA 2877 1070 ...
 $ DownTime    : int  18 31 35 58 59 34 21 37 28 21 ...
 $ Loss        : num  678 1988 12204 NA NA ...
 $ Date_numeric: num  10228 10229 10230 10230 10230 ...
```

Figure 2.9 Convert Date into Date_numeric and add it as a new column

Third, we check the correlation between loss and date using the Spearman method

```{r}
Loss_Date <- cor(raw_data_frame$Date_numeric, raw_data_frame$Loss, method =
"spearman", use = "complete.obs")
Loss_Date
```

```
[1] -0.001622567
```

Figure 2.10 Correlation between Loss and Date

Fourth, we check the correlation between loss and downtime using the Spearman method

```{r}
Loss_Downtime <- cor(raw_data_frame$DownTime, raw_data_frame$Loss, method =
"spearman", use = "complete.obs")
Loss_Downtime
```

```
[1] 0.7457765
```

Figure 2.11 Correlation between Loss and Downtime

Fifth, we check the correlation between loss and ransom using Spearman method

```r
Loss_Ransom <- cor(raw_data_frame$Ransom, raw_data_frame$Loss, method =
"spearman", use = "complete.obs")
Loss_Ransom
```

```
[1] 0.6479824
```

Figure 2.12 Correlation between Loss And Ransom

Sixth, we check the missing value of the ransom

```r
missing_values_percentage <- (sum(is.na(original_hacking_dataset$Ransom)) /
length(original_hacking_dataset$Loss)) * 10
missing_values_percentage
```

```
[1] 75.4386
```

Figure 2.13 Checking the missing value of the ransom

We see downtime and ransom have a strong correlation and Date is weakly correlated with Loss. But since ransom data is missing more than 70%. We take Date and downtime to be the best for carrying out the Random Forest imputation technique

```r
data_rf <- missForest(raw_data_frame[, c("Loss", "DownTime",
"Date_numeric")])
summary(data_rf$ximp$Loss)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
     30     804    1904    2763    3014   18000
```

Figure 2.14 The output after Random Forrest Imputation

Then, we tested to see whether it shows the data was imputed with an acceptable error range using “OOB Error Estimate” (GeeksforGeeks, 2023). OOB(Out of Bag) Error Estimate is a score that evaluates the performance of ensemble models by using data that was not included in the training, (GeeksforGeeks, 2023). It provides an unbiased assessment of the model’s accuracy, (GeeksforGeeks, 2023)

```r
print("OOB Error Estimate:")
print(data_rf$OOBerror)
```

```
[1] "OOB Error Estimate:"
    NRMSE
0.2680968
```

Figure 2.15 Checking the error for Imputation

Putting the imputed Loss to the original Loss Data

```r
raw_data_frame$Loss <- data_rf$ximp$Loss
summary(raw_data_frame$Loss)
```

Figure 2.16 Place the imputed data to the original loss data

Putting the imputed data into a different vector for better accessibility

```r
impute_data <- raw_data_frame
str(impute_data)
```

```
'data.frame':   212093 obs. of  13 variables:
 $ Date        : Date, format: "1998-01-02" ...
 $ Notify      : chr  "Team CodeZero" "Feliz" "Optiklenz(LOU)"
"Team CodeZero" ...
 $ URL         : chr  "http://www.janet-jackson.com"
"http://cariari.ucr.ac.cr" "http://marin.k12.ca.us"
"http://www.dm.af.mil" ...
 $ IP          : chr  "" "" "" "" ...
 $ Country     : chr  "UNKNOWN" "COSTARICA" "UNITED STATES"
"AFGHANISTAN" ...
 $ OS          : chr  "Unknown" "Unknown" "Unknown" "Unknown"
...
 $ WebServer   : chr  "Unknown" "Unknown" "Unknown" "Unknown"
...
 $ Encoding    : chr  "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Lang        : chr  "NULL" "NULL" "Helvetica" "NULL" ...
 $ Ransom      : int  NA NA 2034 NA NA 2316 195 NA 2877 1070
...
 $ DownTime    : int  18 31 35 58 59 34 21 37 28 21 ...
 $ Loss        : num  678 1988 12204 6076 6076 ...
 $ Date_numeric: num  10228 10229 10230 10230 10230 ...
```

Figure 2.17 Making the imputed data better readability

Removing Date_numeric from imputed data as it is not needed in our analysis.

```{r}
impute_data$Date_numeric <- NULL
str(impute_data)
```

```
'data.frame':   212093 obs. of  12 variables:
 $ Date      : Date, format: "1998-01-02" ...
 $ Notify    : chr  "Team CodeZero" "Feliz" "Optiklenz(LOU)"
"Team CodeZero" ...
 $ URL       : chr  "http://www.janet-jackson.com"
"http://cariari.ucr.ac.cr" "http://marin.k12.ca.us"
"http://www.dm.af.mil" ...
 $ IP        : chr  "" "" "" "" ...
 $ Country   : chr  "UNKNOWN" "COSTARICA" "UNITED STATES"
"AFGHANISTAN" ...
 $ OS        : chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ WebServer: chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ Encoding : chr  "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Lang      : chr  "NULL" "NULL" "Helvetica" "NULL" ...
 $ Ransom    : int  NA NA 2034 NA NA 2316 195 NA 2877 1070 ...
 $ DownTime : int  18 31 35 58 59 34 21 37 28 21 ...
 $ Loss      : num  678 1988 12204 6076 6076 ...
```

Figure 2.18 Calculating numbers of outliers after imputation

```{r}
Q1 <- quantile(impute_data$Loss, 0.25, na.rm = TRUE)
Q3 <- quantile(impute_data$Loss, 0.75, na.rm = TRUE)
IQR_Value <- Q3 - Q1
IQR_outliers <- sum(impute_data$Loss < (Q1 - 1.5 * IQR_Value) |
impute_data$Loss > (Q3 + 1.5 * IQR_Value), na.rm = TRUE)

cat("The Number of outliers for Loss (IQR method):",
IQR_outliers, "\n")
```

```
The Number of outliers for Loss (IQR method): 20872
```

Figure 2.19 Outliers Calculation

Visualization of Outliers and skewness after Imputation

```{r}
ggplot(impute_data, aes(y = Loss)) +
  geom_boxplot() +
  labs(title = "Distribution of Loss After Imputation")
theme_minimal()
```

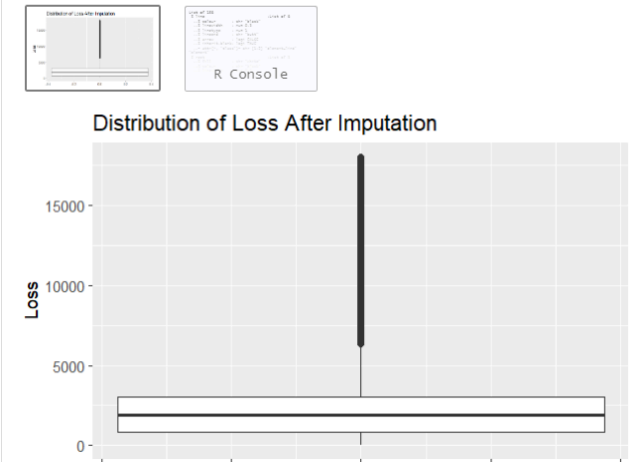


Figure 2.20 Outliers through boxplot after imputation

We can see that the outliers did increase from 20,870 to 20,282 (590). Even though it increased, it is not that significant. We can visualize it to see how much it changed for Figure 2.5. There were no significant changes seen in both of the figure 2.20 and 2.5

```r
p1 ← ggplot(impute_data, aes(x = Loss)) +
  geom_histogram(aes(y = ..density..), bins = 30, fill = "lightblue", color = "black") +
  geom_density(color = "orange", linewidth = 1) +
  labs(title = "Distribution of Loss After Imputation Of Random Forest",
       subtitle = paste("Skewness:", round(skewness(raw_data_frame$Loss, na.rm = TRUE), 3)),
       x = "Loss",
       y = "Density") +
  theme_minimal()
print(p1)
```

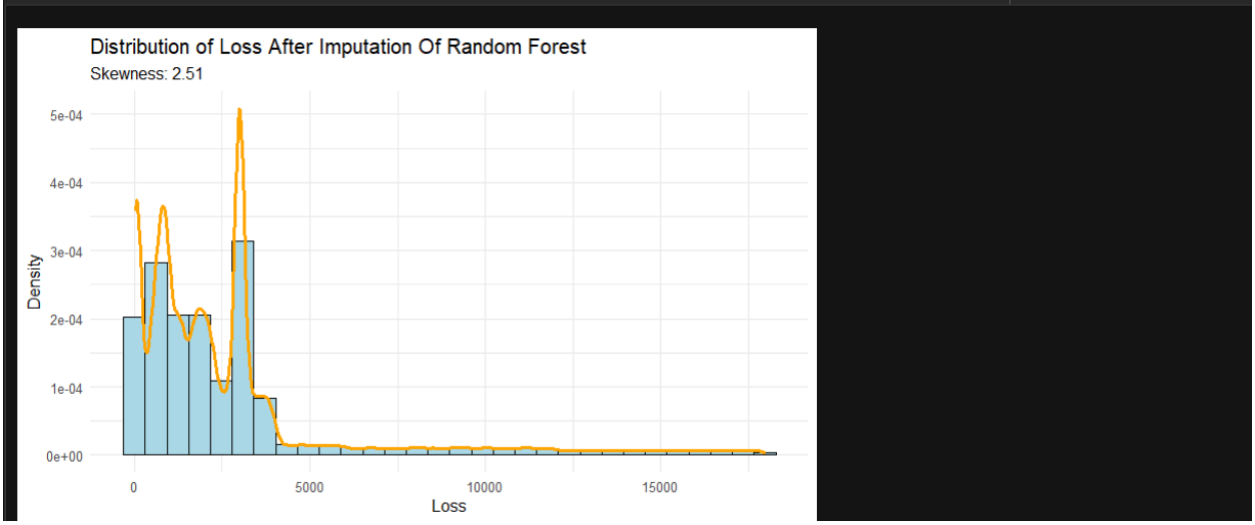


Figure 2.21 Skewness after Imputation

In Figure 2.21, we check for skewness again. It is seen from Figure 2.3, that the skewness changed from 2.331 to 2.51 with an increase of (2.51-2.331) of 0.179. The skewness also showed that the random imputation was imputated perfectly without significantly changing the distribution of the data for Loss column.

```
data_mean ← original_hacking_dataset
data_mean$Loss[is.na(data_mean$Loss)] ← mean(data_mean$Loss, na.rm = TRUE)

data_median ← original_hacking_dataset
data_median$Loss[is.na(data_median$Loss)] ← median(data_median$Loss, na.rm = TRUE)

cat("Skewness Comparison:\n",
    "1. Original data (excluding NA): ", skewness(original_hacking_dataset$Loss, na.rm = TRUE), "\n",
    "2. Mean imputation: ", skewness(data_mean$Loss), "\n",
    "3. Median imputation: ", skewness(data_median$Loss), "\n",
    "4. After Random Forest imputation: ", skewness(data_rf$ximp$Loss), "\n")
```

```
Skewness Comparison:
 1. Original data (excluding NA):  2.331037
 2. Mean imputation:  2.542172
 3. Median imputation:  2.6401
 4. After Random Forest imputation:  2.510275
```

Figure 2.22 Summary of the skewness value

We also compared with mean, and median imputation compared to the random forest imputation. We see that random forest had a closer skewness to the original skewness. This concludes that random forest imputation is best for the Loss column.

2.2.2 Independent Variables

- **Country**

```
view(country_raw$Country)
```

	x
1	UNKNOWN
2	COSTARICA
3	UNITED STATES
4	AFGHANISTAN
5	AFGHANISTAN
6	UNKNOWN
7	ANDORRA
8	NA
9	UNKNOWN
10	UNKNOWN
11	UNKNOWN
12	NA
13	UNKNOWN
14	UNKNOWN
15	UNKNOWN
16	UNKNOWN
17	UNKNOWN
18	NA
19	UNKNOWN
20	UNKNOWN
21	NA
22	UNKNOWN
23	UNKNOWN
24	SWEDEN
25	UNKNOWN
26	UNKNOWN
27	NETHERLANDS
28	UNKNOWN
29	UNKNOWN
30	UNKNOWN
31	UNKNOWN
32	UNITED KINGDOM

Figure 3.0 View Country Rows

Checking out the data for the “Country” column

We view the first 1000 rows of data for the country column. The country names from the country column have uppercase and “NA” missing values.

Since the data is not unique and it has repeated country names.

```
unique(country_raw$Country)
```

```
  [1] "UNKNOWN"                "COSTARICA"              "UNITED STATES"
  [4] "AFGHANISTAN"            "ANDORRA"                "NA"
  [7] "SWEDEN"                 "NETHERLANDS"            "UNITED KINGDOM"
 [10] "INDIA"                  "MEXICO"                 "ITALY"
 [13] "THAILAND"               "JAPAN"                  "BRAZIL"
 [16] "AUSTRALIA"              "CHINA"                  "MALAYSIA"
 [19] "RUSSIAN FEDERATION"     "GERMANY"                "SOUTH AFRICA"
 [22] "PAKISTAN"               "CZECH REPUBLIC"         "ARGENTINA"
 [25] "BELGIUM"                "DENMARK"                "ROMANIA"
 [28] "COLOMBIA"               "FRANCE"                 "CANADA"
 [31] "HUNGARY"                "NIUE"                   "TONGA"
 [34] "SINGAPORE"              "AUSTRIA"                "SLOVENIA"
 [37] "INDONESIA"              "SOUTH KOREA"            "PORTUGAL"
 [40] "BOLIVIA"                "TAIWAN"                 "HONG KONG"
 [43] "YUGOSLAVIA"             "NORWAY"                 "MONTENEGRO"
 [46] "SWITZERLAND"            "PHILIPPINES"            "PANAMA"
 [49] "NEWZEALAND"             "PERU"                   "IRAN"
 [52] "HONDURAS"               "NIGER"                  "NICARAGUA"
 [55] "TURKEY"                 "SAUDIARABIA"            "VIRGINISLANDS(BRITISH)"
 [58] "IRELAND"                "AMERICANSAMOA"          "KUWAIT"
 [61] "CROATIA"                "CHILE"                  "SPAIN"
 [64] "CUBA"                   "BURKINAFASO"            "VENEZUELA"
 [67] "LESOTHO"                "FINLAND"                "ISRAEL"
 [70] "PUERTORICO"             "GEORGIA"                "MACEDONIA"
 [73] "GREECE"                 "URUGUAY"                "ERITREA"
 [76] "HAITI"                  "QATAR"                  "LUXEMBOURG"
 [79] "SRILANKA"               "MONGOLIA"               "UKRAINE"
 [82] "BULGARIA"               "MOROCCO"                "SLOVAKIA"
 [85] "GHANA"                  "ESTONIA"                "GUAM"
 [88] "ALBANIA"                "PARAGUAY"               "ECUADOR"
 [91] "POLAND"                 "MOLDOVA"                "SURINAME"
[187] "United State"           "United Kingd"           "Germany"
[190] "Indonesia"              "Russian Fede"           "Sweden"
[193] "Malaysia"               "Brazil"                 "France"
[196] "Viet Nam"               "Australia"              "China"
[199] "Portugal"               "Denmark"                "Syrian Arab"
[202] "Singapore"              "Bulgaria"               "Thailand"
[205] "Switzerland"            "Netherlands\""          "Italy"
[208] "Poland"                 "Canada"                 "Ukraine"
[211] "Belarus"                "Japan"                  "Hong Kong"
[214] "Israel"                 "Turkey"                 "Czech Republ"
[217] "India"                  "Ireland"                "Pakistan"
[220] "Bolivia"                "Costa Rica"             "Chile"
[223] "Hungary"                "Mexico"                 "New Zealand\""
[226] "Georgia"                "Taiwan, Prov"           "Philippines"
[229] "Colombia"               "Kazakhstan"             "Macedonia, T"
[232] "Lithuania"              "Belgium"                "Oman"
[235] "Spain"                  "Argentina"              "Moldova, Rep"
[238] "Rwanda"                 "Austria"                "Croatia"
[241] "Iceland"                "Korea, Repub"           "Greece"
[244] "Slovenia"               "Virgin Islan"           "Mongolia"
[247] "Romania"                "Nigeria\""              "Iran, Islami"
[250] "European Uni"           "South Africa"           "French Polyn"
[253] "Puerto Rico"            "Armenia"                "Cayman Islan"
[256] "Serbia"                 "Slovakia"               "New Caledoni"
[259] "United States"          "Russian Federation"     "Netherlands"
[262] "United Kingdom"         "Morocco"                "Sri Lanka"
[265] "Kyrgyzstan"             "Saudi Arabia"           "French Polynesia"
[268] "European Union"         "Ecuador"                "Czech Republic"
[271] "Bhutan"                 "El Salvador"            "Egypt"
[274] "Cyprus"                 "Iran"                   "Virgin Islands"
[277] "Cayman Islands"         "Taiwan"                 "Senegal"
[280] "Algeria"                "Korea"                  "Nepal"
[283] "Tunisia"                "New Zealand"            "Azerbaijan"
```

Figure 3.1 Checking unique columns of data for the “Country” column

We use this code to inspect what is wrong with the data. We can see some country names that have uppercase and some of them have misspellings as well. It also has "\" in some of the countries, one example would be "Nigeria\". This must be taken care of before proceeding with the analysis question.

```
[283] "Tunisia"                  "New Zealand"                "Azerbaijan"
[286] "Panama"                   "Macedonia"                  "Norway"
[289] "Finland"                  "Bangladesh"                 "Bosnia and Herzegovina"
[292] "Latvia"                   "Uruguay"                    "Mozambique"
[295] "Syrian Arab Republic"     "Paraguay"                   "Nicaragua"
[298] "Luxembourg"               "Satellite Provider"         "Estonia"
[301] "Moldova"                  "Albania"                    "Uzbekistan"
[304] "Togo"                     "New Caledonia"              "Yemen"
[307] "Lebanon"                  "Peru"                       "Brunei Darussalam"
[310] "Kenya"                    "Maldives"                   "Botswana"
[313] "Honduras"                 "Libyan Arab Jamahiriya"     "Iraq"
[316] "Dominican Republic"       "Nigeria"                    "Cuba"
[319] "Kuwait"                   "Venezuela"                  "United Arab Emirates"
[322] "Sudan"                    "Zambia"                     "Cameroon"
[325] "Zimbabwe"                 "Jordan"                     "Niger"
[328] "Madagascar"               "Namibia"                    "Unknown"
[331] "America"                  "WestEuro"                   "EastEuro"
[334] "SouthAmerica"             "Asia"                       "Palestinian Territory"
[337] "Oseania"                  "MiddleEast"                 "Tajikistan"
[340] "Africa"                   "Jamaica"                    "Ethiopia"
[343] "Mali"                     "Tanzania"                   "Anonymous Proxy"
[346] "Cote D'Ivoire"            "Mauritius"                  "Burkina Faso"
[349] "Vanuatu"                  "Samoa"                      "Malawi"
[352] "Uganda"                   "Ghana"                      "Cook Islands"
[355] "Cape Verde"               "Cambodia"                   "Lao People's Democratic Republic"
```

```r
country_clean ← country_raw %>%
  mutate(Country = str_to_title(str_trim(Country)))

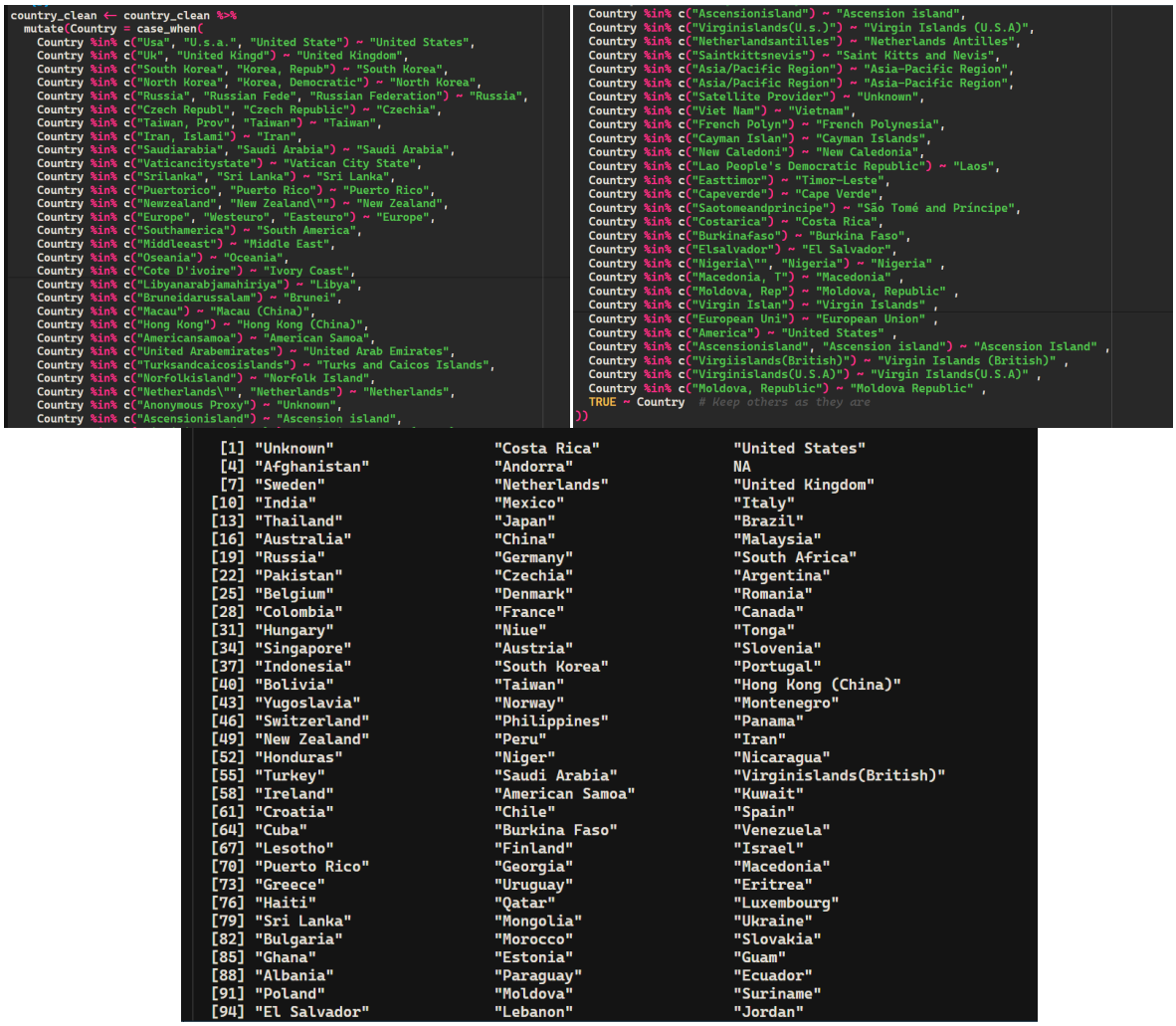
# Checking Country Column
unique(country_clean$Country)
```

```
 [1] "Unknown"                    "Costarica"
 [3] "United States"              "Afghanistan"
 [5] "Andorra"                    "NA"
 [7] "Sweden"                     "Netherlands"
 [9] "United Kingdom"             "India"
[11] "Mexico"                     "Italy"
[13] "Thailand"                   "Japan"
[15] "Brazil"                     "Australia"
[17] "China"                      "Malaysia"
[19] "Russian Federation"         "Germany"
[21] "South Africa"               "Pakistan"
[23] "Czech Republic"             "Argentina"
[25] "Belgium"                    "Denmark"
[27] "Romania"                    "Colombia"
[29] "France"                     "Canada"
[31] "Hungary"                    "Niue"
[33] "Tonga"                      "Singapore"
[35] "Austria"                    "Slovenia"
[37] "Indonesia"                  "South Korea"
[39] "Portugal"                   "Bolivia"
[41] "Taiwan"                     "Hong Kong"
[43] "Yugoslavia"                 "Norway"
[45] "Montenegro"                 "Switzerland"
[47] "Philippines"                "Panama"
[49] "Newzealand"                 "Peru"
[51] "Iran"                       "Honduras"
[53] "Niger"                      "Nicaragua"
[55] "Turkey"                     "Saudiarabia"
[57] "Virginislands(British)"     "Ireland"
[59] "Americansamoa"              "Kuwait"
[61] "Croatia"                    "Chile"
[63] "Spain"                      "Cuba"
[65] "Burkinafaso"                "Venezuela"
[67] "Lesotho"                    "Finland"
[69] "Israel"                     "Puertorico"
[71] "Georgia"                    "Macedonia"
[73] "Greece"                     "Uruguay"
[75] "Eritrea"                    "Haiti"
[77] "Qatar"                      "Luxembourg"
[79] "Srilanka"                   "Mongolia"
[81] "Ukraine"                    "Bulgaria"
[83] "Morocco"                    "Slovakia"
[85] "Ghana"                      "Estonia"
[87] "Guam"                       "Albania"
[89] "Paraguay"                   "Ecuador"
[91] "Poland"                     "Moldova"
[93] "Suriname"                   "Elsalvador"
[95] "Lebanon"                    "Jordan"
[97] "Northern Mariana Islands"   "Egypt"
[99] "Tuvalu"                     "Bosnia And Herzegovina"
101] "Gabon"                      "United Arabemirates"
103] "Barbados"                   "Lithuania"
105] "Liechtenstein"              "Nepal"
107] "Togo"                       "Zambia"
109] "Guinea"                     "Vaticancitystate"
111] "Armenia"                    "Turksandcaicosislands"
113] "Mauritius"                  "Mozambique"
115] "Bhutan"                     "Iceland"
117] "Macau"                      "Bahamas"
119] "Ascensionisland"            "Algeria"
121] "Caledonia"                  "Oman"
123] "Bahrain"                    "Latvia"
125] "Kenya"                      "Seychelles"
127] "Samoa"                      "Bermuda"
129] "Norfolkisland"              "Iraq"
[129] "Norfolkisland"             "Iraq"
[131] "Trinidad And Tobago"       "Libyanarabjamahiriya"
[133] "Antigua And Barbuda"       "Jamaica"
[135] "Malta"                     "San Marino"
[137] "Senegal"                   "Burundi"
[139] "Monaco"                    "Kazakhstan"
[141] "Botswana"                  "Cyprus"
[143] "Montserrat"                "Tunisia"
[145] "Zimbabwe"                  "Belarus"
[147] "Belize"                    "Bruneidarussalam"
[149] "Cameroon"                  "Dominican Republic"
[151] "Bangladesh"                "Greenland"
[153] "Congo"                     "Virginislands(U.s.)"
[155] "Faroe Islands"             "Namibia"
[157] "Kyrgyzstan"                "Micronesia"
[159] "Fiji"                      "Easttimor"
[161] "Capeverde"                 "Azerbaijan"
[163] "Tajikistan"                "Saotomeandprincipe"
[165] "Turkmenistan"              "Madagascar"
[167] "Netherlandsantilles"       "Benin"
[169] "Saintkittsnevis"           "Europe"
[171] "Somalia"                   "Guatemala"
[173] "Vietnam"                   "Mauritania"
[175] "Serbia"                    "Sri Lanka"
[177] "Guernsey"                  "United Arab Emirates"
[179] "Cambodia"                  "Asia/Pacific Region"
[181] "Anonymous Proxy"           "Satellite Provider"
[183] "Syrian Arab Republic"      "Palestinian Territory"
[185] "Papua New Guinea"          "Rwanda"
[187] "United State"              "United Kingd"
[189] "Russian Fede"              "Viet Nam"
[191] "Syrian Arab"               "Netherlands\""
[193] "Czech Republ"              "Costa Rica"
```

Figure 3.2 Formatting the country names in the "Country" column

First, we standardize the names to capital, where we transform the strings from uppercase letters. We also remove duplicate unique country names such as “UNITED STATES”, “United States” to “United States” and so on.

Then, we format the country names to its proper spelling



```
country_clean <- country_clean %>%
  mutate(Country = case_when(
    Country %in% c("Usa", "U.s.a.", "United State") ~ "United States",
    Country %in% c("Uk", "United Kingd") ~ "United Kingdom",
    Country %in% c("South Korea", "Korea, Repub") ~ "South Korea",
    Country %in% c("North Korea", "Korea, Democratic") ~ "North Korea",
    Country %in% c("Russia", "Russian Fede", "Russian Federation") ~ "Russia",
    Country %in% c("Czech Republ", "Czech Republic") ~ "Czechia",
    Country %in% c("Taiwan, Prov", "Taiwan") ~ "Taiwan",
    Country %in% c("Iran, Islami") ~ "Iran",
    Country %in% c("Saudiarabia", "Saudi Arabia") ~ "Saudi Arabia",
    Country %in% c("Vaticancitystate") ~ "Vatican City State",
    Country %in% c("Srilanka", "Sri Lanka") ~ "Sri Lanka",
    Country %in% c("Puertorico", "Puerto Rico") ~ "Puerto Rico",
    Country %in% c("Newzealand", "New Zealand\"") ~ "New Zealand",
    Country %in% c("Europe", "Westeuro", "Easteuro") ~ "Europe",
    Country %in% c("Southamerica") ~ "South America",
    Country %in% c("Middleeast") ~ "Middle East",
    Country %in% c("Oseania") ~ "Oceania",
    Country %in% c("Cote D'ivoire") ~ "Ivory Coast",
    Country %in% c("Libyanarabjamahiriya") ~ "Libya",
    Country %in% c("Bruneidarussalam") ~ "Brunei",
    Country %in% c("Macau") ~ "Macau (China)",
    Country %in% c("Hong Kong") ~ "Hong Kong (China)",
    Country %in% c("Americansamoa") ~ "American Samoa",
    Country %in% c("United Arabemirates") ~ "United Arab Emirates",
    Country %in% c("Turksandcaicosislands") ~ "Turks and Caicos Islands",
    Country %in% c("Norfolkisland") ~ "Norfolk Island",
    Country %in% c("Netherlands\"", "Netherlands") ~ "Netherlands",
    Country %in% c("Anonymous Proxy") ~ "Unknown",
    Country %in% c("Ascensionisland") ~ "Ascension island",
    Country %in% c("Ascensionisland") ~ "Ascension island",
    Country %in% c("Virginislands(U.s.)") ~ "Virgin Islands (U.S.A)",
    Country %in% c("Netherlandsantilles") ~ "Netherlands Antilles",
    Country %in% c("Saintkittsnevis") ~ "Saint Kitts and Nevis",
    Country %in% c("Asia/Pacific Region") ~ "Asia-Pacific Region",
    Country %in% c("Asia/Pacific Region") ~ "Asia-Pacific Region",
    Country %in% c("Satellite Provider") ~ "Unknown",
    Country %in% c("Viet Nam") ~ "Vietnam",
    Country %in% c("French Polyn") ~ "French Polynesia",
    Country %in% c("Cayman Islan") ~ "Cayman Islands",
    Country %in% c("New Caledoni") ~ "New Caledonia",
    Country %in% c("Lao People's Democratic Republic") ~ "Laos",
    Country %in% c("Easttimor") ~ "Timor-Leste",
    Country %in% c("Capeverde") ~ "Cape Verde",
    Country %in% c("Saotomeandprincipe") ~ "São Tomé and Príncipe",
    Country %in% c("Costarica") ~ "Costa Rica",
    Country %in% c("Burkinafaso") ~ "Burkina Faso",
    Country %in% c("Elsalvador") ~ "El Salvador",
    Country %in% c("Nigeria\"", "Nigeria") ~ "Nigeria" ,
    Country %in% c("Macedonia, T") ~ "Macedonia" ,
    Country %in% c("Moldova, Rep") ~ "Moldova, Republic" ,
    Country %in% c("Virgin Islan") ~ "Virgin Islands" ,
    Country %in% c("European Uni") ~ "European Union" ,
    Country %in% c("America") ~ "United States" ,
    Country %in% c("Ascensionisland", "Ascension island") ~ "Ascension Island" ,
    Country %in% c("Virgiislands(British)") ~ "Virgin Islands (British)" ,
    Country %in% c("Virginislands(U.S.A)") ~ "Virgin Islands(U.S.A)" ,
    Country %in% c("Moldova, Republic") ~ "Moldova Republic" ,
    TRUE ~ Country  # Keep others as they are
))
```

```
 [1] "Unknown"          "Costa Rica"           "United States"
 [4] "Afghanistan"      "Andorra"              NA
 [7] "Sweden"           "Netherlands"          "United Kingdom"
[10] "India"            "Mexico"               "Italy"
[13] "Thailand"         "Japan"                "Brazil"
[16] "Australia"        "China"                "Malaysia"
[19] "Russia"           "Germany"              "South Africa"
[22] "Pakistan"         "Czechia"              "Argentina"
[25] "Belgium"          "Denmark"              "Romania"
[28] "Colombia"         "France"               "Canada"
[31] "Hungary"          "Niue"                 "Tonga"
[34] "Singapore"        "Austria"              "Slovenia"
[37] "Indonesia"        "South Korea"          "Portugal"
[40] "Bolivia"          "Taiwan"               "Hong Kong (China)"
[43] "Yugoslavia"       "Norway"               "Montenegro"
[46] "Switzerland"      "Philippines"          "Panama"
[49] "New Zealand"      "Peru"                 "Iran"
[52] "Honduras"         "Niger"                "Nicaragua"
[55] "Turkey"           "Saudi Arabia"         "Virginislands(British)"
[58] "Ireland"          "American Samoa"       "Kuwait"
[61] "Croatia"          "Chile"                "Spain"
[64] "Cuba"             "Burkina Faso"         "Venezuela"
[67] "Lesotho"          "Finland"              "Israel"
[70] "Puerto Rico"      "Georgia"              "Macedonia"
[73] "Greece"           "Uruguay"              "Eritrea"
[76] "Haiti"            "Qatar"                "Luxembourg"
[79] "Sri Lanka"        "Mongolia"             "Ukraine"
[82] "Bulgaria"         "Morocco"              "Slovakia"
[85] "Ghana"            "Estonia"              "Guam"
[88] "Albania"          "Paraguay"             "Ecuador"
[91] "Poland"           "Moldova"              "Suriname"
[94] "El Salvador"      "Lebanon"              "Jordan"
```

Figure 3.3: Correcting misspelling of country names in the “Country” column.

All of the countries are in proper spelling. It is ready for imputation of missing values.

Before imputation, we check for IP addresses to determine whether it is in the correct IPV4 and IPV6 format. Since we are using the Maxmind IP database which is known as GeoLite2-Country.mmdb, (*MaxMind*, n.d.). The database will be used to impute country columns using library rgeolocate.

```
unique(country_clean$IP)
```

```
  [1] ""                "137.138.142.239" "127.0.0.1"       "209.204.238.62"  "64.33.66.222"
  [6] "208.55.144.148"  "217.150.50.40"   "200.193.55.4"    "200.202.124.84"  "211.0.14.242"
 [11] "209.208.134.68"  "199.185.234.209" "210.164.250.50"  "194.85.152.141"  "142.165.199.91"
 [16] "216.189.192.25"  "216.149.29.133"  "195.224.187.61"  "129.116.227.93"  "63.83.236.34"
 [21] "207.13.246.70"   "209.240.243.42"  "210.225.110.1"   "66.40.229.252"   "129.119.114.5"
 [26] "161.58.230.51"   "207.198.74.130"  "216.219.157.243" "207.198.78.84"   "64.21.89.48"
 [31] "200.201.133.25"  "200.252.123.180" "64.21.89.45"     "195.226.96.138"  "199.231.144.40"
 [36] "194.219.23.249"  "64.21.89.35"     "216.15.230.141"  "66.201.198.5"    "64.21.89.43"
 [41] "12.224.18.195"   "211.101.141.151" "207.211.45.176"  "200.222.67.220"  "206.203.45.131"
 [46] "200.222.67.206"  "66.75.234.216"   "206.222.212.17"  "216.38.207.73"   "64.225.254.71"
 [51] "206.117.10.184"  "216.149.29.179"  "216.149.29.193"  "210.12.218.213"  "216.157.71.142"
 [56] "199.231.144.47"  "64.156.139.221"  "194.219.23.147"  "208.16.155.203"  "194.219.23.161"
 [61] "194.219.23.62"   "194.219.23.78"   "203.199.143.3"   "200.214.44.181"  "211.105.9.79"
 [66] "209.35.45.231"   "208.133.93.142"  "64.239.44.18"    "207.188.221.27"  "64.21.89.115"
 [71] "143.106.105.2"   "64.21.89.106"    "200.204.104.166" "212.217.47.150"  "195.33.64.95"
 [76] "200.204.146.135" "165.132.130.116" "211.202.1.62"    "211.42.44.3"     "131.236.68.20"
 [81] "207.249.1.2"     "202.106.155.45"  "195.14.149.21"   "200.135.231.222" "194.234.202.17"
 [86] "131.107.65.37"   "193.28.197.254"  "217.115.142.66"  "161.58.201.130"  "210.56.11.123"
 [91] "203.101.137.60"  "216.169.108.12"  "211.22.141.250"  "200.160.244.17"  "142.165.199.140"
 [96] "205.252.62.18"   "66.78.50.253"    "211.21.96.118"   "202.155.113.14"  "211.21.79.46"
[101] "24.232.76.246"   "200.16.169.49"   "211.105.66.107"  "163.152.33.185"  "216.71.31.148"
[106] "66.96.229.199"   "212.227.118.195" "64.251.95.2"     "200.248.222.139" "161.58.224.72"
[111] "140.239.15.52"   "216.156.252.15"  "216.247.174.47"  "216.13.209.147"  "216.149.28.83"
[116] "209.209.186.183" "213.82.162.86"   "128.194.164.42"  "216.22.200.37"   "193.130.128.3"
[121] "194.129.106.6"   "200.28.152.130"  "65.114.67.4"     "207.1.3.205"     "63.73.65.229"
[126] "193.220.34.235"  "139.95.253.11"   "205.252.62.135"  "210.80.150.2"    "64.65.56.131"
[131] "211.23.14.195"   "210.171.113.194" "200.131.253.2"   "140.111.133.193" "211.38.42.186"
[136] "211.41.56.172"   "63.119.175.13"   "211.20.120.203"  "211.226.16.4"    "209.238.94.227"
[141] "200.23.171.5"    "211.42.174.10"   "211.53.209.206"  "212.45.5.227"    "211.47.67.247"
[146] "200.225.212.42"  "202.64.182.67"   "62.213.1.125"    "195.13.14.5"     "64.176.169.229"
[151] "216.156.152.128" "210.190.52.194"  "210.163.169.56"  "213.208.134.46"  "195.5.220.70"
```

Figure 3.4: Using view() to check for if IP addresses are in the correct form or not.

We can see there are some empty strings present. The IP is not correctly formatted to Ipv4 and Ipv6.

Then we impute “Unknowns” to each missing row in the column IP.

```
country_clean$IP[country_clean$IP == ""] <- "Unknown"
country_clean$Country[is.na(country_clean$Country)] <- "Unknown"

# Counting "Unknown" in country column
sum(country_clean$Country == "Unknown")
```

```
[1] 67311
```

Figure 3.5: Imputing “Unknowns” to missing columns and checking how many missing columns are there.

We see there are around 67,311 rows of missing values. Then we format the IPs to its proper Ipv4 and Ipv6 format

```r
# Remove NA values in the 'IP' column
country_clean <- country_clean %>% filter(!is.na(IP))

# Trim whitespace
country_clean$IP <- str_trim(country_clean$IP)

# Remove invalid characters
country_clean$IP <- str_replace_all(country_clean$IP, "[^0-9.:a-fA-F]", "")

# Define regex patterns for IPv4 and IPv6
ipv4_pattern <- "^\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}\\.\\d{1,3}$"
ipv6_pattern <- "^[a-fA-F0-9]{1,4}(:[a-fA-F0-9]{1,4}){7}$"

# Validate IP addresses
country_clean$Valid_IP <- str_detect(country_clean$IP, ipv4_pattern) | str_detect(country_clean$IP,
ipv6_pattern)

# Extract invalid IPs
invalid_ips <- country_clean %>% filter(!Valid_IP) %>% select(IP)

# Replace invalid IPs with "0.0.0.0"
country_clean$IP[!country_clean$Valid_IP] <- "0.0.0.0"

# Seeing newly formatted IP addresses
unique(country_clean$IP)

# Checking all column
summary(country_clean)

# Dropping "Valid IP" from the dataset
country_clean$Valid_IP <- NULL
```

```r
# Viewing the dataset
view(country_clean)
```

```
 [1] "0.0.0.0"         "137.138.142.239" "127.0.0.1"       "209.204.238.62"  "64.33.66.222"
 [6] "208.55.144.148"  "217.150.50.40"   "200.193.55.4"    "200.202.124.84"  "211.0.14.242"
[11] "209.208.134.68"  "199.185.234.209" "210.164.250.50"  "194.85.152.141"  "142.165.199.91"
[16] "216.189.192.25"  "216.149.29.133"  "195.224.187.61"  "129.116.227.93"  "63.83.236.34"
[21] "207.13.246.70"   "209.240.243.42"  "210.225.110.1"   "66.40.229.252"   "129.119.114.5"
[26] "161.58.230.51"   "207.198.74.130"  "216.219.157.243" "207.198.78.84"   "64.21.89.48"
[31] "200.201.133.25"  "200.252.123.180" "64.21.89.45"     "195.226.96.138"  "199.231.144.40"
[36] "194.219.23.249"  "64.21.89.35"     "216.15.230.141"  "66.201.198.5"    "64.21.89.43"
[41] "12.224.18.195"   "211.101.141.151" "207.211.45.176"  "200.222.67.220"  "206.203.45.131"
[46] "200.222.67.206"  "66.75.234.216"   "206.222.212.17"  "216.38.207.73"   "64.225.254.71"
[51] "206.117.10.184"  "216.149.29.179"  "216.149.29.193"  "210.12.218.213"  "216.157.71.142"
[56] "199.231.144.47"  "64.156.139.221"  "194.219.23.147"  "208.16.155.203"  "194.219.23.161"
[61] "194.219.23.62"   "194.219.23.78"   "203.199.143.3"   "200.214.44.181"  "211.105.9.79"
[66] "209.35.45.231"   "208.133.93.142"  "64.239.44.18"    "207.188.221.27"  "64.21.89.115"
[71] "143.106.105.2"   "64.21.89.106"    "200.204.104.166" "212.217.47.150"  "195.33.64.95"
[76] "200.204.146.135" "165.132.130.116" "211.202.1.62"     "211.42.44.3"     "131.236.68.20"
[81] "207.249.1.2"     "202.106.155.45"  "195.14.149.21"   "200.135.231.222" "194.234.202.17"
[86] "131.107.65.37"   "193.28.197.254"  "217.115.142.66"  "161.58.201.130"  "210.56.11.123"
[91] "203.101.137.60"  "216.169.108.12"  "211.22.141.250"  "200.160.244.17"  "142.165.199.140"
[96] "205.252.62.18"   "66.78.50.253"    "211.21.96.118"   "202.155.113.14"  "211.21.79.46"
[101] "24.232.76.246"  "200.16.169.49"   "211.105.66.107"  "163.152.33.185"  "216.71.31.148"
[106] "66.96.229.199"  "212.227.118.195" "64.251.95.2"     "200.248.222.139" "161.58.224.72"
[111] "140.239.15.52"  "216.156.252.15"  "216.247.174.47"  "216.13.209.147"  "216.149.28.83"
[116] "209.209.186.183" "213.82.162.86"   "128.194.164.42"  "216.22.200.37"   "193.130.128.3"
[121] "194.129.106.6"  "200.28.152.130"  "65.114.67.4"     "207.1.3.205"     "63.73.65.229"
[126] "193.220.34.235" "139.95.253.11"   "205.252.62.135"  "210.80.150.2"    "64.65.56.131"
[131] "211.23.14.195"  "210.171.113.194" "200.131.253.2"   "140.111.133.193" "211.38.42.186"
[136] "211.41.56.172"  "63.119.175.13"   "211.20.120.203"  "211.226.16.4"    "209.238.94.227"
[141] "200.23.171.5"   "211.42.174.10"   "211.53.209.206"  "212.45.5.227"    "211.47.67.247"
[146] "200.225.212.42" "202.64.182.67"   "62.213.1.125"    "195.13.14.5"     "64.176.169.229"
[151] "216.156.152.128" "210.190.52.194" "210.163.169.56"  "213.208.134.46"  "195.5.220.70"
[156] "212.15.64.41"   "193.122.31.123"  "193.136.160.68"  "200.205.162.228" "209.123.32.29"
```

```
      Date               Notify              URL                IP              Country
 Length:212093      Length:212093      Length:212093      Length:212093      Length:212093
 Class :character   Class :character   Class :character   Class :character   Class :character
 Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character


      OS              WebServer          Encoding            Lang               Ransom
 Length:212093      Length:212093      Length:212093      Length:212093      Min.   : 100
 Class :character   Class :character   Class :character   Class :character   1st Qu.: 817
 Mode  :character   Mode  :character   Mode  :character   Mode  :character   Median :1543
                                                                             Mean   :1546
                                                                             3rd Qu.:2272
                                                                             Max.   :3000
                                                                             NA's   :160000

    DownTime            Loss          Valid_IP
 Min.   :  7.00   Min.   :   30   Mode:logical
 1st Qu.:20.00    1st Qu.:  804   TRUE:212093
 Median :32.00    Median : 1904
 Mean   :31.89    Mean   : 2763
 3rd Qu.:43.00    3rd Qu.: 3014
 Max.   :60.00    Max.   :18000
```

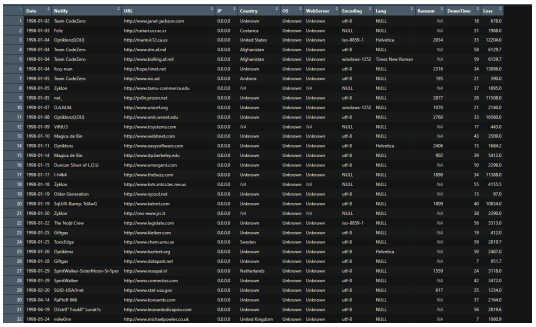


Figure 3.6: Carrying out the formatting and fixing the IP addresses.

We use the unique() function for checking the newly formatted data for the IP column which was correctly formatted. Then we use summary() to check for the IP addresses column, there is an

extra column named Valid_IP. We remove that column and use the view() function to check if it was successfully removed or not.

We then impute the countries in the country column

```
# Set the path to your GeoLite2 database
db_path ← "C:/Asia Pacific University/All Module Notes/Semister-3/Programming For Data
Analysis/Assignment/GeoLite2-Country.mmdb"  # Change this path based on your system

# Function to get country from IP
get_country ← function(ip) {
  result ← maxmind(ip, db_path, fields = "country_name")
  return(ifelse(is.na(result), "Unknown", result))
}

# Apply function to "Unknown" country values
country_clean$Country[country_clean$Country == "Unknown"] ← sapply(country_clean$IP[country_clean$Country ==
"Unknown"], get_country)

# Checking if the imputation was correct or not
sum(country_clean$Country == "Unknown")
```

```
[1] 13394
```

Figure 3.7: Imputation of Country using “GeoLite2-Country.mmdb”

It works by checking each IP address from the database in the list and matching the IP address in our dataset that has missing values with the country column, where we can see that the missing values "Unknowns" count is reduced to 13394. It reduced about (67311 - 13394) 53,917 missing values. This makes the dataset proper for analysis.

Before we move into for analysis part. It is better to check whether the data is in the correct format after the imputation process.

```
unique(country_clean$Country)
class(country_clean$Country)
```

```
[[1]]
[1] "Unknown"

[[2]]
[1] "Costa Rica"

[[3]]
[1] "United States"

[[4]]
[1] "Afghanistan"

[[5]]
[1] "Andorra"

[[6]]
[1] "Sweden"

[[7]]
[1] "Netherlands"

[[8]]
[1] "United Kingdom"

[[9]]
[1] "India"

[[10]]
[1] "Mexico"

[[11]]
[1] "Italy"
```

Figure 3.8: Checking the data for the “Country” column after imputation.

We can see that the data is corrupted due to changing class from "char" to "list", which might hinder us in our analysis.

```
country_clean$Country ← sapply(country_clean$Country, function(x) paste(unlist(x), collapse = ", "))
country_clean$Country ← as.character(country_clean$Country)  # Ensure it's stored as character

# Checking for Class
class(country_clean$Country)

# Checking for country
unique(country_clean$Country)

# Checking number of "Unknown"
sum(country_clean$Country == "Unknown")
```

```
[1] "character"
```

```
  [1] "Unknown"                 "Costa Rica"
  [3] "United States"           "Afghanistan"
  [5] "Andorra"                 "Sweden"
  [7] "Netherlands"             "United Kingdom"
  [9] "India"                   "Mexico"
 [11] "Italy"                   "Thailand"
 [13] "Japan"                   "Brazil"
 [15] "Australia"               "China"
 [17] "Malaysia"                "Russia"
 [19] "Germany"                 "South Africa"
 [21] "Pakistan"                "Czechia"
 [23] "Argentina"               "Belgium"
 [25] "Denmark"                 "Romania"
 [27] "Colombia"                "France"
 [29] "Canada"                  "Hungary"
 [31] "Niue"                    "Tonga"
 [33] "Singapore"               "Austria"
 [35] "Slovenia"                "Indonesia"
 [37] "South Korea"             "Portugal"
 [39] "Bolivia"                 "Taiwan"
 [41] "Hong Kong (China)"       "Yugoslavia"
 [43] "Norway"                  "Montenegro"
 [45] "Switzerland"             "Philippines"
 [47] "Panama"                  "New Zealand"
 [49] "Peru"                    "Iran"
 [51] "Honduras"                "Niger"
```

```
 [51] "Honduras"                "Niger"
 [53] "Nicaragua"               "Turkey"
 [55] "Saudi Arabia"            "Virginislands(British)"
 [57] "Ireland"                 "American Samoa"
 [59] "Kuwait"                  "Croatia"
 [61] "Chile"                   "Spain"
 [63] "Cuba"                    "Burkina Faso"
 [65] "Venezuela"               "Lesotho"
 [67] "Finland"                 "Israel"
 [69] "Puerto Rico"             "Georgia"
 [71] "Macedonia"               "Greece"
 [73] "Uruguay"                 "Eritrea"
 [75] "Haiti"                   "Qatar"
 [77] "Luxembourg"              "Sri Lanka"
 [79] "Mongolia"                "Ukraine"
 [81] "Bulgaria"                "Morocco"
 [83] "Slovakia"                "Ghana"
 [85] "Estonia"                 "Guam"
 [87] "Albania"                 "Paraguay"
 [89] "Ecuador"                 "Poland"
 [91] "Moldova"                 "Suriname"
 [93] "El Salvador"             "Lebanon"
 [95] "Jordan"                  "Northern Mariana Islands"
 [97] "Egypt"                   "Tuvalu"
 [99] "Bosnia And Herzegovina"  "Gabon"
[101] "United Arab Emirates"    "Barbados"
[103] "Lithuania"               "Liechtenstein"
[105] "Nepal"                   "Togo"
[107] "Zambia"                  "Guinea"
[109] "Vatican City State"      "Armenia"
[111] "Turks and Caicos Islands" "Mauritius"
[113] "Mozambique"              "Bhutan"
[115] "Iceland"                 "Macau (China)"
```

Figure 3.9: Converting The “List” to “Character” for “Country” column.

So we converted the "List" to “Character” and checked for "Unknowns" to see if any data was lost along the process of conversion. The output showcases the data is formatted correctly with no sign of corruption.

- **Date**

```
#Clean variable Date
asm_data$Date <- as.Date(asm_data$Date, format = "%Y-%m-%d")
asm_data<- asm_data %>%
  mutate(Year = lubridate::year(Date))

unique(asm_data$Date)
length(unique(asm_data$Date))
unique(asm_data$Year)
```

Figure 4.0 Source code for converting and extracting of Date variable

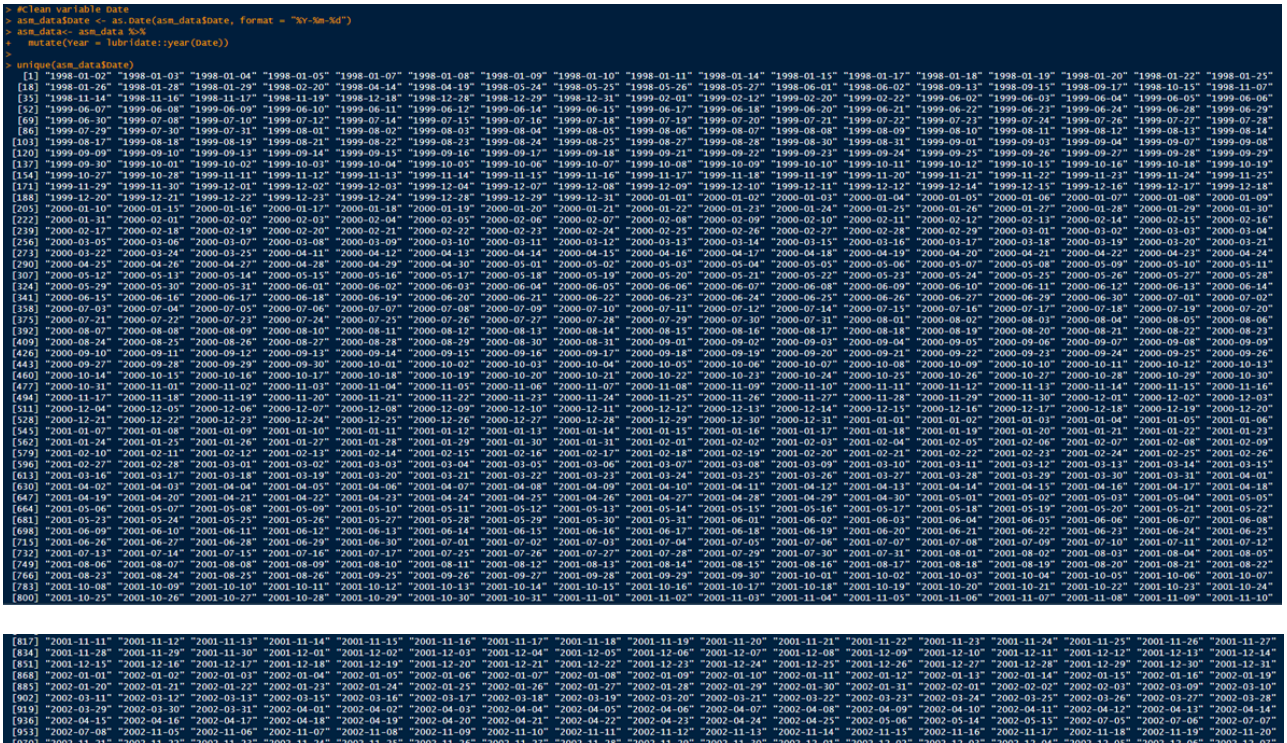


Figure 4.1 Output of Figure 4.0

Figure 2.2.1 shows the cleaning and extracting information from a date variable in a dataset. The format "%Y-%m-%d" specifies that the dates are in the format of Year-Month-Day (e.g., "2024-02-15"). It extracts the year from each date and adds it as a new column named Year, by extracting the year from Date using the lubridate::year() function. After the transformation, the script checks for unique dates in the dataset using unique(), counts the number of unique dates with length(unique()), and verifies the unique years extracted with unique().

- **Web Server**

```r
#Clean variable WebServer
# Convert to lowercase
asm_data$WebServer <- str_to_lower(asm_data$WebServer)

# Remove version numbers (if not needed)
asm_data$WebServer <- str_replace(asm_data$WebServer, "/.*", "")

# Correct typos and inconsistencies
asm_data$WebServer <- str_replace(asm_data$WebServer, "ideawebse", "ideawebserver")

# Remove special characters
asm_data$WebServer <- str_replace_all(asm_data$WebServer, "[^[:alnum:]]", "")
asm_data <- asm_data %>% filter(WebServer!= "")
```

Figure 4.2 Source Code for cleaning the WebServer variable

```
> # Remove special characters
> asm_data$WebServer <- str_replace_all(asm_data$WebServer, "[^[:alnum:]]", "")
> asm_data <- asm_data %>% filter(WebServer!= "")
> unique(asm_data$WebServer)
  [1] "unknown"                                  "iis"                                  "gws"
  [4] "solarissunos"                             "apache"                               "21674154150"
  [7] "iis50"                                    "zeus"                                 "iis40"
 [10] "iis51"                                    "iis60"                                "thttpd"
 [13] "iis30"                                    "rapidsite"                            "artblast"
 [16] "coffeemaker"                              "ideawebserverrver"                    "redhatsecure"
 [19] "concentrichostashurbanipal"               "website"                              "roxenchallenger"
 [22] "web602"                                   "netscapeenterprise"                   "ideawebserver"
 [25] "zope"                                     "noyb"                                 "sambar"
 [28] "sonataser"                                "litespeed"                            "nginx"
 [31] "lighttpd"                                 "yts"                                  "sunonewebserver"
 [34] "resin"                                    "av"                                   "dinahttpdserver"
 [37] "oracleas"                                 "squid"                                "lasso"
 [40] "silverstreamserver"                       "sunjavasystemwebserver"               "hpchaiserver"
 [43] "cloudflarenginx"                          "powermodbyweb4hostnet"                "webaccelerationbyhttp"
 [46] "blockdos"                                 "apacheboosterbyhttp"                  "modsecuritybydevonilcom"
 [49] "ngxopenresty"                             "unix5"                                "ghs"
 [52] "sputnik2aus"                              "overseeturingv100"                    "pcx"
 [55] "tomahawkultra"                            "sputnik6aus"                          "microsoftiis"
 [58] "microsoftofficeweb50pub"                  "pnsboxhttpproxyservice"               "securedbyfs4hostcom"
 [61] "rackcorpcdn"                              "oracleiplanetwebserver"               "quotquot"
 [64] "xartvisualserverlinux15artvisualnet"      "mythrilserver40extention"             "whatchutalkin039boutwillis"
 [67] "akamaighost"                              "scicubelimited"                       "safe3webfirewall"
 [70] "ats"                                      "genericowebserverv546"                "ecpay"
 [73] "hosting"                                  "chamber88nginx"                       "dungeon9"
 [76] "gse"                                      "exaserver"                            "isetwwwisetcombrapache"
 [79] "mbog"                                     "bydrsrvcom"                           "dimofinfhosting"
 [82] "taranahost"                               "traidntsecurity"                      "wpengine"
 [85] "ibmhttpserver"                            "perldancer13071"                      "sputnik3aus"
 [88] "freerice"                                 "webserver"                            "bigip"
 [91] "sec"                                      "s8aus"                                "advancedhostingbyhttp"
 [94] "sputnik4aus"                              "redhatvnconfigbyinforedhatvnnet"      "hostinstallnginx"
 [97] "hostvnnet10reef"                          "minigun"                              "securedbyhlouncom"
[100] "kasperskylabs"                            "cherrypy"                             "sffe"
[103] "webdevel"                                 "sunglassfishenterpriseserverv211"     "poweredbytotalinwebhostingsolutionswwwtotalincomsupport"
[106] "xkrypton"                                 "powermodbyoho"                        "rtwebserver"
[109] "pws"                                      "udwebspace30"                         "modsecurity259enabled"
[112] "gwiseguy"                                 "jinoru"                               "basehttp"
[115] "fs5"                                      "depdiknaswebserver"                   "webserv"
[118] "varnish"                                  "beast"                                "mircrosoftofficeweb50pub"
[121] "wisepanelwebserver"                       "mye1"                                 "aserver"
[124] "httpd"                                    "thoriixplanetserver"                  "tbgf4"
[127] "cherokee"                                 "x1356"                                "webserverx"
[130] "oracleapplicationserver10g"               "masterws"                             "wukong"
[133] "dfserver"                                 "virataem"                             "lotusdomino"
```

Figure 4.3 Output of Figure 4.2

```r
# Group generic names
asm_data$WebServer[asm_data$WebServer %in% c("webserver", "website")] <- "other"

# Group similar servers
asm_data <- asm_data %>%
  mutate(
    WebServer = case_when(
      str_detect(WebServer, "unknown")~ "unknown",
      str_detect(WebServer, "apache") ~ "apache",
      str_detect(WebServer, "lighttpd") ~ "lighttpd",
      str_detect(WebServer, "nginx") ~ "nginx",
      str_detect(WebServer, "iis") ~ "iis",
      TRUE ~ "other"  # Keep the original value if no match
    )
  )
```

Figure 4.4 Source Code for grouping the WebServer into different types

```
> # Group generic names
> asm_data$WebServer[asm_data$WebServer %in% c("webserver", "website")] <- "other"
>
> # Group similar servers
> asm_data <- asm_data %>%
+   mutate(
+     WebServer = case_when(
+       str_detect(WebServer, "unknown")~ "unknown",
+       str_detect(WebServer, "apache") ~ "apache",
+       str_detect(WebServer, "lighttpd") ~ "lighttpd",
+       str_detect(WebServer, "nginx") ~ "nginx",
+       str_detect(WebServer, "iis") ~ "iis",
+       TRUE ~ "other"  # Keep the original value if no match
+     )
+   )
> unique(asm_data$WebServer)
[1] "unknown"  "iis"      "other"    "apache"   "nginx"    "lighttpd"
> length(unique(asm_data$WebServer))
[1] 6
```

Figure 4.5 Output of Figure 4.4

It first converts all values to lowercase to ensure uniformity by using str_to_lower(). Next, it removes version numbers, corrects typos (e.g., "ideawebse" to "ideawebserver"), and eliminates special characters to maintain consistency with str_replace() and str_replace_all(), leaving only alphanumeric characters. Empty values are filtered out to prevent missing data issues. Additionally, generic names like "webserver" and "website" are grouped under "other", while commonly known web servers such as "apache", "lighttpd", "nginx", and "iis" are categorized under their respective names. Any remaining values are classified as "other" to simplify the dataset and improve analysis. Similar web servers such as "apache," "nginx," "lighttpd," and "iis" are classified into their respective categories using case_when(), while all other values default to "other." This ensures meaningful categorization for analysis.

```
asm_data <- asm_data %>% distinct()
nrow(asm_data)
```

Figure 4.6 Source Code for Remove duplicate rows from the dataset & check number of rows of the data

```
> asm_data <- asm_data %>% distinct()
> nrow(asm_data)
[1] 204358
```

Figure 4.7 Output of Figure 4.6

Finally, duplicate rows are removed using distinct(), reducing the dataset to 204,358 rows of unique records. This cleaning process enhances data quality and consistency, making it more suitable for further analysis.

- **OS**

```
#1. DATA CHECKING
#1.1 #Checking unique values
str(os_data$OS)
unique(os_data$OS)

#1.2 Check NA values
na_count <- sum(is.na(os_data$OS))
cat("\nNumber of NA values before:", na_count)

missing_percentage <- round(((na_count / length(data$OS)) * 100),4)
cat("\nMissing Percentage : ", missing_percentage,"%\n")

#1.3 Checking for Whitespaces
# 1. Check for leading whitespace (spaces at the start)
leading_space <- sum(grepl("^\\s+",os_data$OS))
cat("\nEntries with leading whitespace:", leading_space, "\n")

# 2. Check for trailing whitespace (spaces at the end)
trailing_space <- sum(grepl("\\s+$",os_data$OS))
cat("Entries with trailing whitespace:", trailing_space, "\n")

# 3. Check for multiple spaces between words
middle_space <- sum(grepl("\\s{2,}",os_data$OS))
cat("Entries with extra spaces between words:", middle_space, "\n")
```

Figure 5.0 Source Code of Data Preparation or Data Checking in OS

```
unique(os_data$OS)
 [1] "Unknown"
 [3] "Linux"
 [5] "IRIX"
 [7] "SCO Unix"
 [9] "SolarisSunOS"
[11] "Win NT9x"
[13] "HP-UX"
[15] "Tru64"
[17] "Unix"
[19] "NovellNetware"
[21] "Aix"
[23] "windows"
[25] "SCO UNIX"
[27] "WINDOWS"
[29] "BSDOS"
[31] "FreeBsd"
[33] "OpenBsd"
[35] "NetBSDOpenBSD"
[37] "NT4Windows 98"
[39] "Windows 2000"
[41] "Win .NET"
[43] "VM"
[45] "Win 2003"
[47] "FreeB"
[49] "Win N"
[51] "Win X"
[53] "Solaris 9/10"
[55] "Citri"
[57] "F5 Big-IP"
[59] "Juniper IVE OS 7.X"
[61] "Cisco embedded"
[63] "Netgear DG834G WAP or Western Digital WD TV media player"
[65] "Microsoft Windows 7|Phone|Vista|2008"
[67] "Microsoft Windows Vista|2008|7"
[69] "HP P2000 G3 NAS device"
[71] "Microsoft Windows 7|Phone|2008|Vista"
[73] "Netopia embedded"
[75] "D-Link embedded"
[77] "Microsoft Windows Vista|2008|7|Phone"
[79] "HP LaserJet"
[81] "Crestron XPanel control system"
[83] "HP 4000M ProCurve switch (J4121A)"
[85] "Juniper SA4000 SSL VPN gateway (IVE OS 7.0)"
[87] "Microsoft Windows Server 2003 SP1 or SP2"
[89] "Microsoft Windows Vista"
[91] "Samsung embedded"
[93] "Microsoft Windows 7|Vista"
[95] "EnGenius embedded"
[97] "Microsoft Windows Server 2008 R2 or Windows 8"
[99] "Microsoft Windows Server 2003 Enterprise Edition"
[101] "HTC Windows PocketPC/CE"
[103] "Microsoft Windows Phone|7|2008|Vista"
[105] "Cisco Windows PocketPC/CE"
[107] "Microsoft Windows Vista|2008|7|2012|Longhorn"
[109] "Dish Network Hopper media device"
[111] "Citrix Access Gateway VPN gateway"
[113] "Cisco PIX OS 8.X|6.X"
[115] "Konica Minolta embedded"
[117] "IBM AIX 6.X|5.X"
[119] "Juniper JUNOS 8.X|9.X|10.X|11.X"
"Windows"
"FreeBSD"
"MacOS"
"BSDI"
"Digital Unix"
"Digital UNIX"
"AIX"
"OpenBSD"
"NetBSD"
"Win 2000"
"BSDi"
"Win XP"
"linux"
"LINUX"
NA
"FreeBsD"
"OS2"
"Solaris"
"Solaris 8"
"unknown"
"MacOSX"
"Compaq Tru64"
"Win 2"
"Solar"
"NetBS"
"Unkno"
"Win 2008"
"F5 Bi"
"iPXE 1.X"
"Cisco Unified Communications Manager VoIP adapter"
"F5 Networks embedded"
"Citrix embedded"
"Crestron 2-Series"
"AVtech embedded"
"Win 2012"
"Google Android 4.X"
"D-Link DIR-300 WAP"
"Microsoft Windows Phone|Vista|2008|7"
"Linksys WRV54G WAP"
"OVH/Shared"
"Motorola 2210-02 ADSL modem"
"Cisco IOS 12.X"
"Microsoft Windows Vista SP2 or Windows 7 Ultimate SP0 - SP1"
"Microsoft Windows 7 Ultimate"
"Vonage V-Portal VoIP adapter"
"Blue Coat SGOS 6.X"
"Linksys embedded"
"FreeNAS 0.7.2 (FreeBSD 7.3-RELEASE)"
"Microsoft Windows Vista|7"
"Microsoft Windows 7 Ultimate Beta (build 7000)"
"HP embedded"
"Microsoft Windows 7"
"Efficient Networks embedded"
"Epson embedded"
"Microsoft Windows Vista|2008|7|Longhorn|2012"
"EnGenius ESR-9250 WAP"
"Thecus embedded"
"Fortinet embedded"
"Netgear embedded"
"Netopia 3386 ADSL router"
```

Figure 5.1 Unique Values in OS Column

```
> #1.2 Check NA values
> na_count <- sum(is.na(os_data$OS))
> cat("\nNumber of NA values before:", na_count)

Number of NA values before: 3
> missing_percentage <- round(((na_count / length(os_data$OS)) * 100),4)
> cat("\nMissing Percentage : ", missing_percentage,"%\n")

Missing Percentage :  0.0014 %
```

Figure 5.2 Missing values and its percentage

```
Summary of whitespaces checking:
Entries with leading whitespace: 0
Entries with trailing whitespace: 0
Entries with extra spaces between words: 0
```

Figure 5.3 This is to check the summary of whitespaces

```
#2.1 Imputation Using MODE
#2.1.1 Imput Mode to the missing value
getMode <- function(x) {
  uniqx <- unique(x)
  uniqx[which.max(tabulate(match(x, uniqx)))]
}

#2.1.2 Get mode of OS
os_mode <- getMode(os_data$OS)
cat("\nMode value:", os_mode)

#2.1.3 Replace NA with mode
data$OS[is.na(os_data$OS)] <- os_mode

#2.1.4 Check if imputation worked
cat("\nMissing values after:", sum(is.na(data$OS)))
```

Figure 5.4 Imputation using Mode

```
> #2.1.2 Get mode of OS
> os_mode <- getMode(os_data$OS)
> cat("\nMode value:", os_mode)

Mode value: Linux

> #2.1.3 Replace NA with mode
> os_data$OS[is.na(os_data$OS)] <- os_mode

> #2.1.4 Check if imputation worked
> cat("\nMissing values after:", sum(is.na(os_data$OS)))

Missing values after: 0
```

Figure 5.5 The result after imputation

Here, we can see that Linux is the most used operating system. We impute the data using mode.

```r
os_data$OS <- case_when(
  os_data$OS %in% c("Unknown", "unknown", "Unkno") ~ "Unknown",

  # Windows Family
  os_data$OS %in% c("windows", "WINDOWS", "Windows", "win") ~ "Windows",
  os_data$OS %in% c("Win 2003", "Windows 2003", "Microsoft Windows Server 2003", "Microsoft Windows Server 2003 Enterprise Edition","Win .NET" ) ~ "Windows Server 2003",
  os_data$OS %in% c("Win XP", "Windows XP") ~ "Windows XP",
  os_data$OS %in% c("Win 2000", "Windows 2000") ~ "Windows 2000",
  os_data$OS %in% c("Win 2008", "Windows 2008") ~ "Windows Server 2008",
  os_data$OS %in% c("Win 2012", "Windows 2012") ~ "Windows Server 2012",
  os_data$OS %in% c("Win NT9x", "NT4Windows 98") ~ "Windows NT",
  os_data$OS %in% c("Windows 2", "Windows N") ~ "Windows",
  os_data$OS =="Windows X" ~ "Windows 10",
  grepl("Server 2003.*SP", os_data$OS) ~ "Windows Server 2003",
  grepl("Server 2008.*R2.*Windows 8", os_data$OS) ~ "Windows Server 2008",
  grepl("Vista SP2.*Windows 7", os_data$OS) ~ "Windows Vista",
  grepl("Microsoft Windows.*7|Phone|Vista|2008", os_data$OS) ~ "Windows 7",
  grepl("Microsoft Windows Phone", os_data$OS) ~ "Windows Phone",
  grepl("Windows.*PocketPC|Windows.*CE", os_data$OS) ~ "Windows CE",

  # Linux Family
  os_data$OS %in% c("linux", "LINUX") ~ "Linux",

  # Unix Family
  os_data$OS %in% c("Unix", "UNIX") ~ "Unix",
  os_data$OS %in% c("SCO Unix", "SCO UNIX") ~ "SCO UNIX",
  os_data$OS %in% c("Digital Unix", "Digital UNIX") ~ "Digital UNIX",
  os_data$OS %in% c("Tru64", "Compaq Tru64") ~ "Tru64 UNIX",
  os_data$OS == "SolarisSunOS" ~ "SunOS",
  os_data$OS %in% c("Solaris", "Solar") ~ "Solaris",
  os_data$OS %in% c("Solaris 9/10", "Solaris 8") ~ "Solaris",

  # BSD Family
  os_data$OS %in% c("BSDI", "BSDi", "BSDOS") ~ "BSD",
  os_data$OS %in% c("FreeBSD", "FreeBsd", "FreeBsD", "FreeB") ~ "FreeBSD",
  os_data$OS %in% c("OpenBSD", "OpenBsd") ~ "OpenBSD",
  os_data$OS %in% c("NetBSD", "NetBs", "NetBS") ~ "NetBSD",
  os_data$OS == "NetBSD and OpenBSD" ~ "BSD",

  # Mac Family
  os_data$OS %in% c("MacOS", "MacOSX") ~ "MacOS",

  # Other Systems
  os_data$OS == "IRIX" ~ "IRIX",
  os_data$OS %in% c("AIX", "Aix", "IBM AIX", "IBM AIX 6.X|5.x") ~ "AIX",
  os_data$OS == "HP-UX" ~ "HP-UX",
  os_data$OS == "NovellNetware" ~ "Novell Netware",
  os_data$OS == "OS2" ~ "OS/2",
  os_data$OS == "VM" ~ "Virtual Machine",
  os_data$OS == "Citri" ~ "Citrix",

  #Network/Security Devices - remove version numbers
  grepl("Cisco PIX", os_data$OS) ~ "Cisco PIX OS",
  grepl("Cisco IOS", os_data$OS) ~ "Cisco IOS",
  grepl("Juniper.*JUNOS", os_data$OS) ~ "Juniper JUNOS",
  grepl("Juniper.*IVE", os_data$OS) ~ "Juniper IVE",
  grepl("BlueC Coat", os_data$OS) ~ "Blue Coat SGOS",
  grepl("Android", os_data$OS) ~ "Android",
  grepl("iPXE", os_data$OS) ~ "iPXE",
  grepl("FreeNAS", os_data$OS) ~ "FreeBSD",
  os_data$OS %in% c("F5 Bi", "F5 Big-IP", "F5 BIG-IP") ~ "F5 BIG-IP",


  # Embedded/Network Devices
  grepl("embedded|LaserJet|NAS device|adapter|gateway", os_data$OS, ignore.case = TRUE) ~ "Embedded System",
  grepl("Creston|Dish Network|device|system", os_data$OS) ~ "Embedded System",
  grepl("router|WAP|switch|modem|adapter", os_data$OS, ignore.case = TRUE) ~ "Network Device",

  # Keep original if no match
  TRUE ~ os_data$OS
)
```

Figure 5.6: Fixing Misspelled Names in Operating System

The code standardizes operating system names in a three-tier process using the grepl() function for pattern matchings and the case_when() function to implement the categorization logic efficiently:

1. Unifying unknown system variations
2. Standardizing different versions of major OS families into consistent formats (Windows, Unix, Linux, BSD, MacOS)
3. Grouping network/embedded system according to keyword identification

3.0 Data Analysis

3.1 Objective 1: To investigate the relationship between country and loss. (Faiyad Mahabub Tasin | TP077983 | CS-DA)

- Data Loading And Importing Important Libraries

```
library(tidyverse)
library(dplyr)
library(stringr)
library(remotes)
library(rgeolocate)
library(ggplot2)
library(ggrepel)
library(scales)
library(reshape2)
library(ggcorrplot)
```

```
setwd("C:/Asia Pacific University/All Module Notes
/Semister-3/Programming For Data Analysis
/Assignment/Assignment")

country_clean ← read.csv("Clean_Loss_Data.csv")
view(country_clean)
```

	Date	Notify	URL	IP	Country	OS	WebServer	Encoding	Lang	Ransom	DownTime	Loss
1	1998-01-02	Team CodeZero	http://www.janet-jackson.com	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	NA	18	678.0
2	1998-01-03	Feliz	http://cariari.ucr.ac.cr	0.0.0.0	Costa Rica	Unknown	Unknown	NULL	NULL	NA	31	1988.0
3	1998-01-04	Optiklenz(LOU)	http://marin.k12.ca.us	0.0.0.0	United States	Unknown	Unknown	iso-8859-1	Helvetica	2034	35	12204.0
4	1998-01-04	Team CodeZero	http://www.dm.af.mil	0.0.0.0	Afghanistan	Unknown	Unknown	utf-8	NULL	NA	58	6129.7
5	1998-01-04	Team CodeZero	http://www.bolling.af.mil	0.0.0.0	Afghanistan	Unknown	Unknown	windows-1252	Times New Roman	NA	59	6129.7
6	1998-01-04	foxy man	http://hope.hinet.net	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	2316	34	13896.0
7	1998-01-05	Team CodeZero	http://www.nic.ad	0.0.0.0	Andorra	Unknown	Unknown	utf-8	NULL	195	21	390.0
8	1998-01-05	Zyklon	http://www.tamu-commerce.edu	0.0.0.0	Unknown	Unknown	NA	NULL	NULL	NA	37	1895.0
9	1998-01-05	net_	http://pr0n.prizon.net	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	2877	28	11508.0
10	1998-01-07	D.A.M.M.	http://www.unicef.org	0.0.0.0	Unknown	Unknown	Unknown	windows-1252	NULL	1070	21	2140.0
11	1998-01-08	Optiklenz(LOU)	http://www.wvlc.wvnet.edu	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	2760	33	16560.0
12	1998-01-09	ViRtU3	http://www.trsystems.com	0.0.0.0	Unknown	Unknown	NA	NULL	NULL	NA	17	443.0
13	1998-01-10	Magica de Bin	http://www.webbnet.com	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	NA	43	3509.0
14	1998-01-11	Optiklenz	http://www.easysoftware.com	0.0.0.0	Unknown	Unknown	Unknown	utf-8	Helvetica	2406	15	1684.2
15	1998-01-14	Magica de Bin	http://www.itp.berkeley.edu	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	902	39	5412.0
16	1998-01-15	Duncan Silver of L.O.U	http://www.emergent.com	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	NA	50	2296.0
17	1998-01-17	I-H4k4	http://www.thebuzz.com	0.0.0.0	Unknown	Unknown	Unknown	NULL	NULL	1898	34	11388.0
18	1998-01-18	Zyklon	http://www.kvtc.mtcs.tec.me.us	0.0.0.0	Unknown	Unknown	NA	NULL	NULL	NA	55	4155.5
19	1998-01-19	Older Generation	http://www.njcool.net	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	NA	13	97.0
20	1998-01-19	SqUiRl & TdAwG	http://www.kalnet.com	0.0.0.0	Unknown	Unknown	Unknown	utf-8	NULL	1809	40	10854.0
21	1998-01-20	Zyklon	http://ceo-www.jrc.it	0.0.0.0	Unknown	Unknown	NA	NULL	NULL	NA	38	2290.0
22	1998-01-22	The NoJd Crew	http://www.legislate.com	0.0.0.0	Unknown	Unknown	Unknown	iso-8859-1	NULL	NA	56	3513.0

Figure 6.0: Loading The Dataset

Before starting any kind of analysis, we first import the data into our R studio. We check out the data using the view() function from library moments to validate whether the dataset has been inserted successfully. We name our dataset as country_clean since we have cleaned our data beforehand in the data cleaning process and start with our analysis.

3.1.1 Analysis 1-1: What are the highest and least countries that experienced most of the losses?

```
summary_stats ← country_clean %>% group_by
(Country) %>%
  summarise(Total_Loss = sum(Loss, na.rm = TRUE),
            Avg_Loss = mean(Loss, na.rm = TRUE),
            Median_Loss = median(Loss, na.rm =
TRUE),
            Loss_SD = sd(Loss, na.rm = TRUE),
            Count = n())
print(summary_stats)
```

A tibble: 225 × 6

Country	Total_Loss	Avg_Loss	Median_Loss	Loss_SD	Count
Afghanistan	13326.4	4442.1333	6129.70	2922.95121	3
Africa	35310.4	2522.1714	2616.90	1286.57869	14
Albania	139921.2	2498.5929	1541.00	3354.82810	56
Algeria	35925.6	3265.9636	2942.70	4420.76399	11
American Samoa	85563.6	3169.0222	1755.00	4378.98622	27
Andorra	14724.7	1840.5875	883.50	2283.38488	8
Angola	1361.8	680.9000	680.90	97.43931	2
Antigua And Barbuda	15560.5	3112.1000	1865.00	2363.24915	5
Argentina	4348069.4	2801.5911	1858.00	3449.39203	1552
Armenia	100868.3	2801.8972	1904.50	2842.49953	36

1-10 of 225 rows Previous 1 2 3 4 5 6 . 23 Next

Figure 6.1: Grouping The Data

To make our analysis easier and less redundant we make new columns such as “Average Loss”, “Total Loss”, etc. We group them out using the dplyr library which would help to form our analysis.

Then, we see the statistics findings for countries with the highest and lowest losses.

```
# Getting Countries with High Losses.
top_countries_loss ← summary_stats %>% arrange(desc(Total_Loss)) %>% select(Country, Total_Loss)

# Getting Countries with Low Losses.
low_countries_loss ← summary_stats %>% arrange(Total_Loss) %>% select(Country, Total_Loss)

# Display
print(top_countries_loss)
print(low_countries_loss)
```

Country	Total_Loss
United States	225798096.4
Unknown	36995095.6
Germany	26866869.6
United Kingdom	22553361.2
Brazil	21843240.7
Spain	19546110.5
France	19052190.4
China	13483381.3
Italy	13452007.1
Canada	12912054.0

1-10 of 225 rows Previous 1 2 3 4 5 6 . 23 Next

Country	Total_Loss
Gambia	53.0
Curaçao	81.0
Cook Islands	831.0
Trinidad and Tobago	902.0
Turkmenistan	988.0
Uganda	1008.0
Saint Kitts and Nevis	1253.0
Norfolk Island	1314.0
Angola	1361.8
Guadeloupe	1829.0

1-10 of 225 rows

Figure 6.2: Statistic Findings For Highest And Lowest Losses

These are the countries that have High Losses and Low Losses worldwide. Where the highest can be seen for the United States and Lowest can be seen for Gambia.

Afterward, we visualize the trend of Losses across the top 20 countries using the ggplot2 library.

```
# Grouping top 20 countries
summary_stats_top ← summary_stats %>% arrange(desc(Total_Loss)) %>% head(20)

# Visualizing the trend of Losses across 10 countries.
ggplot(summary_stats_top, aes(x = reorder(Country, - Total_Loss), y = Total_Loss / 1e6)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))+
  ggtitle(paste("Top 20 Countries by Total Loss")) +
  xlab("Country") +
  ylab("Total Loss (Millions)") +
  scale_y_continuous(labels = label_number(scale = 1, suffix = "M"))
```

Figure 6.3: Visualizing The Trend Of Losses For Top 20 Countries

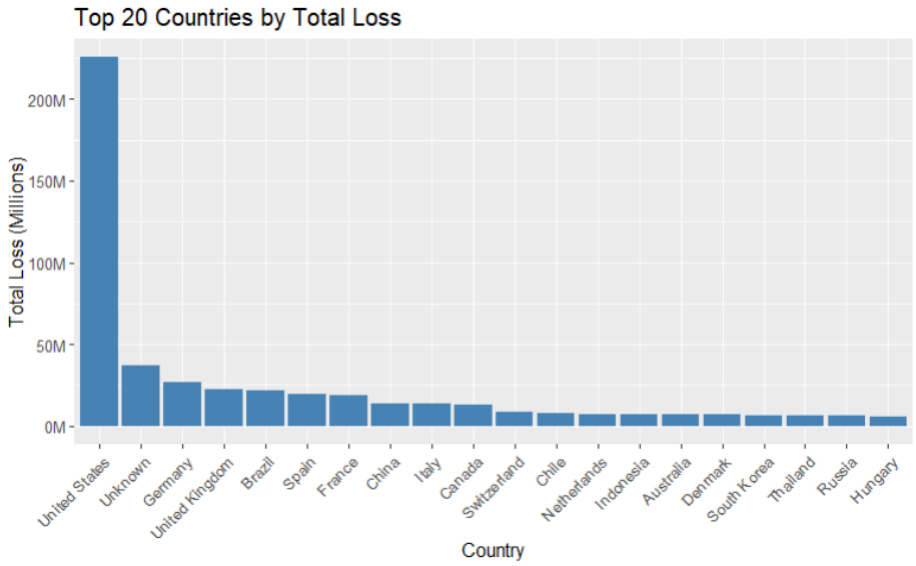


Figure 6.4: The Trend Of Losses For Top 20 Countries

Where this can be visualized by plotting a bar graph since countries are categorical values with loss as its numeric for better visualization and finding meaningful insights. The plot showcases how it is changing for the top 10 countries with the highest being the United States and the lowest top being Hungary.

This shows a visual representation of how it is changing for the least 20 countries

```r
# Grouping Total Loss for least 20 countries
top_20_least <- summary_stats %>%
  arrange(Total_Loss) %>%
  slice_head(n = 20)

# Visualization of loss for least 20 countries
ggplot(top_20_least, aes(x = reorder(Country, Total_Loss), y = Total_Loss / 1e3)) +
  geom_bar(stat = "identity", fill = "steelblue") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  ggtitle("Top 20 Countries with Least Total Loss (in Millions)") +
  xlab("Country") +
  ylab("Total Loss (Thousand)") +
  scale_y_continuous(labels = label_number(scale = 1, suffix = "K")) +
  coord_flip()
```

Figure 6.5: Visualizing The Trend Of Losses For the Top Least Countries

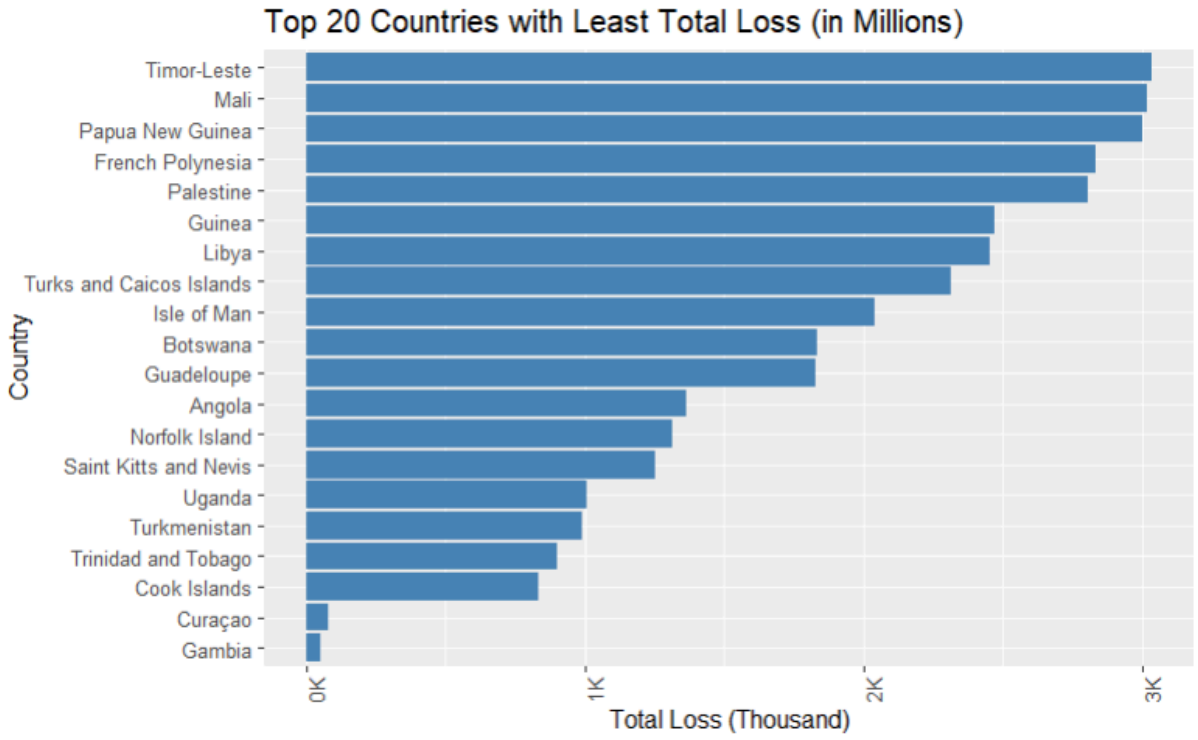


Figure 6.6: The Trend On Losses For Least 20 Countries

Where Gambia is the only country that has lost the least revenue, and Timur-Leste suffered the most revenue loss.

3.1.2 Analysis 1-2: Does total revenue loss for all countries correlate with the frequency of several attacks?

To check whether attack frequency has something to do with the highest and lowest revenue losses. We first see the statistics for their attack frequencies for all countries.

```
# Used to get highest Frequency Attack
high_incidents ← summary_stats %>% arrange(desc(Count)) %>% head(10) %>% arrange(desc(Total_Loss))%>%
  select(Country, Count, Total_Loss)

# Used to get largest Average loss for lowest Frequency Attack
low_incidents_largest ← summary_stats %>% arrange(Count) %>% head(10) %>% arrange(desc(Avg_Loss)) %>%
  select(Country, Count, Avg_Loss)

# Used to get lowest Average loss for lowest Frequency Attack
low_incidents_smallest ← summary_stats %>% arrange(Count) %>% head(10) %>% arrange(Avg_Loss) %>%
  select(Country, Count, Avg_Loss)

# Display
print(high_incidents)
print(low_incidents_smallest)
print(low_incidents_largest)
```

Country	Count	Total_Loss
United States	82052	225798096
Unknown	13394	36995096
Germany	9701	26866870
United Kingdom	8228	22553361
Brazil	7818	21843241
Spain	6993	19546111
France	7003	19052190
China	4793	13483381
Italy	4825	13452007
Canada	4633	12912054

1-10 of 10 rows

Country	Count	Avg_Loss
Gambia	1	53
Curaçao	1	81
Cook Islands	1	831
Norfolk Island	1	1314
Guadeloupe	1	1829
Isle of Man	1	2042
Guinea	1	2473
Bonaire, Sint Eustatius, and Saba	1	3575
Asia-Pacific Region	1	4444
Netherlands Antilles	1	5620

1-10 of 10 rows

Country	Count	Avg_Loss
Netherlands Antilles	1	5620
Asia-Pacific Region	1	4444
Bonaire, Sint Eustatius, and Saba	1	3575
Guinea	1	2473
Isle of Man	1	2042
Guadeloupe	1	1829
Norfolk Island	1	1314
Cook Islands	1	831
Curaçao	1	81
Gambia	1	53

1-10 of 10 rows

Figure 6.7: Statistics For Attack Frequency For All Countries

The statistics showed that the United States had the highest number of attacks which explains its highest revenue loss. The least attack faced was Gambia which had only one attack. But for other countries such as Netherlands Antilles even though the number of attacks was only one, they had to suffer quite a large amount of revenue loss.

It can be visualized using a bar plot that visualizes the revenue loss in blue and the number of attacks in red for the top 10 countries

```
# Select the top 10 countries by number of attacks
attack_stats_top ← summary_stats %>% arrange(desc(Count)) %>% head(10)

# Plot: Number of Attacks & Total Loss
ggplot(attack_stats_top, aes(x = reorder(Country, -Count))) +
  geom_bar(aes(y = Count), stat = "identity", fill = "firebrick", alpha = 0.7) +
  geom_line(aes(y = Total_Loss / 1e6, group = 1), color = "blue", size = 1.2) +
  geom_point(aes(y = Total_Loss / 1e6), color = "blue", size = 3) +
  geom_text(aes(y = Count, label = Count), vjust = -0.5, size = 4) +
  geom_text(aes(y = Total_Loss / 1e6, label = paste0(round(Total_Loss / 1e6, 1), "M")),
            vjust = -0.5, size = 4, color = "blue") +
  scale_x_discrete(guide = guide_axis(angle = 30)) +
  theme(axis.text.x = element_text(hjust = 1, vjust = 1, size = 10)) +
  ggtitle("Top 10 Countries by Number of Attacks & Total Loss") +
  xlab("Country") +
  theme_minimal()
```

Figure 6.8: Visualizing The Trend On Attack Frequencies On Losses For Top 10 Countries

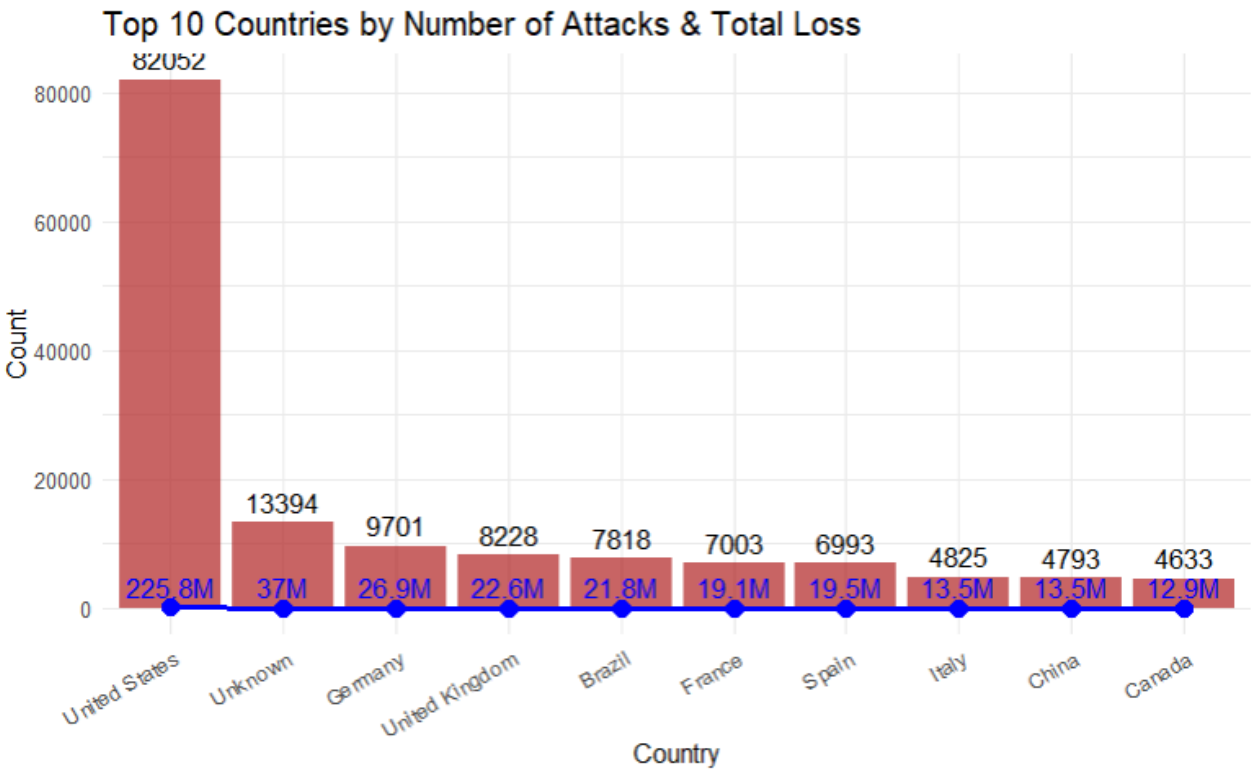


Figure 6.9: The Trend On Losses For Top 20 Countries

In the plot, it shows that the United States suffered the highest number of attacks and also suffered high total revenue loss. The total revenue loss does not always correlate with the number of attacks, since higher attacks can be seen for Germany which has 9,701 attacks and a $26.9M loss whereas the United Kingdom has 8,228 attacks of 22.6M losses, where Germany has greater attack count then the UK but similar revenue loss. This shows that some attacks caused similar financial damage to others.

3.1.3 Analysis 1-3: Do certain countries show a consistent pattern of extreme Loss values, are there any underlying issues that result in these extreme losses?

These showcase the total number of outliers for Loss for each of the country. The highest outliers can be seen for the United States and the least is seen for Africa and below.

```r
# Function For Calculating Outliers for Loss On Each Country
iqr_outlier_detection <- function(country_clean, country_col, loss_col) {
  country_clean %>%
    group_by(!!sym(country_col)) %>%
    mutate(
      Q1 = quantile(!!sym(loss_col), 0.25, na.rm = TRUE),
      Q3 = quantile(!!sym(loss_col), 0.75, na.rm = TRUE),
      IQR = Q3 - Q1,
      Lower_Bound = Q1 - 1.5 * IQR,
      Upper_Bound = Q3 + 1.5 * IQR,
      Is_Outlier = ifelse(!!sym(loss_col) < Lower_Bound | !!sym(loss_col) > Upper_Bound, 1, 0)) %>% ungroup()
}

# Apply the function
country_clean <- iqr_outlier_detection(country_clean, "Country", "Loss")

# Getting the outliers
outlier_counts <- country_clean %>%
  group_by(Country) %>%
  summarise(Total_Outliers = sum(Is_Outlier, na.rm = TRUE)) %>%
  arrange(desc(Total_Outliers)) %>%
  filter(Total_Outliers > 0)

print(outlier_counts)
```

A tibble: 10 x 4

Country	Total_Outliers	Total_Cases	Outlier_Percentage
United States	8067	82052	9.831570
Unknown	1296	13394	9.675974
Germany	958	9701	9.875271
United Kingdom	794	8228	9.649976
Brazil	772	7818	9.874648
Spain	709	6993	10.138710
France	666	7003	9.510210
China	493	4793	10.285834
Italy	489	4825	10.134715
Canada	465	4633	10.036693

1-10 of 10 rows

Figure 6.10: Statistics On Loss Outliers For All Countries

The United States has the highest number of Total Outliers (8067). Despite having such a high number of outliers, the percentage of outliers is smaller than others which is 9.83%. This indicates that US experiences a high volume of loss incidents in general and does not necessarily suffer from unusual spikes of extreme losses, as countries like China (10.28%), Italy (10.13%), and Spain (10.14%) have a higher percentage outliers then USA, this concludes that these countries have a slightly more frequent extreme losses then USA.

We, then visualize the percentage of outliers by countries

```r
ggplot(outlier_counts %>% head(10),
       aes(x = reorder(Country, -Outlier_Percentage), y = Outlier_Percentage)) +
  geom_bar(stat = "identity", fill = "red") +
  geom_text(aes(label = sprintf("%.2f%%", Outlier_Percentage)), vjust = -0.5) +
  ggtitle("Top 10 Countries with Highest Outlier Percentage") +
  xlab("Country") +
  ylab("Outlier Percentage %") +
  theme_minimal()
```

Figure 6.11: Visualizing The Percentage Of Outliers for Top 10 Countries

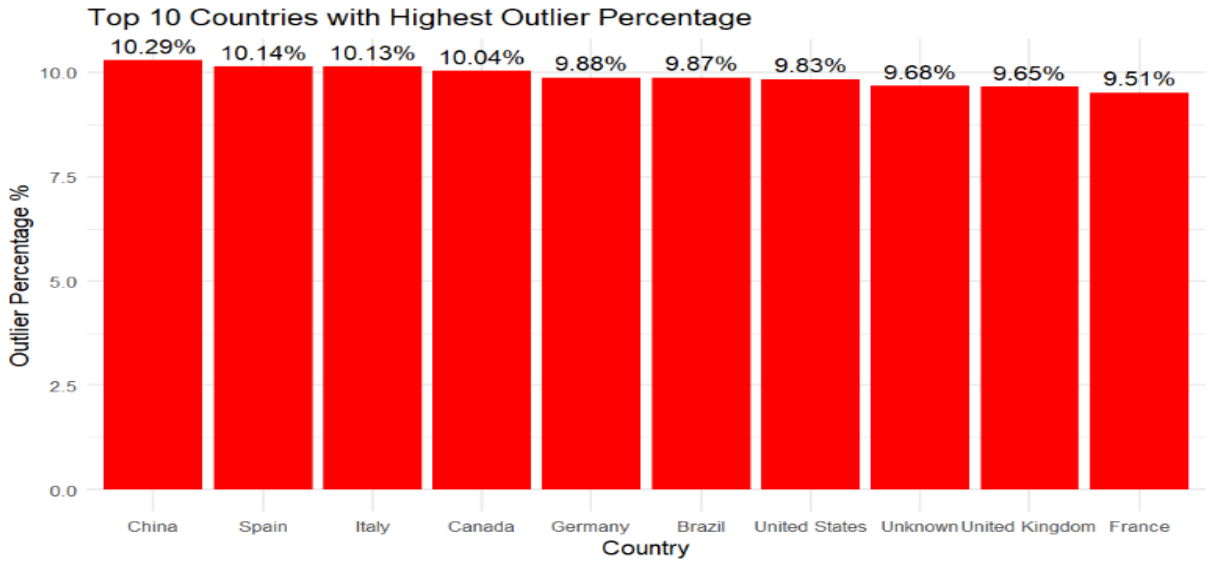


Figure 6.12: The Trend On The Percentage Of Outliers for Top 10 Countries

This plot shows the percentage outliers for the top 10 countries. China suffers the highest outlier percentage. This shows that the country experiences extreme loss events more frequently. Even though in the statistics, we can see that the U.S. had the highest number of total outliers but its percentage outlier is only 9.83%. It is not the highest like that of China. This proves a point, where extreme losses occur at a similar rate as other countries. As for China, Spain, and Italy they have slightly more frequent extreme loss events as their percentage outliers is more the USA where they would have a greater tendency towards financial losses.

Multiple box plots were used to visualize the distribution of "Loss Amount" across the top 10 countries with the highest number of outliers.

```
ggplot(country_outliers %>% filter(Country %in% head(outlier_counts$Country, 10)),
       aes(x = reorder(Country, Loss, median), y = Loss, fill = as.factor(is_outlier))) +
  geom_boxplot(outlier.colour = "red", outlier.size = 2) +
  ggtitle("Loss Distribution with Outliers by Country") +
  xlab("Country") +
  ylab("Loss Amount") +
  scale_fill_manual(name = "Outlier Classification",
                    values = c("black", "red"),
                    labels = c("Normal", "Outlier")) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Figure 6.13: Visualizing The Distribution Of Loss Amount With Country

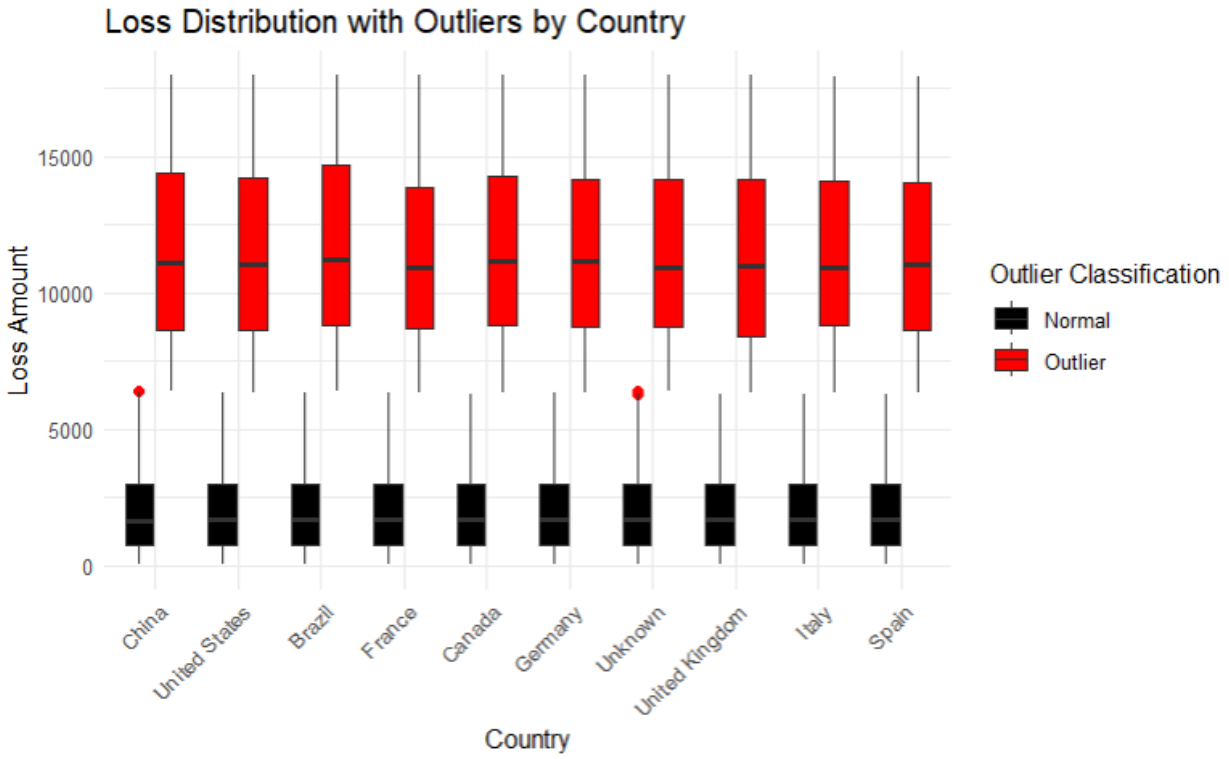


Figure 6.14: The Trend On The Distribution Of Loss Amount With Country

It distinguishes between normal loss values (black boxplots) and outliers (red boxplots). Outliers are substantially higher in loss amount where median loss for outliers (red) is significantly higher than that of normal transactions (black). The outliers show a wider range (longer whisker) indicating high variability. The structure of the red boxplot is quite similar which shows a consistent trend. The red dots indicate individual extreme values present above China And "Unknown", this means that they had significantly higher losses than others in the same group. In conclusion, overall outlier distribution is similar across countries which may indicate that the underlying issue may be systemic rather than region-specific.

3.1.4 Analysis 1-4: Do other factors affect extreme loss values?

A Pearson correlation matrix is displayed

```r
# Merge `summary_stats` with `outlier_counts`
country_features <- summary_stats %>%
  left_join(outlier_counts, by = "Country")

# Compute correlation matrix (excluding Country column)
cor_matrix <- cor(country_features %>% select(-Country), use = "complete.obs")

# Display correlation matrix
print(cor_matrix)
```

Figure 6.15: Conducting Statistical Analysis For Pearson Correlation Matrix

```
                  Total_Loss    Avg_Loss Median_Loss    Loss_SD       Count Total_Outliers  Total_Cases
Total_Loss         1.000000000 0.006370923 -0.02068441 0.04018018 0.999991311     0.99997505 0.999991311
Avg_Loss           0.006370923 1.000000000  0.74574093 0.81321165 0.005806282     0.00663265 0.005806282
Median_Loss       -0.020684415 0.745740933  1.00000000 0.35886746 -0.020901362    -0.02081689 -0.020901362
Loss_SD            0.040180178 0.813211654  0.35886746 1.00000000 0.039577019     0.04049109 0.039577019
Count              0.999991311 0.005806282 -0.02090136 0.03957702 1.000000000     0.99995236 1.000000000
Total_Outliers     0.999975051 0.006632650 -0.02081689 0.04049109 0.999952361     1.00000000 0.999952361
Total_Cases        0.999991311 0.005806282 -0.02090136 0.03957702 1.000000000     0.99995236 1.000000000
Outlier_Percentage 0.051022802 0.171890131 -0.16380675 0.36872820 0.050396674     0.05222831 0.050396674
                  Outlier_Percentage
Total_Loss                0.05102280
Avg_Loss                  0.17189013
Median_Loss              -0.16380675
Loss_SD                   0.36872820
Count                     0.05039667
Total_Outliers            0.05222831
Total_Cases               0.05039667
Outlier_Percentage        1.00000000
```

Figure 6.16: Display Of Values For Pearson Correlation Matrix

Which shows correlation coefficients between different variables related to loss, count, and outliers where it ranges from “-1 to +1”. “+1” shows a perfect positive correlation and “-1” shows a perfect negative correlation.

We then visualize it in a correlation matrix or heat map

```r
# Visualize correlation heat map
ggcorrplot(cor_matrix, method = "circle", lab = TRUE)
```

Figure 6.17: Visualization Of Correlation Pearson Correlation Matrix

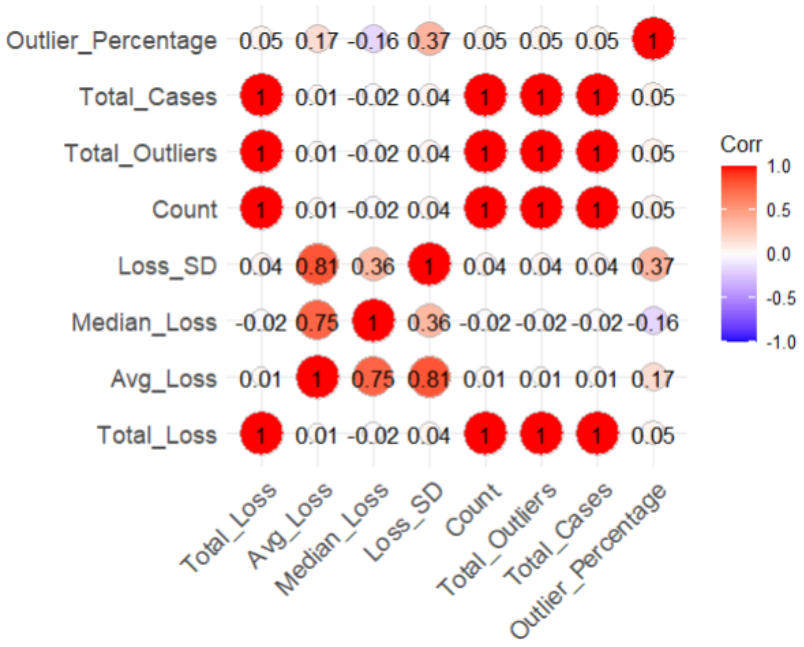


Figure 6.18: The Trend For Pearson Correlation Matrix

That shows a pairwise correlation between different variables related to loss, outliers and case counts. The intensity of the color and size of the circles indicate the strength of the correlation. There is a high correlation between standard deviation and both average loss and median loss which suggests that as the standard deviation of losses increases overall losses also increase. The low correlation between total outliers and total loss suggests that having a high number of outliers does not mean the total loss will be significantly impacted. The total cases and other loss-related variables have zero correlation where having more cases does not necessarily result in higher losses.

Summary Findings

- The United States has the highest total loss and Gambia has the lowest total loss. When we see the trend for the top 20 countries that suffered for least loss and the most loss, we see that Timor-Leste suffered the most loss.
- The findings reveal that the United States experienced the highest number of attacks with the highest revenue loss, whereas Gambia faced only one attack where it experienced the lowest revenue loss. However, after we delve deeper into our findings we discover that revenue loss is not always proportional. For instance, the Netherlands Antilles experienced only one attack but suffered a significant financial loss. Similarly, Germany had more attacks (9,701) than the United Kingdom (8,228), yet both had comparable revenue losses ($26.9M and $22.6M, respectively). This indicates that some attacks cause disproportionately higher financial damage than others.
- In the analysis, we see that the United States has the highest number of total outliers but its percentage is lower compared to other countries. This suggests that the U.S. experiences frequent loss incidents. In contrast, China has the highest percentage of outliers indicating a greater tendency toward extreme financial loss. The outlier analysis tends to showcase significantly higher than normal transactions. This may lead to more systemic issues rather than region-specific.
- The correlation analysis shows a strong relationship between standard deviation and both average and median loss. This indicates high variability in losses leads to increased overall losses. The low correlation between total outliers and total loss suggests that a high number of outliers does not significantly impact loss-related variables where more cases do not lead to higher financial losses.

Extra Feature

```
# Visualize correlation heat map
ggcorrplot(cor_matrix, method = "circle", lab = TRUE)
```

Figure 6.19: Code For Plotting Correlation Pearson Correlation Matrix

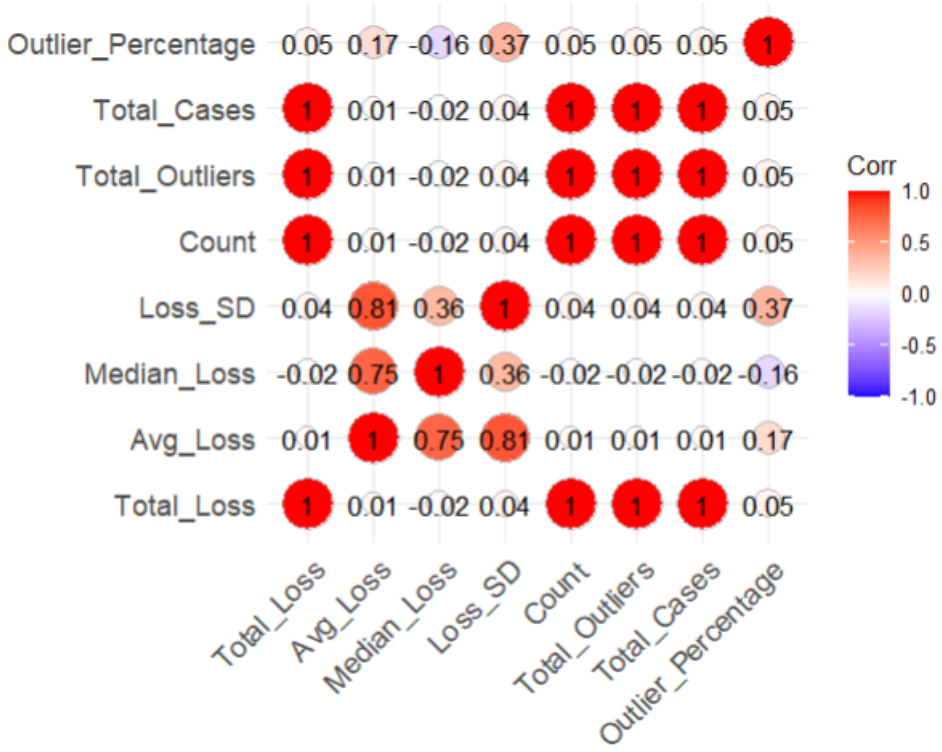


Figure 6.20: Visualization Of Pearson Correlation Matrix

This is a correlation matrix created using ggcorrplot library. It visually represents the correlation between different variables in a dataset. It helps us to visualize which variables are strongly related to one another.

3.2 Objective 2: To examine the relationship between operating systems and losses. (Emmanuel Naranendhya Tyatan | TP078574 | CS-DA)

- Put the dataset into a temporary data frame for further analysis

```
analysis_data <- data.frame(
  Loss = os_data$Loss,
  OS_Groups1 = os_data$OS,
  Date = os_data$Date
)
```

Figure 7.0 Temporary data frame

3.2.1 Analysis 2-1: Which operating systems are associated with highest losses?

Steps for Conducting the First Analysis Question:

a) Group losses by OS type

b) Calculate average and total losses per OS

```
# Analysis 2.1: Which operating system are associated with highest losses?
os_loss_summary <- analysis_data %>%   #Group by OS
  group_by(OS_Groups1) %>%
  summarise(
    total_loss = sum(Loss, na.rm = TRUE),
    avg_loss = mean(Loss, na.rm = TRUE),
    attack_count = n(),
    sd_loss = sd(Loss, na.rm = TRUE)
  ) %>%
  arrange(desc(total_loss))

#Visualize the total loss using bar chart
ggplot(head(os_loss_summary, 10), aes(x = reorder(OS_Groups1, -total_loss), y = total_loss)) +
  geom_bar(stat = "Identity", fill = "steelblue") +
  theme_minimal() +
  labs(
  title = "Top 10 Operating System by Total Loss",
  x = "Operating System",
  y = "Total Loss ($)") +

  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Figure 7.1 Source Code to get which OS has the most financial losses

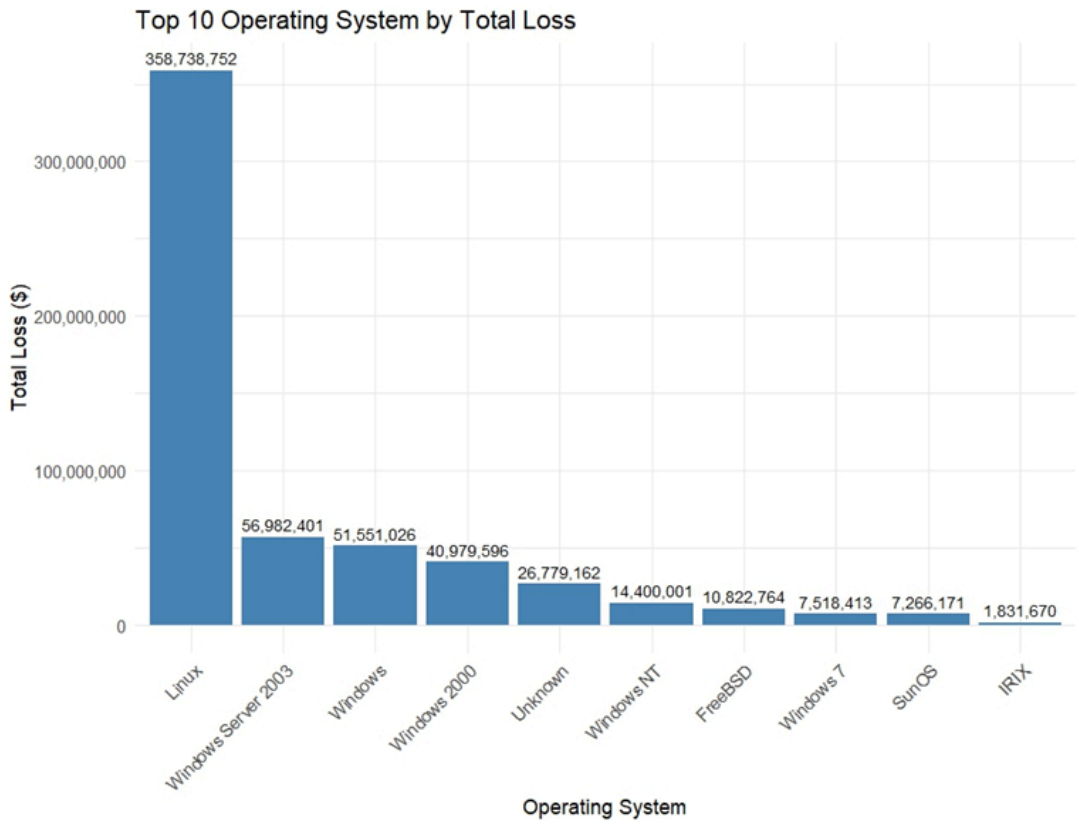


Figure 7.2 Bar chart of top 10 OS by total loss

The bar graph illustrates the spread of monetary loss over various operating systems, beginning with Linux having the highest total loss value of $358.7 million by a wide margin. Following this is Windows Server 2003 with losses of $57 million, and other Windows operating systems also reflecting considerable losses with a combined total of approximately $92.5 million. The gap between Linux and other systems' losses indicates Linux's dominant position in server environments which makes it a prime target for hackers. This trend indicates that market share and deployment environment are large factors in an operating system's exposure to financial loss via cyber attacks.

3.2.2 Analysis 2-2: Has OS vulnerability changed over time?

Steps for Conducting the Second Analysis Question:

a) Analyze OS-related losses across years

```r
# 2.2.1 Analyse OS related losses accross years
yearly_os_losses <- analysis_data %>%
  mutate(Year = format(Date, "%Y")) %>%
  group_by(Year, OS_Groups1) %>%
  summarise(
    total_loss = sum(Loss, na.rm = TRUE),
    avg_loss = mean(Loss, na.rm = TRUE),
    attack_count = n(),
    .groups = "drop"
  ) %>%
  arrange(Year, desc(total_loss))

#Visualization
ggplot(yearly_os_losses,
       aes(x = Year,
           y = total_loss,
           color = OS_Groups1,
           group = OS_Groups1)) +
  geom_line(linewidth = 1) +  # Make lines thicker for better visibility
  geom_point(size = 3) +      # Add points to show exact values
  theme_minimal() +
  labs(
    title = "Financial Impact by Operating System Over Time (1998-2015)",
    x = "Year",
    y = "Total Loss ($)",
    color = "Operating System"
  ) +
  scale_y_continuous(labels = scales::comma) +
  scale_x_discrete(breaks = seq(1998, 2016, 2)) +
  theme(
    legend.position = "bottom",
    legend.text = element_text(size = 8),
    legend.key.size = unit(0.8, "lines"),
    legend.box = "vertical",
    legen.ncol = 2,
    axis.text.x = element_text(angle = 45, hjust = 1, size = 10),
    plot.margin = margin(t = 20, r = 120, b = 20, l = 20),
    legend.spacing.y = unit(0.5, "lines")
  )
```

Figure 7.3 Source Code to get analyse the relation between OS and loss across years

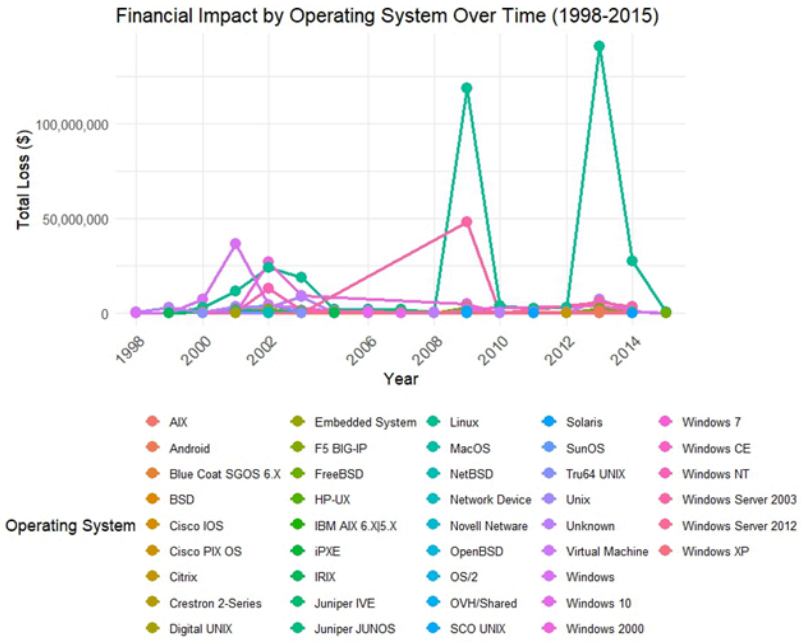


Figure 7.4 Line Chart of financial impact over OS over time

The line graph above shows financial loss against different operating systems between 1998 and 2015 and illustrates evident temporal patterns in the impacts of cyber-attacks. The Linux platforms experienced two significant spikes in loss in 2010 to around $110 million and in 2013 to over $130 million, both being the highest financial impacts ever registered in the database. Windows operating systems, particularly Windows 7, experienced moderate but consistent losses of approximately around 2002, peaking at approximately $35 million. All other operating systems experienced relatively lower and consistent loss patterns during the time. This trend is in line with the previous discovery of Linux's overall dominance in total losses. But now exposing the time distribution of these security breaches

b) Track changes in OS vulnerability patterns

```
# Create year-over-year changes for heatmap
yoy_changes <- analysis_data %>%
  mutate(Year = format(Date, "%Y")) %>%  # Extract year from date
  group_by(Year, OS_Groups1) %>%         # Group by Year and OS
  summarise(
    TotalLoss = sum(Loss, na.rm = TRUE)  # Calculate total loss per year/OS
  ) %>%
  group_by(OS_Groups1) %>%               # Group by OS for YoY calculation
  arrange(Year) %>%
  mutate(
    YoYChange = (TotalLoss - lag(TotalLoss)) / lag(TotalLoss) * 100
  ) %>%
  ungroup()
```

```
# Create heatmap
p2 <- ggplot(yoy_changes, aes(x = Year, y = OS_Groups1, fill = YoYChange)) +
  geom_tile() +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    midpoint = 0,
    name = "Year-over-Year\nChange in Losses (%)",
    limits = c(-100, 100),
    oob = scales::squish
  ) +
  theme_minimal() +
  labs(
    title = "Year-over-Year Changes in OS Vulnerabilities",
    subtitle = "Red indicates increase in losses, blue indicates decrease",
    x = "Year",
    y = "Operating System"
  ) +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1),
    legend.position = "right",
    plot.title = element_text(size = 12, face = "bold"),
    plot.subtitle = element_text(size = 10),
    panel.grid.major = element_blank(),
    panel.grid.minor = element_blank()
  )
p2
```

Figure 7.5 Source Code to track changes in OS vulnerability patterns

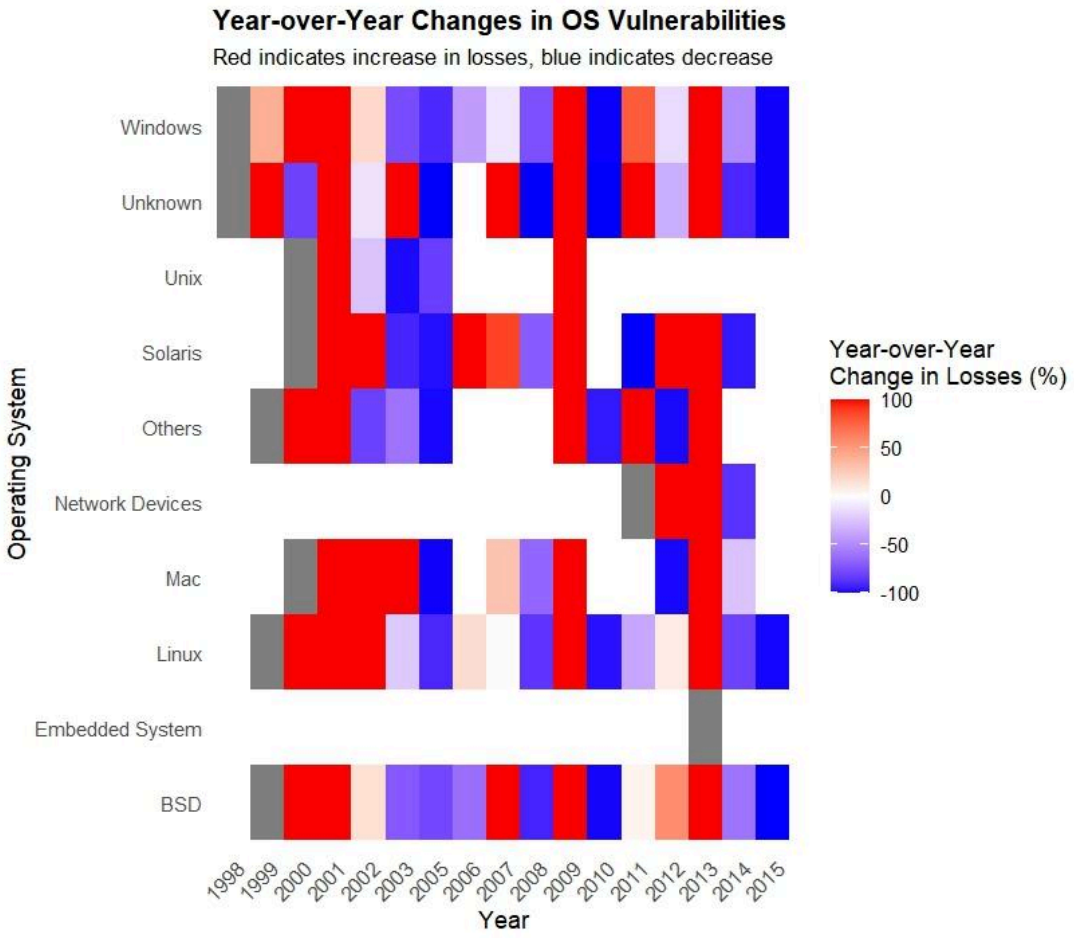


Figure 7.6 Heat Map of OS Vulnerability Analysis

The heatmap graph reveals dynamic trends of operating system vulnerability across time series. Higher losses are signified by dark red squared and lower losses are denoted by blue. The intensity of color increases for higher changes. Linux and Windows have the most dynamic patterns with high proportions of changes from significant increases (dark red) to falls (dark blue) in losses. Most Operating systems show a consistent pattern of loss reduction over the last few years (2014-2015). Suggesting better security measures or threat management by all platforms.

3.2.3 Analysis 2-3: Is there a relationship between OS popularity and loss severity?

Steps for Conducting the Third Analysis Question:

a) Compare OS market share with loss patterns

b) Analyze if popular OS face higher losses

```
# Analysis 3: Is there a relationship between OS popularity and loss severity?
# Compare OS market share with loss pattern
os_market_share <- analysis_data %>%
  group_by(OS_Groups1) %>%
  summarise(
    attack_count= n(),
    market_share = n() / nrow(data) * 100,
    total_loss = sum(Loss, na.rm = TRUE),
    avg_loss = mean(Loss, na.rm = TRUE)
    ) %>%
  arrange(desc(market_share))

# Analyze if popular OS face higher Losses
popularity_loss <- os_market_share %>%
  mutate(
    loss_per_share = total_loss / market_share,
    popularity_category = case_when(
      market_share > mean(market_share) + sd(market_share) ~ "Very Popular",
      market_share > mean(market_share) ~ "Popular",
      market_share < mean(market_share) - sd(market_share) ~ "Rare",
      TRUE ~ "Less Popular"
    )
  )

#Visualization
ggplot(popularity_loss, aes(x = market_share, y = total_loss)) +
  geom_point(aes(size = attack_count, color = popularity_category), alpha = 0.7) +
  scale_size_continuous(range = c(4,15)) +
  scale_y_continuous(
    labels = scales::comma_format(),
    breaks = seq(0, 3e8, by = 5e7)
  ) +
  theme_minimal() +
  labs(
    title = "OS Market Share vs Losses Analysis",
    subtitle = "Bubble Size: Attack Count",
    x = "Market Share (%)",
    y = "Total Loss",
    size = "Attack Count",
    color = "Popularity Category"
  )
```

Figure 7.7 Source Code to compare OS with loss patterns

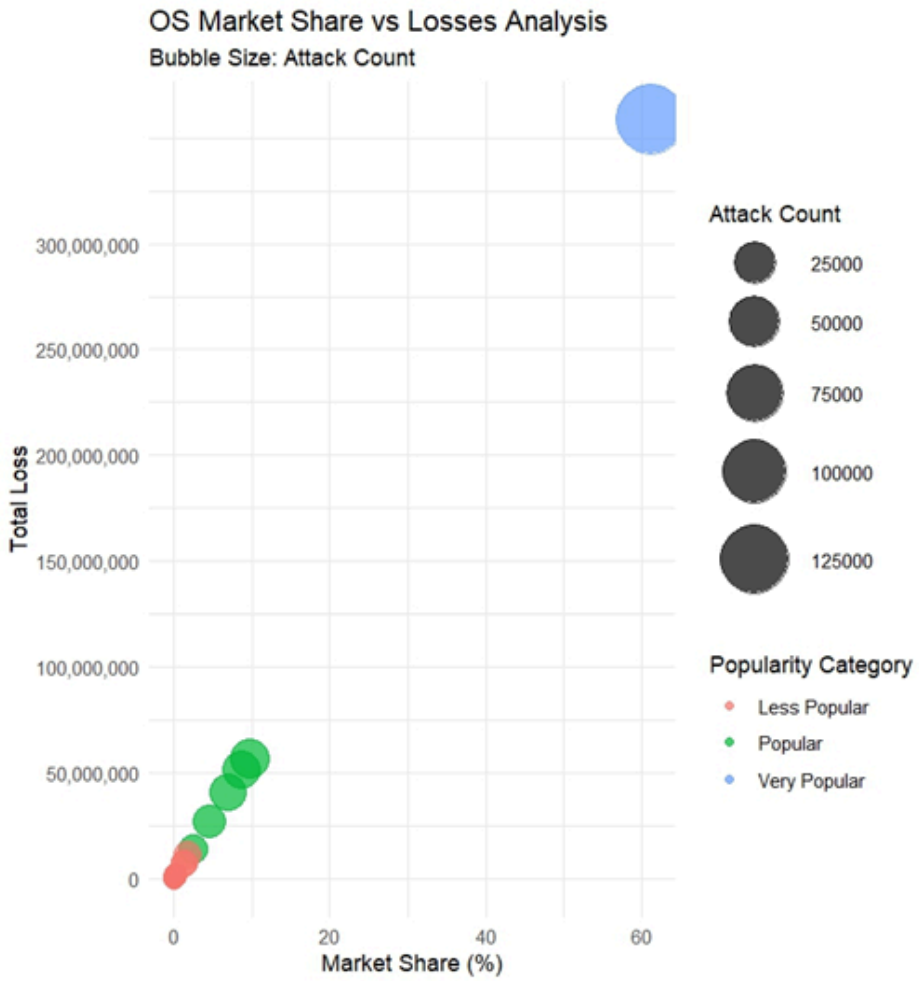


Figure 7.8 Bubble Chart of comparison between OS market share and losses analysis

In this analysis, the visualization of the bubble chart (Figure 7.8) reveals that operating systems with higher market share are proportionally more vulnerable to security threats. The most used OS has around 60% market share and highest losses (large blue bubble). Whereas popular system have moderate exposure (green bubble with 20% share), and less popular system cluster at minimal market shares and losses (small red bubble).

3.2.4 Analysis 2-4: How does OS Patch Level Affect Loss?

Steps for Conducting the Fourth Analysis Question:

a) Group losses by OS update status

```
# Analysis 4: How does OS patch level affect financial losses
# Identify version numbers and patch status
os_version_analysis <- analysis_data %>%
  mutate(
    version_number = str_extract(OS_Groups1, "\\d+(\\.\\d+)*"),
    patch_status = case_when(
      !is.na(version_number) ~ "Patched",
      TRUE ~ "Unpatched"
    )
  ) %>%
  group_by(OS_Groups1, patch_status) %>%
  summarise(
    total_loss = sum(Loss,na.rm = TRUE),
    avg_loss = mean(Loss, na.rm = TRUE),
    attack_count = n()
  )

print(os_version_analysis)
```

Figure 7.9 Source Code to analyse how OS patch level affect financial losses

```
# A tibble: 42 × 5
# Groups:   OS_Groups1 [42]
   OS_Groups1         patch_status total_loss avg_loss attack_count
   <chr>              <chr>             <dbl>    <dbl>        <int>
 1 AIX                Unpatched       749710.    2929.          256
 2 Android            Unpatched          615      615             1
 3 BSD                Unpatched      1774005.    2620.          677
 4 Blue Coat SGOS 6.X Patched            1767.    1767.            1
 5 Cisco IOS          Unpatched          1860      930             2
 6 Cisco PIX OS       Unpatched          2081     2081             1
 7 Citrix             Unpatched         80544.    3356.           24
 8 Crestron 2-Series  Patched           24529     4088.            6
 9 Digital UNIX       Unpatched         15869     1443.           11
10 Embedded System    Unpatched        318759.    2724.          117
# i 32 more rows
```

Figure 7.10 List of top 10 OS patch status with financial loss status

In here, we can see the first 10 rows indicate concerning facts that all these systems are mostly running without crucial security fixes. Starting with BSD systems that shows significant vulnerability with total losses of $1,774,005 from 667 attacks. Only 2 systems maintain patched status with current security updates, highlighting a significant security weakness across most platforms.

b) Compare losses between patched vs unpatched systems

```
# Compare Losses between patched and unpatched system
patch_comparison <- os_version_analysis %>%
  group_by(patch_status) %>%
  summarise(
    total_loss = sum(total_loss),
    avg_loss = mean(avg_loss),
    attack_count = sum(attack_count)
  )

#Visualization
ggplot(os_version_analysis,
       aes(x = OS_Groups1, y = avg_loss, fill = patch_status)) +
  geom_bar(stat = "identity", position = "dodge") +
  theme_minimal() +
  labs(
    title = "Average Losses: Patched vs Unpatched Systems",
    x = "Operating System Group",
    y = "Average Loss ($)",
    fill = "Patch Status"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Figure 7.11 Source Code that compares losses between OS patched and unpatched system

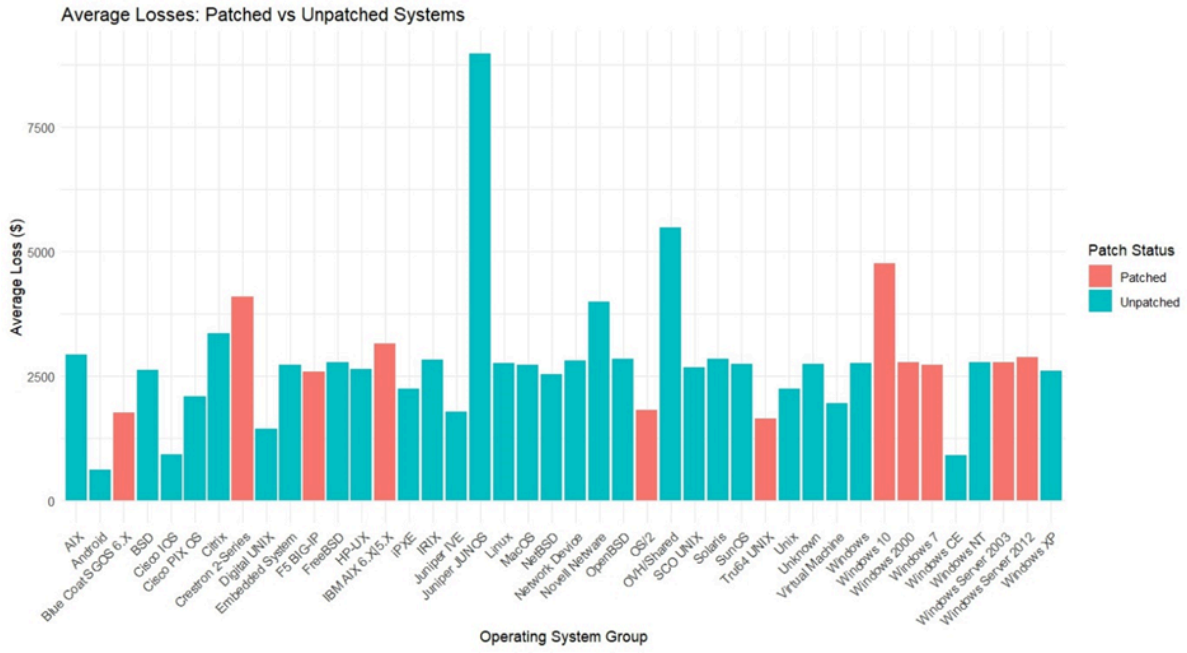


Figure 7.12 Bar Chart indicate patched and unpatched OS versions

This bar chart compares average loss to unpatched and patched operating systems. It can be tell that this data shows insignificant patterns, logically the patched more system is the lesser you has revenue loss. This pattern shows some patched OS versions still got high loss, even though it is safe in security wise, it might give the possibility that there are other factors that guarantee lower financial impact from attacks.

Summary Findings

- In Analysis 2-1, the Linux system exhibited the highest financial vulnerability, with total losses of $358.7 million, significantly higher than other operating systems. Windows Server 2003 ranked second at $57M, indicating that market-dominant systems are proportionally more vulnerable.
- Analysis 2-2 said temporal analysis identified two peak loss spikes in the Linux system during 2010 ($110M) and 2013 ($130M), whereas the Windows system experienced steady but moderate losses. Surprisingly, all systems showed declining losses after 2014, suggesting improved security measures.
- Analysis 2-3 shows the market share analysis is strongly linked to attack and loss frequency. More frequently used systems with greater market share experienced proportionally greater losses and amount of attacks, whereas lesser known systems incurred minimal economic losses
- Analysis 2-4 illustrates that most systems do not have security patches installed with them, only 11 of the systems are in the patched state which mostly are Windows. This general lack of patching has engaged substantial security issues across platforms

3.3 Objective 3: The find the correlation between downtime and loss (Lau Wei Wen| TP079439 | CS-DA)

```
#---------------Data Import--------------------
#Set Path
getwd()
setwd("/Users/weiwen/Documents/PFDA/assignment")
getwd()

#View File List
list.files()

#Import Raw Dataset
raw_df <- read.csv("Clean_Loss_Data.csv", header = TRUE)



#Show the dataset details
head(raw_df)
str(raw_df)
summary(raw_df)
```

Figure 8.0 Importing dataset

```
#---------------Data Cleaning--------------------
#Install Packages
InstallPackages <- function(){
  install.packages("dplyr")
  install.packages("ggplot2")
  install.packages("tidyr")
  install.packages("stringr")
  install.packages("scales")
  install.packages("lubridate")

}
InstallPackages()

#Load Package
LoadPackages <- function(){
  library(dplyr)
  library(ggplot2)
  library(tidyr)
  library(stringr)
  library(scales)
  library(lubridate)

}
LoadPackages()
```

Figure 8.1 Importing necessary libraries

```
#Initial data structure
glimpse(raw_df)

#removing unnecessary columns
cl_df_ <- raw_df %>%
  select(-c(WebServer, Lang, Ransom))

# Replace empty strings, "NULL", "UNKNOWN", and 0 with NA
cl_df_[cl_df_ == ""] <- NA
cl_df_[cl_df_ == "NULL"] <- NA
cl_df_[cl_df_ == "UNKNOWN"] <- NA
cl_df_[cl_df_ == 0] <- NA
cl_df_[cl_df_ == "Unknown"] <- NA

# Convert DownTime to numeric
cl_df_$DownTime <- as.numeric(cl_df_$DownTime)
```

Figure 8.2 Data cleaning (1)

```
> # Check for missing values
> missing_values <- sum(is.na(cl_df_$DownTime))
> print(missing_values)
[1] 0
>
> # Check for negative or zero values
> invalid_values <- cl_df_ %>% filter(DownTime <= 0)
> print(invalid_values)
[1] Date     Notify   URL      IP       Country  OS       Encoding DownTime Loss
<0 rows> (or 0-length row.names)
```

Figure 8.3 Data cleaning (2)

```
> # Remove extreme outliers (e.g., using 1.5*IQR rule)
> Q1 <- quantile(cl_df_$DownTime, 0.25)
> Q3 <- quantile(cl_df_$DownTime, 0.75)
> IQR_value <- Q3 - Q1
> lower_bound <- Q1 - 1.5 * IQR_value
> upper_bound <- Q3 + 1.5 * IQR_value
>
> # Filter out extreme outliers
> df_clean <- cl_df_ %>% filter(DownTime >= lower_bound & DownTime <= upper_bound)
>
> # Display cleaned summary
> summary(df_clean$DownTime)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   7.00   20.00   32.00   31.89   43.00   60.00
>
> # Save cleaned data
> write.csv(df_clean, "hackingData_cleaned.csv", row.names = FALSE)
```

Figure 8.4 Data cleaning (3)

3.3.1 Analysis 3-1: What patterns exist between downtime duration and financial losses?

```
> #Analysis 3-1
> #(a) Group losses by downtime ranges
> df_clean <- df_clean %>%
+   mutate(Downtime_Category = case_when(
+     DownTime < 1 ~ "Short (<1 hour)",
+     DownTime >= 1 & DownTime < 5 ~ "Moderate (1-5 hours)",
+     DownTime >= 5 & DownTime < 24 ~ "Long (5-24 hours)",
+     DownTime >= 24 ~ "Critical (24+ hours)"
+   ))
>
> # View downtime categories
> table(df_clean$Downtime_Category)

Critical (24+ hours)    Long (5-24 hours)
              145086                67007
```

Figure 8.5 Group losses by downtime ranges

```
> #(b) Calculate average and total losses per downtime period
> downtime_loss_summary <- df_clean %>%
+   group_by(Downtime_Category) %>%
+   summarise(
+     Avg_Loss = mean(Loss, na.rm = TRUE),
+     Total_Loss = sum(Loss, na.rm = TRUE),
+     Count = n()
+   ) %>%
+   arrange(desc(Total_Loss))
>
> print(downtime_loss_summary)
# A tibble: 2 × 4
  Downtime_Category    Avg_Loss Total_Loss  Count
  <chr>                   <dbl>      <dbl>  <int>
1 Critical (24+ hours)    3727. 540770298. 145086
2 Long (5-24 hours)        674.  45182391.  67007
```

Figure 8.6 Calculate average and total losses per downtime period

```
#(c) Identify critical downtime thresholds
ggplot(df_clean, aes(x = DownTime, y = Loss)) +
  geom_point(alpha = 0.5, color = "blue") +
  geom_smooth(method = "lm", color = "red") +
  labs(title = "Downtime vs. Financial Loss", x = "Downtime (hours)", y = "Loss ($)") +
  theme_minimal()
```

Figure 8.7 Identify critical downtime thresholds

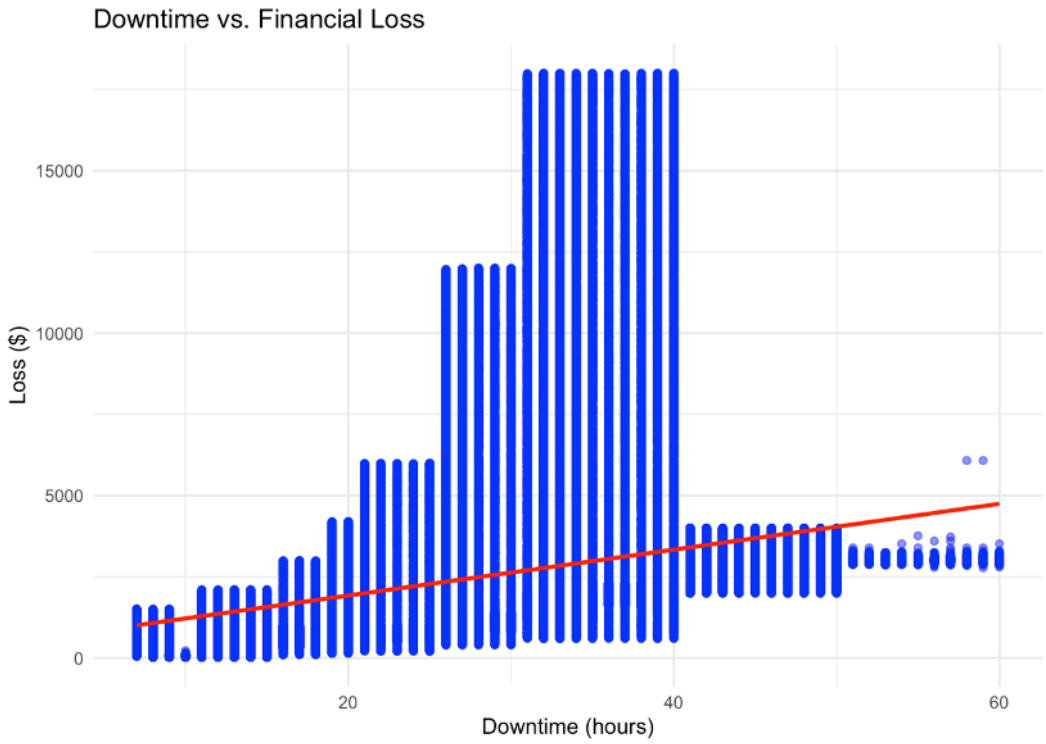


Figure 8.8: Output of Figure 8.7

This scatter plot in figure 8.8 illustrates the correlation between downtime length and financial loss to ascertain if prolonged downtime results in increased losses. The red line indicates a linear regression trend illustrating the overall association, whereas each blue point signifies an individual data input. The ascending trajectory of the red regression line in the graphic indicates a positive correlation between downtime and financial loss. This indicates that prolonged downtime generally results in greater financial loss. The point distribution, however, reveals outliers, indicating significant losses at shorter outage durations, suggesting further contributing factors. By identifying these abnormalities, organizations can formulate more efficient risk mitigation strategies that minimize adverse financial impacts.

3.3.2 Analysis 3-2: How has the downtime-loss relationship changed over time?

```
> #Analysis 3-2
> #(a) Analyze downtime-related losses across years
> df_clean$Year <- year(df_clean$Date)
>
> yearly_loss_summary <- df_clean %>%
+   group_by(Year) %>%
+   summarise(
+     Avg_Downtime = mean(DownTime, na.rm = TRUE),
+     Total_Loss = sum(Loss, na.rm = TRUE)
+   )
>
> print(yearly_loss_summary)
# A tibble: 17 × 3
    Year Avg_Downtime Total_Loss
   <dbl>        <dbl>      <dbl>
 1  1998         35.2    203882.
 2  1999         31.6   2696453.
 3  2000         32.1  11740943.
 4  2001         31.9  57490296.
 5  2002         31.8  78729330.
 6  2003         31.9  38912603.
 7  2005         32.1   2929014.
 8  2006         31.7   2687878.
 9  2007         32.0   2835424.
10  2008         28.5    457645.
11  2009         31.9 178944916.
12  2010         32.1   4051330.
13  2011         32.2   3024087.
14  2012         32.3   3130487.
15  2013         31.9 163166086.
16  2014         32.0  34623504.
17  2015         31.9    328812
```

Figure 8.9 Analyze downtime-related losses across years

```
#(b) Track changes in downtime impact patterns
df_clean %>%
  group_by(Year) %>%
  summarise(Avg_Loss = mean(Loss, na.rm = TRUE)) %>%
  ggplot(aes(x = Year, y = Avg_Loss)) +
  geom_line(color = "blue", linewidth = 1) +
  labs(title = "Trend of Average Loss Over Time", x = "Year", y = "Average Loss") +
  theme_minimal()
```

Figure 8.10 Track changes in downtime impact patterns

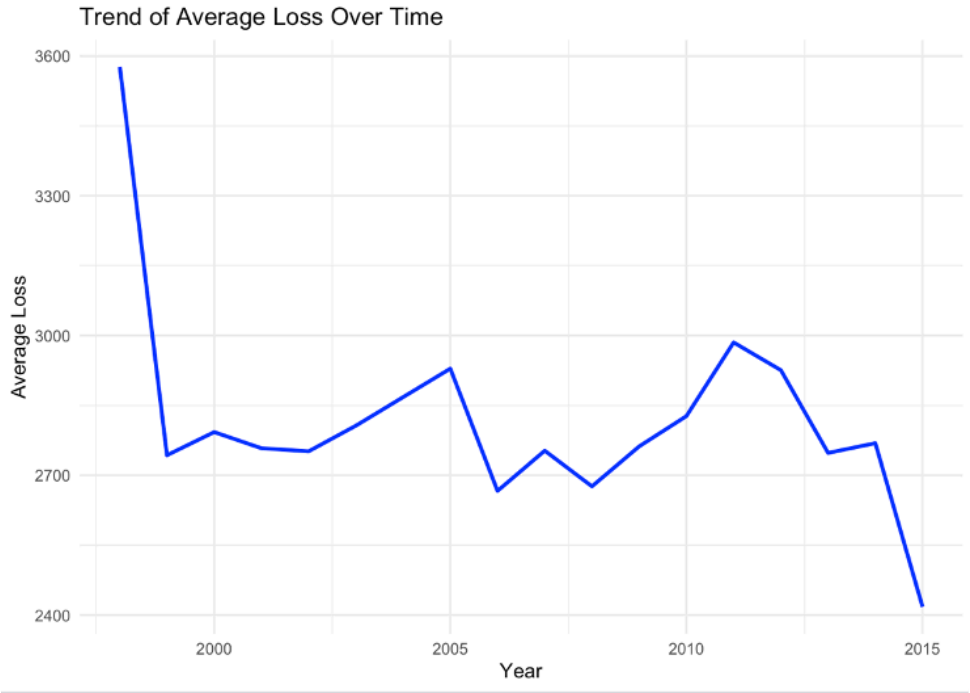


Figure 8.11: Output of Figure 8.10

This method analyzes the temporal trend of average financial losses to identify patterns or fluctuations in financial impact. The annual average financial loss was calculated subsequent to the categorization of the sample by year. A line graph was next used to show trends in the obtained figures. The graph illustrates that the average financial loss fluctuates over time, reaching its highest point in 1998 before exhibiting variability in subsequent years. The decline in recent years suggests the potential for improved risk reduction or downtime management strategies. However, there are instances when financial losses increase, potentially signaling significant events that increase the expense of downtime. This trend analysis helps the comprehension of long-term patterns and the formulation of future strategies to reduce financial harm.

```
> #(c) Identify Trends in Downtime Severity
> df_clean <- df_clean %>%
+   mutate(Downtime_Severity = case_when(
+     DownTime < 1 ~ "Low (<1 hr)",
+     DownTime >= 1 & DownTime < 5 ~ "Medium (1-5 hrs)",
+     DownTime >= 5 & DownTime < 24 ~ "High (5-24 hrs)",
+     DownTime >= 24 ~ "Critical (24+ hrs)"
+   ))
>
> # Check distribution
> table(df_clean$Downtime_Severity)

Critical (24+ hrs)    High (5-24 hrs)
            145086              67007
>
> #Severity over time
> downtime_severity_trends <- df_clean %>%
+   group_by(Year, Downtime_Severity) %>%
+   summarise(Count = n()) %>%
+   ungroup()
`summarise()` has grouped output by 'Year'. You can override using the `.groups` argument.
>
> print(downtime_severity_trends)
# A tibble: 34 x 3
    Year Downtime_Severity  Count
   <dbl> <chr>              <int>
 1  1998 Critical (24+ hrs)    41
 2  1998 High (5-24 hrs)       16
 3  1999 Critical (24+ hrs)   656
 4  1999 High (5-24 hrs)      327
 5  2000 Critical (24+ hrs)  2905
 6  2000 High (5-24 hrs)     1299
 7  2001 Critical (24+ hrs) 14260
 8  2001 High (5-24 hrs)     6584
 9  2002 Critical (24+ hrs) 19477
10  2002 High (5-24 hrs)     9136
# i 24 more rows
```

Figure 8.12 Identify trends in downtime severity

3.3.3 Analysis 3-3: How does downtime affect overall business loss?

```
#Analysis 3-3
#(a) Categorize downtime into different ranges
df_clean <- df_clean %>%
  mutate(Downtime_Category = case_when(
    DownTime < 1 ~ "Short (<1 hour)",
    DownTime >= 1 & DownTime < 5 ~ "Moderate (1-5 hours)",
    DownTime >= 5 & DownTime < 24 ~ "Long (5-24 hours)",
    DownTime >= 24 ~ "Critical (24+ hours)"
  ))
```

Figure 8.13 Categorize downtime into different ranges

```
#(b) Calculate the average and total loss per downtime range
df_loss_summary <- df_clean %>%
  group_by(Downtime_Category) %>%
  summarise(
    Avg_Loss = mean(Loss, na.rm = TRUE),
    Total_Loss = sum(Loss, na.rm = TRUE),
    Count = n()
  )
```

Figure 8.14 Calculate the average and total loss per downtime range

```
> #(c) Identify which downtime category contributes the most to total losses
> df_loss_summary %>%
+   arrange(desc(Total_Loss))
# A tibble: 2 × 4
  Downtime_Category    Avg_Loss Total_Loss  Count
  <chr>                   <dbl>      <dbl>  <int>
1 Critical (24+ hours)    3727. 540770298. 145086
2 Long (5-24 hours)        674.  45182391.  67007
```

Figure 8.15 Identify which downtime category contributes the most to total losses

The data in figure 8.15 indicates that significant downtimes (exceeding 24 hours) are the primary contributors to financial losses, amounting to $540,770,298 across 145,086 events. The average loss incurred from each critical outage is $3,727, markedly exceeding that of lesser downtime intervals. In contrast, extended downtimes (5-24 hours) result in a total loss of $45,182,391 across 67,007 events, yielding an average loss of $674 per occurrence. This indicates that although prolonged downtimes remain significant, the cost repercussions escalate considerably for longer interruptions. The results underscore the necessity for proactive methods to mitigate downtime, as extended downtimes result in significantly greater financial losses.

3.3.4 Analysis 3-4: How has the financial impact of downtime evolved over time?

```
#Analysis 3-4
#(a) Analyze the trend of downtime-related losses across years
df_yearly <- df_clean %>%
  mutate(Year = year(Date)) %>%
  group_by(Year) %>%
  summarise(
    Avg_Downtime = mean(DownTime, na.rm = TRUE),
    Avg_Loss = mean(Loss, na.rm = TRUE),
    Total_Loss = sum(Loss, na.rm = TRUE)
  )
```

Figure 8.16 Analyze the trend of downtime-related losses across years

```
#(b) Compare yearly downtime trends with financial impact
ggplot(df_yearly, aes(x = Year)) +
  geom_col(aes(y = Avg_Loss), fill = "blue", alpha = 0.6) +
  geom_line(aes(y = Avg_Downtime * 10000), color = "red", size = 1) +
  labs(
    title = "Yearly Trend of Downtime and Financial Loss",
    x = "Year",
    y = "Financial Loss ($)",
    caption = "Red line: Avg Downtime (scaled up for visibility)"
  ) +
  theme_minimal()
```

Figure 8.17 Compare yearly downtime trends with financial impact

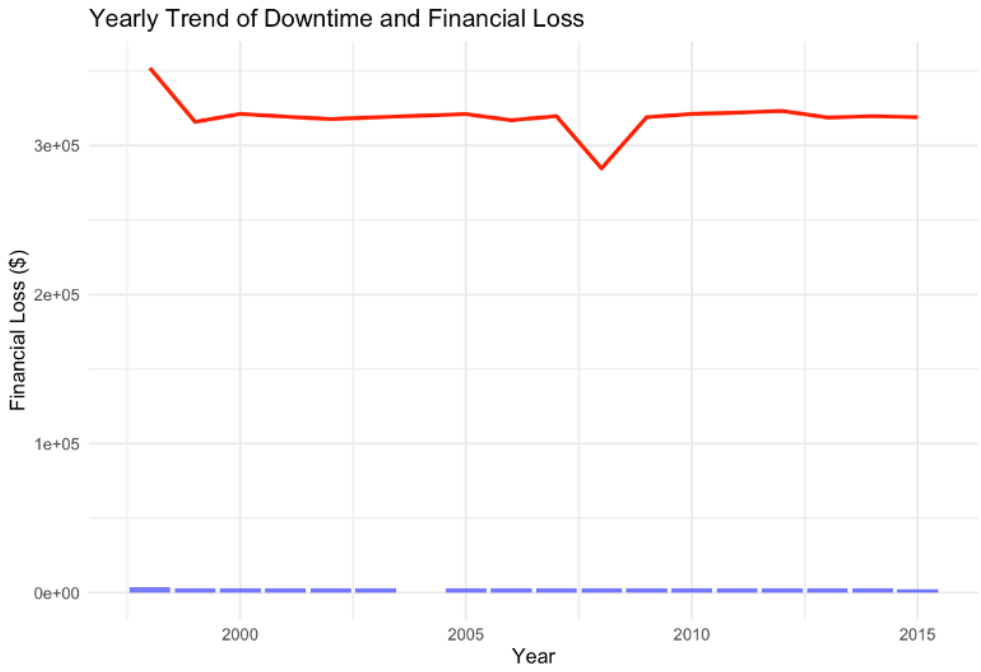


Figure 8.18: Output of Figure 8.17

To determine the annual average financial losses and identify the years with the most significant financial repercussions due to downtime, the dataset was classified by year to ascertain the average annual loss. The five years with the most significant financial impact were identified by arranging the data in descending order based on the average loss. The statistics indicate that the years with the most significant financial losses due to downtime were 1998, 2011, 2005, 2012, and 2010. These data suggest that cybersecurity attacks or system failures may have adversely impacted financial stability during these periods.

```
> #(c) Identify years where downtime had the most significant financial impact
> df_yearly %>%
+   arrange(desc(Avg_Loss)) %>%
+   head(5)  # Show the top 5 years with the highest average loss
# A tibble: 5 × 4
   Year Avg_Downtime Avg_Loss Total_Loss
  <dbl>        <dbl>    <dbl>      <dbl>
1  1998         35.2    3577.    203882.
2  2011         32.2    2985.   3024087.
3  2005         32.1    2929.   2929014.
4  2012         32.3    2926.   3130487.
5  2010         32.1    2827.   4051330.
>
```

Figure 8.19 Identify years where downtime had the most significant financial impact

We examined the average annual financial loss in figure 8.18 to identify the years when downtime had the biggest financial impact. The average loss was calculated after the dataset was categorized by year. We determined the top five years with the greatest financial impact by arranging the findings in descending order by average loss. In 1998, the average loss per incident was the highest at $3,577, with an average downtime of 35.2 hours. The years 2011, 2005, and 2012 followed closely, each with an average downtime of approximately 32 hours and losses ranging from $2,900 to $3,000 per incident. While 2010 had a slightly lower average loss of $2,827, it recorded the highest total loss of $4,051,330, suggesting a higher frequency of incidents. By identifying key years when downtime had a major impact on financial losses, this analysis helps identify trends and develop risk-reduction plans.

Summary Findings

- In analysis 3-1, the upward slope of the red regression line in figure 8.9 shows a positive correlation between downtime and financial loss. This suggests that extended downtime typically leads to increased financial losses.
- In analysis 3-2, the graph in figure 8.12 demonstrates that the average financial loss changes over time, peaking in 1998 before displaying fluctuations in the following years. The recent decline indicates the possibility of enhanced risk reduction or downtime management methods.
- In analysis 3-3, prolonged downtimes (5-24 hours) result in a total loss of $45,182,391 across 67,007 incidents, producing an average loss of $674 per incident. This suggests that while extended downtimes are significant, the financial consequences increase considerably with longer disruptions.
- In analysis 3-4, the five years with the most notable financial impact were determined by arranging the data in figure 8.19 in descending order according to the average loss. The data reveal that the years with the greatest substantial revenue losses attributed to downtime were 1998, 2011, 2005, 2012, and 2010. The statistics indicate that cybersecurity attacks or system breakdowns may have negatively affected financial stability during these intervals.

3.4 Objective 4: To determine if web server versions affect financial losses from attacks (Yuke Liew Nyuk Kee | TP079074 | CS-DA)

```
getwd()
setwd("C:/Users/yukel/OneDrive/Documents/PFDA/assignment")
asm_data <- read.csv("Clean_Loss_Data.csv", encoding = "ISO-8859-1")

install.packages(c("dplyr", "ggplot2", "corrplot", "maps", "reshape2"))
install.packages("ggcorrplot")
install.packages("tidyr")
```

Figure 9.0 Importing the dataset & installing packages

```
library(dplyr)
library(ggplot2)
library(corrplot)
library(maps)
library(reshape2)
library(ggcorrplot)
library(tidyr)
library(stringr)
library(scales)
library(lubridate)

View(asm_data)
nrow(asm_data)
ncol(asm_data)
head(asm_data)
tail(asm_data)
str(asm_data)
colnames(asm_data)
class(asm_data)
summary(asm_data)
```

Figure 9.1 Loading the libraries

```
> nrow(asm_data)
[1] 212093
> ncol(asm_data)
[1] 12
> head(asm_data)
       Date         Notify                          URL   IP       Country      OS WebServer     Encoding            Lang Ransom DownTime    Loss
1 1998-01-02  Team CodeZero http://www.janet-jackson.com <NA>       UNKNOWN Unknown   Unknown        utf-8            NULL     NA       18   678.0
2 1998-01-03          Feliz     http://cariari.ucr.ac.cr <NA>     COSTARICA Unknown   Unknown         NULL            NULL     NA       31  1988.0
3 1998-01-04 Optiklenz(LOU)       http://marin.k12.ca.us <NA> UNITED STATES Unknown   Unknown   iso-8859-1       Helvetica   2034       35 12204.0
4 1998-01-04  Team CodeZero         http://www.dm.af.mil <NA>   AFGHANISTAN Unknown   Unknown        utf-8            NULL     NA       58  6078.3
5 1998-01-04  Team CodeZero    http://www.bolling.af.mil <NA>   AFGHANISTAN Unknown   Unknown windows-1252 Times New Roman     NA       59  6078.3
6 1998-01-04       foxy man        http://hope.hinet.net <NA>       UNKNOWN Unknown   Unknown        utf-8            NULL   2316       34 13896.0
> tail(asm_data)
            Date        Notify                                        URL            IP      Country    OS WebServer Encoding Lang Ransom DownTime    Loss
212088 2015-09-01 4Ri3 6Ondr0n9      http://ofertapatos.com.br/xxx.htm 177.154.48.130 SouthAmerica Linux    Apache    utf-8 <NA>   2723       31 16338.0
212089 2015-09-01         dowoh  http://kabarkalteng.info/wp-login.php  202.43.182.28       Unknown Linux    Apache    utf-8 <NA>     NA       58  3127.9
212090 2015-09-01 josef paradox       http://www.vakinankaratra.gov.mg  41.190.237.22        Africa Linux    Apache    utf-8 <NA>     NA       44  3930.0
212091 2015-09-01 4Ri3 6Ondr0n9 http://valasysb2bmarketing.com/xxx.htm 66.199.227.122       Unknown Linux    Apache    utf-8 <NA>     NA       37  1729.0
212092 2015-09-01   BlueTornado                 http://cercialani.com   72.29.72.141       Unknown Linux    Apache    utf-8 <NA>    146        7    73.0
212093 2015-09-01 4Ri3 6Ondr0n9        http://shoppingsukh.com/xxx.htm 66.199.227.122       Unknown Linux    Apache    utf-8 <NA>   1947       26  7788.0
> str(asm_data)
'data.frame':   212093 obs. of  12 variables:
 $ Date     : chr  "1998-01-02" "1998-01-03" "1998-01-04" "1998-01-04" ...
 $ Notify   : chr  "Team CodeZero" "Feliz" "Optiklenz(LOU)" "Team CodeZero" ...
 $ URL      : chr  "http://www.janet-jackson.com" "http://cariari.ucr.ac.cr" "http://marin.k12.ca.us" "http://www.dm.af.mil" ...
 $ IP       : chr  NA NA NA NA ...
 $ Country  : chr  "UNKNOWN" "COSTARICA" "UNITED STATES" "AFGHANISTAN" ...
 $ OS       : chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ WebServer: chr  "Unknown" "Unknown" "Unknown" "Unknown" ...
 $ Encoding : chr  "utf-8" "NULL" "iso-8859-1" "utf-8" ...
 $ Lang     : chr  "NULL" "NULL" "Helvetica" "NULL" ...
 $ Ransom   : int  NA NA 2034 NA NA 2316 195 NA 2877 1070 ...
 $ DownTime : int  18 31 35 58 59 34 21 37 28 21 ...
 $ Loss     : num  678 1988 12204 6078 6078 ...
> colnames(asm_data)
 [1] "Date"      "Notify"    "URL"       "IP"        "Country"   "OS"        "WebServer" "Encoding"  "Lang"      "Ransom"    "DownTime"  "Loss"
> class(asm_data)
[1] "data.frame"
> summary(asm_data)
     Date              Notify              URL                IP              Country             OS              WebServer           Encoding            Lang              Ransom
 Length:212093      Length:212093      Length:212093      Length:212093      Length:212093      Length:212093      Length:212093      Length:212093      Length:212093      Min.   : 100
 Class :character   Class :character   Class :character   Class :character   Class :character   Class :character   Class :character   Class :character   Class :character   1st Qu.: 817
 Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Mode  :character   Median :1543
                                                                                                                                                                             Mean   :1546
                                                                                                                                                                             3rd Qu.:2272
                                                                                                                                                                             Max.   :3000
                                                                                                                                                                             NA's   :160000
    DownTime          Loss
 Min.   : 7.00   Min.   :   30
 1st Qu.:20.00   1st Qu.:  804
 Median :32.00   Median : 1904
 Mean   :31.89   Mean   : 2763
 3rd Qu.:43.00   3rd Qu.: 3013
 Max.   :60.00   Max.   :18000
```

Figure 9.2 General Information of data

3.4.1 Analysis 4-1: Which web servers experience the highest losses?

```
#Q1
# Step 1: Group losses by web server type
loss_by_server <- asm_data %>%
  group_by(WebServer) %>%
  summarize(
    total_loss = sum(Loss),
    avg_loss = mean(Loss),
    num_attacks = n()
  )
View(loss_by_server)
# Sort by total loss in descending order
most_vulnerable <- loss_by_server %>%
  arrange(desc(total_loss))
# Print the top 10 most vulnerable servers
webserver_loss<- head(most_vulnerable, 5)
webserver_loss

# Step 2: Visualize the top 10 using a bar chart with y-axis in millions
Q1_chart <- ggplot(webserver_loss, aes(x =total_loss / 1e6, y = reorder(WebServer, total_loss))) +
  geom_col(fill = "pink") +
  geom_text(aes(label = sprintf("%.2f", total_loss/1e6)), hjust = -0.1, size = 3.0)+
  labs(title = "Top 5 Web Servers by Total Financial Loss (in Millions)",
    x = "Total Loss (Millions)", y = "Web Server" ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
#Step 3: Print the chart
print(Q1_chart)
```

Figure 9.3 Source Code to find Top 5 Web Server which have most financial Loss

	WebServer	total_loss	avg_loss	num_attacks
1	apache	315645073.3	2762.903	114244
2	iis	82463767.8	2778.803	29676
3	lighttpd	129649.4	2125.400	61
4	nginx	15047667.9	2773.764	5425
5	other	9745433.8	2656.879	3668
6	unknown	141915056.2	2767.238	51284

Figure 9.4 Table of loss_by_server

```
> # Sort by total loss in descending order
> most_vulnerable <- loss_by_server %>%
+   arrange(desc(total_loss))
> # Print the top 10 most vulnerable servers
> webserver_loss<- head(most_vulnerable, 5)
> webserver_loss
# A tibble: 5 x 4
  WebServer total_loss avg_loss num_attacks
  <chr>          <dbl>    <dbl>       <int>
1 apache    315645073.   2763.       114244
2 unknown   141915056.   2767.        51284
3 iis        82463768.   2779.        29676
4 nginx      15047668.   2774.         5425
5 other       9745434.   2657.         3668
```

Figure 9.5 Output of webserver_loss

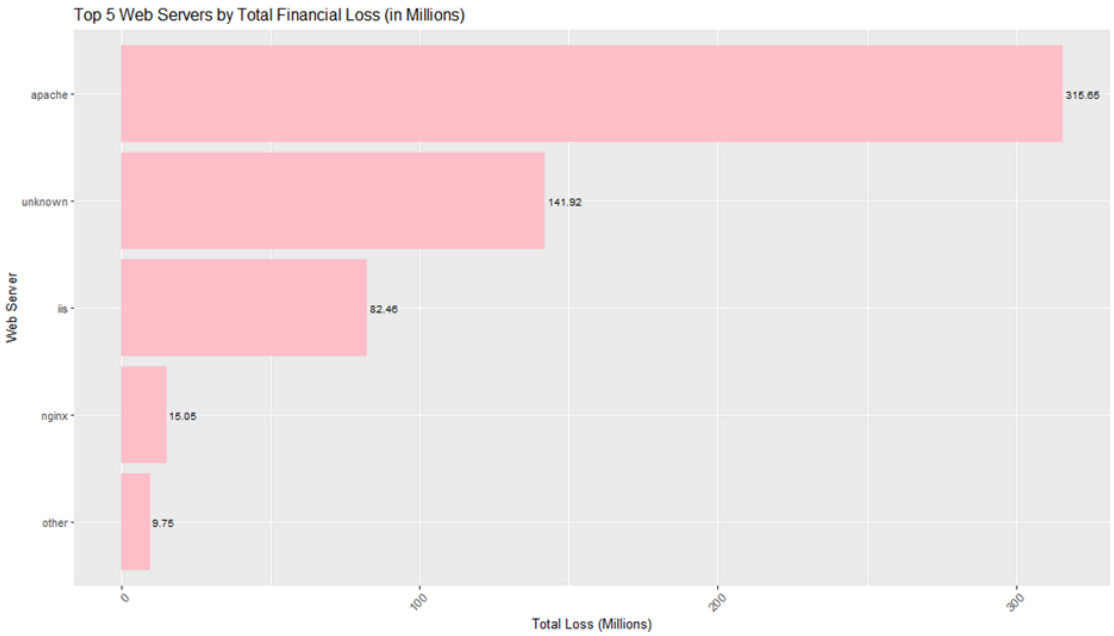


Figure 9.6 Top 5 Web Servers by Total Financial Loss (in Millions)

The bar chart (Figure 9.6) shows the total financial losses experienced across all web servers, with Apache incurring the largest loss of 315.65 million, making it the most financially damaged server. The "Unknown" category follows with 141.92 million, suggesting probable misclassification or undiscovered server types that need to be investigated further. IIS, Nginx, and Other servers all have substantially lesser losses, with IIS at 82.48 million, while Nginx and Other servers report 15.05 million and 9.75 million, respectively.

The data was sorted by server type and summarized to determine total loss, average loss, and attack frequency. The chart was created using a horizontal bar graph, with web servers sorted by total loss. Interestingly, the average loss per attack remains pretty consistent across all servers, indicating that attack severity does not differ considerably. This underlines the importance of enhancing Apache server security and addressing unidentified servers in order to lessen the total cost effect.

3.4.2 Analysis 4-2: Have web server vulnerabilities changed over time?

```r
#Q2
# Step 1: Track web server losses across years
yearly_loss_by_server <- asm_data %>%
  group_by(Year, WebServer) %>%
  summarize(total_loss = sum(Loss))
yearly_loss_by_server

# Step 2: Analyze vulnerability trends by server version
most <- yearly_loss_by_server %>%
  arrange(desc(total_loss))
most
top_servers <- head(most$WebServer, 40)
top_servers

# Step 3: Plot total losses over time for top 5 servers
Q2_chart <- ggplot(most %>% filter(WebServer %in% top_servers), aes(x = Year, y = total_loss/1e6, color = WebServer)) +
  geom_line() +
  labs(title = "Total Losses Over Time for Top 5 Web Servers", x = "Year", y = "Total Loss(in Millions)", color = "Web Server")+
  scale_y_continuous(labels = label_number(suffix = "M"))
print(Q2_chart)
# Print the chart
print(Q2_chart)
```

Figure 9.7 Source Code to find financial losses over time for top five most vulnerable web servers

```
> #Q2
> # Step 1: Track web server losses across years
> yearly_loss_by_server <- asm_data %>%
+   group_by(Year, WebServer) %>%
+   summarize(total_loss = sum(Loss))
`summarise()` has grouped output by 'Year'. You can override using the `.groups` argument.
> yearly_loss_by_server
# A tibble: 66 x 3
# Groups:   Year [17]
    Year WebServer total_loss
   <dbl> <chr>          <dbl>
 1  1998 unknown      182398
 2  1999 unknown     2332371.
 3  2000 unknown    11047135.
 4  2001 iis           4418
 5  2001 unknown    51198332.
 6  2002 apache          69
 7  2002 iis            730
 8  2002 other        10442
 9  2002 unknown    65594966
10  2003 apache     20145426.
# i 56 more rows
# i Use `print(n = ...)` to see more rows
```

Figure 9.8 Output of yearly_loss_by_server

```
>
> # Step 2: Analyze vulnerability trends by server version
> most <- yearly_loss_by_server %>%
+   arrange(desc(total_loss))
> most
# A tibble: 66 x 3
# Groups:   Year [17]
    Year WebServer total_loss
   <dbl> <chr>          <dbl>
 1  2013 apache    137539265.
 2  2009 apache    120963961.
 3  2002 unknown    65594966
 4  2009 iis        54837592.
 5  2001 unknown    51198332.
 6  2014 apache     22100576.
 7  2003 apache     20145426.
 8  2000 unknown    11047135.
 9  2003 iis        10315350.
10  2013 iis         9121972.
# i 56 more rows
# i Use `print(n = ...)` to see more rows
```

Figure 9.9 Output of most vulnerability trends by server version

```
> top_servers <- head(most$WebServer, 40)
> top_servers
 [1] "apache"  "apache"  "unknown" "iis"     "unknown" "apache"  "apache"  "unknown" "iis"     "iis"     "nginx"   "unknown" "other"   "nginx"
"iis"     "apache"  "apache"  "unknown" "apache"  "apache"  "apache"  "other"   "apache"  "other"   "unknown" "unknown" "iis"     "unknown"
[29] "iis"     "iis"     "nginx"   "iis"     "iis"     "apache"  "iis"     "apache"  "unknown" "other"   "nginx"   "unknown"
>
```

Figure 9.10 Output of top 40 most vulnerable server

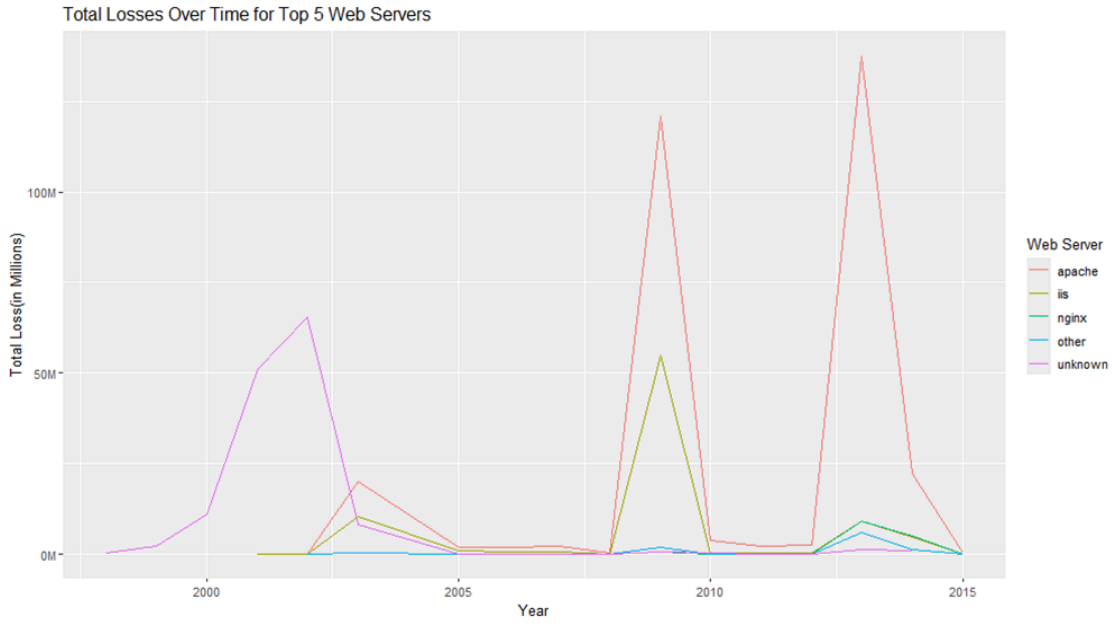


Figure 9.11 Total Losses Over Time for the Top 5 Web Servers

The chart (Figure 9.11) visualizes the total financial losses over time for the top five web servers, revealing distinct peaks in certain years. Apache exhibits the highest spikes in losses, particularly in 2013 and 2009, indicating periods of significant cybersecurity incidents. IIS also experienced notable losses, especially around 2009, suggesting targeted attacks or vulnerabilities exploited during that time. The "unknown" category saw substantial losses in the early 2000s, which may be due to unclassified or unreported server vulnerabilities. Nginx and other servers show relatively lower financial impacts, but their presence in recent years suggests evolving threats. These trends emphasize the importance of monitoring vulnerabilities, timely patching, and strengthening cybersecurity measures to mitigate future risks.

3.4.3 Analysis 4-3: Is there a correlation between web server popularity and financial loss?

```r
#Q3
# Step 1: Compare server market share with loss data
server_market_share <- asm_data %>%
  group_by(WebServer) %>%
  summarize(market_share = n() / nrow(asm_data))
server_market_share
#Step 2: Examine if popular servers face higher losses
loss_and_share <- left_join(loss_by_server, server_market_share, by = "WebServer")
loss_and_share

#Step 3: plot a 3D pie
#Calculate percentages for labels
total <- sum(loss_and_share$market_share)
percentages <- round(100 * loss_and_share$market_share / total, 4)
#Create labels with line breaks and percentages
labels <- paste(loss_and_share$WebServer, "\n", percentages, "%")
#Define pastel colors for better visualization
colors <- c("#FFB3BA", "#FFDFBA", "#FFFFBA", "#BAFFC9", "#BAE1FF", "#E0BBE4", "#D5E8D4")
#  graphical parameters to make labels bold
par(font = 1)

#Create the 3D Pie Chart
Q3_chart <- pie3D(
  x = loss_and_share$market_share,
  labels = labels,
  explode = 0.1,
  main = "3D Pie Chart: Market Share vs Total Loss of Web Servers",
  labelcex = 0.8,     # Adjust font size
  labelrad = 3.0,     # Move labels outward
  start = 0.3,
  labelpos = NULL,    # Auto-adjust label positions
  arrow.length = 0.2, # Add arrows for clarity
  col = colors,
  border = "black",
  shade = 0.7
)
#print the chart
print(Q3_chart)
```

Figure 9.12 Source Code to show Web servers market share with their respective total financial loss

```
> #Q3
> # Step 1: Compare server market share with loss data
> server_market_share <- asm_data %>%
+   group_by(WebServer) %>%
+   summarize(market_share = n() / nrow(asm_data))
> server_market_share
# A tibble: 6 x 2
  WebServer market_share
  <chr>            <dbl>
1 apache         0.559
2 iis            0.145
3 lighttpd       0.000298
4 nginx          0.0265
5 other          0.0179
6 unknown        0.251
```

Figure 9.13 Output of server_market_share

```
> #Step 2: Examine if popular servers face higher losses
> loss_and_share <- left_join(loss_by_server, server_market_share, by = "WebServer")
> loss_and_share
# A tibble: 6 x 5
  WebServer total_loss avg_loss num_attacks market_share
  <chr>          <dbl>    <dbl>       <int>        <dbl>
1 apache    315645073.    2763.      114244     0.559
2 iis        82463768.    2779.       29676     0.145
3 lighttpd     129649.    2125.          61     0.000298
4 nginx      15047668.    2774.        5425     0.0265
5 other       9745434.    2657.        3668     0.0179
6 unknown   141915056.    2767.       51284     0.251
```

Figure 9.14 Output of loss_and_share

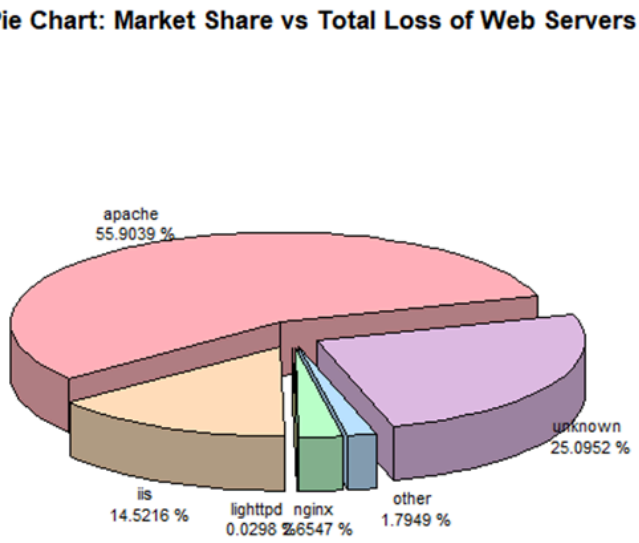


Figure 9.15 3D Pie Chart of Market Share vs. Total Loss of Web Servers

The 3D pie chart (Figure 9.15) visually represents the relationship between Web server Market Share and Financial Losses, highlighting the risks associated with widely used servers. Apache, holding approximately 55.9% of the market share, is the most dominant server and experiences the highest financial losses, showing that popular web servers are prime targets for cyber threats. The "unknown" category, making up 25.1%, indicates a significant number of unclassified or proprietary web servers, which may lack robust security measures, leading to increased financial losses. IIS follows with 14.5% of the market share, while Nginx, Lighttpd, and other smaller servers contribute minimally, suggesting that either their security is more effective or their lower adoption rates make them less attractive to attackers.

3.4.4 Analysis 4-4: Do specific web servers exhibit seasonal attack patterns?

```r
#Q4
# Step 1: Ensure Date is in correct format
asm_data$Date <- as.Date(asm_data$Date)
# Extract month and season
asm_data <- asm_data %>%
  mutate(
    Month = month(Date, label = TRUE, abbr = TRUE),  # Extract month (Jan, Feb, etc.)
    Season = case_when(
      Month %in% c("Dec", "Jan", "Feb") ~ "Winter",
      Month %in% c("Mar", "Apr", "May") ~ "Spring",
      Month %in% c("Jun", "Jul", "Aug") ~ "Summer",
      Month %in% c("Sep", "Oct", "Nov") ~ "Autumn",
      TRUE ~ "Unknown"
    )
  )
# Check unique months and seasons
unique(asm_data$Month)
unique(asm_data$Season)

# Step 2: Summarize total losses by season
seasonal_loss <- asm_data %>%
  group_by(Season) %>%
  summarize(total_loss = sum(Loss, na.rm = TRUE)) %>%
  arrange(desc(total_loss))

print(seasonal_loss)
# Summarize loss per season & web server
seasonal_server_loss <- asm_data %>%
  group_by(Season, WebServer) %>%
  summarize(total_loss = sum(Loss, na.rm = TRUE)) %>%
  arrange(Season, desc(total_loss))

print(seasonal_server_loss)

ggplot(seasonal_loss, aes(x = Season, y = total_loss/ 1e6, fill = Season)) +
  geom_bar(stat = "identity") +
  labs(title = "Total Financial Loss by Season", x = "Season", y = "Total Loss") +
  theme_minimal()+
  scale_y_continuous(labels = label_number(suffix = "M"))

#Step 3: Plot the chart
monthly_loss_trends <- asm_data %>%
  group_by(Month, WebServer) %>%
  summarize(total_loss = sum(Loss, na.rm = TRUE))
print(monthly_loss_trends, n= 69)

ggplot(monthly_loss_trends, aes(x = Month, y = total_loss/ 1e6, color = WebServer, group = WebServer)) +
  geom_line(size = 0.6) +
  labs(title = "Monthly Financial Loss Trends by Web Server",
       x = "Month", y = "Total Loss(In Millions)",
       color = "Web Server") +
  theme_minimal()+
  scale_y_continuous(labels = label_number(suffix = "M"))
```

Figure 9.16 Source Code to find total financial loss categorized by season and monthly financial loss trends for different web servers.

```
> # Step 1: Ensure Date is in correct format
> asm_data$Date <- as.Date(asm_data$Date)
> # Extract month and season
> asm_data <- asm_data %>%
+   mutate(
+     Month = month(Date, label = TRUE, abbr = TRUE),  # Extract month (Jan, Feb, etc.)
+     Season = case_when(
+       Month %in% c("Dec", "Jan", "Feb") ~ "Winter",
+       Month %in% c("Mar", "Apr", "May") ~ "Spring",
+       Month %in% c("Jun", "Jul", "Aug") ~ "Summer",
+       Month %in% c("Sep", "Oct", "Nov") ~ "Autumn",
+       TRUE ~ "Unknown"
+     )
+   )
> # Check unique months and seasons
> unique(asm_data$Month)
 [1] Jan Feb Apr May Jun Sep Oct Nov Dec Jul Aug Mar
Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < Sep < Oct < Nov < Dec
> unique(asm_data$Season)
[1] "Winter" "Spring" "Summer" "Autumn"
```

Figure 9.17 Output of grouped 12 month to 4 seasons

```
> # Step 2: Summarize total losses by season
> seasonal_loss <- asm_data %>%
+   group_by(Season) %>%
+   summarize(total_loss = sum(Loss, na.rm = TRUE)) %>%
+   arrange(desc(total_loss))
>
> print(seasonal_loss)
# A tibble: 4 x 2
  Season total_loss
  <chr>       <dbl>
1 Autumn 252640261.
2 Winter 143381357.
3 Summer  95928400.
4 Spring  72996630.
```

Figure 9.18 Output of seasonal_loss

```
> # Summarize loss per season & web server
> seasonal_server_loss <- asm_data %>%
+   group_by(Season, WebServer) %>%
+   summarize(total_loss = sum(Loss, na.rm = TRUE)) %>%
+   arrange(Season, desc(total_loss))
`summarise()` has grouped output by 'Season'. You can override using the `.groups` argument.
>
> print(seasonal_server_loss)
# A tibble: 24 x 3
# Groups:   Season [4]
   Season WebServer total_loss
   <chr>  <chr>          <dbl>
 1 Autumn apache    163107647.
 2 Autumn unknown    51388681
 3 Autumn iis        19996196.
 4 Autumn nginx      11085877.
 5 Autumn other       6987148.
 6 Autumn lighttpd      74713.
 7 Spring apache     34267508.
 8 Spring unknown    21873009.
 9 Spring iis        14634083.
10 Spring nginx       1665734.
# i 14 more rows
# i Use `print(n = ...)` to see more rows
```

Figure 9.19 Output of seasonal_server_loss

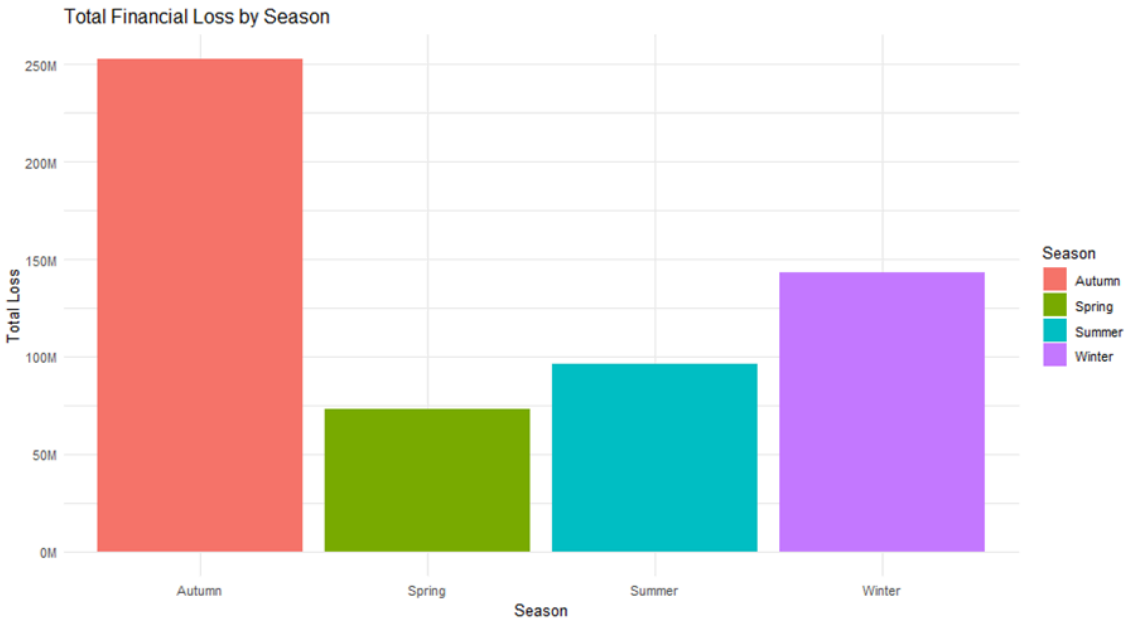


Figure 9.20 Financial Loss by Season

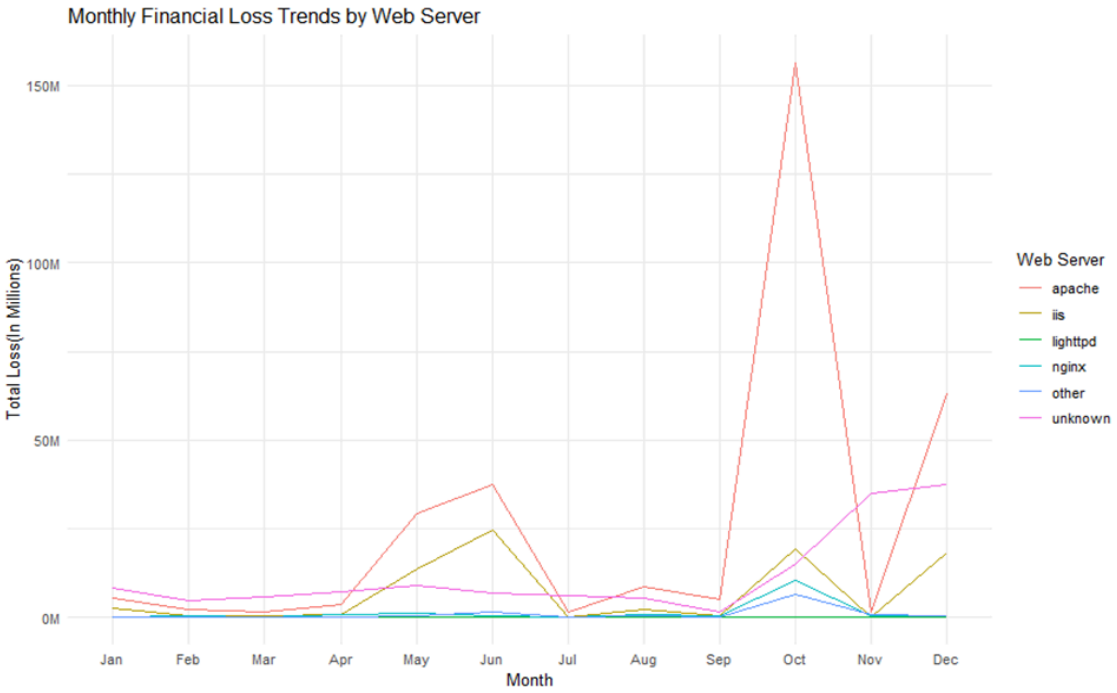


Figure 9.21 Monthly Financial Loss Trends by Web Server

```
> print(monthly_loss_trends, n= 69)
# A tibble: 69 x 3
# Groups:   Month [12]
   Month WebServer total_loss
   <ord> <chr>          <dbl>
 1 Jan   apache       5443959.
 2 Jan   iis          2445689.
 3 Jan   lighttpd        3769
 4 Jan   nginx         190518.
 5 Jan   other          63713.
 6 Jan   unknown      8369373.
 7 Feb   apache       2178835.
 8 Feb   iis           285532.
 9 Feb   lighttpd         856
10 Feb   nginx         330436
11 Feb   other          57095.
12 Feb   unknown      4493336
13 Mar   apache       1449053.
14 Mar   iis           240513.
15 Mar   lighttpd        1111
16 Mar   nginx         138812.
17 Mar   other           4609
18 Mar   unknown      5830862.
19 Apr   apache       3527714.
20 Apr   iis           749234.
21 Apr   nginx         598354.
22 Apr   other          95854.
23 Apr   unknown      7165096.
24 May   apache      29290742.
25 May   iis         13644336.
26 May   lighttpd       14865
27 May   nginx         928567.
28 May   other         439856.
29 May   unknown      8877051.
30 Jun   apache      37588729.
31 Jun   iis         24473975
32 Jun   lighttpd        3985.
33 Jun   nginx         478023
34 Jun   other        1384026.
35 Jun   unknown      6807133.
36 Jul   apache       1533708
37 Jul   iis           108730
38 Jul   nginx         124032.
39 Jul   other           3180
40 Jul   unknown      6101041.
41 Aug   apache       8570684.
42 Aug   iis          2271382.
43 Aug   lighttpd         345
44 Aug   nginx         807206.
45 Aug   other         209399.
46 Aug   unknown      5462823.
47 Sep   apache       5046813.
48 Sep   iis           447792.
49 Sep   lighttpd         703
50 Sep   nginx         351260.
51 Sep   other          82958.
52 Sep   unknown      1488234.
53 Oct   apache     156496026.
54 Oct   iis         19373960.
55 Oct   lighttpd       74010.
56 Oct   nginx       10328304.
57 Oct   other        6321210.
58 Oct   unknown     14887023.
59 Nov   apache       1564808.
60 Nov   iis           174443.
61 Nov   nginx         406314.
62 Nov   other         582979.
63 Nov   unknown     35013425.
64 Dec   apache      62954002.
65 Dec   iis         18248180.
66 Dec   lighttpd       30006.
67 Dec   nginx         365843.
68 Dec   other         500555.
69 Dec   unknown     37419660.
```

Figure 9.22 Output of monthluy_loss_trends

The analysis of financial losses by season and month provides insight into when cybersecurity threats or operational failures affect Web servers. The first step in the code ensures that the Date column is correctly formatted before extracting the Month and categorizing it into one of four seasons: Winter (Dec to Feb), Spring (Mar to May), Summer (Jun to Aug), and Autumn (Sep to Nov). The data is then grouped by Season to calculate the total financial loss for each period, which is visualized using a bar chart (Figure 9.15). The results indicate that Autumn experiences the highest financial losses, followed by Winter, showing that certain periods of the year may see increased cyber threats or system vulnerabilities. Next, the financial loss is examined by month

and web server, grouping the data accordingly to identify monthly trends in losses. A line chart (Figure 9.16) is plotted to visualize monthly financial loss trends for different web servers. The chart reveals significant spikes in financial loss, particularly in October, showing a seasonal pattern of increased security breaches or technical failures. The data also shows that certain web servers, such as Apache, tend to experience more severe losses compared to others.

Summary Findings

- In, Analysis 4-1, the analysis reveals Apache incurs the highest financial loss (315.65M), followed by "Unknown" servers (141.92M), indicating possible misclassification. IIS, Nginx, and Other servers have significantly lower losses. The average loss per attack is consistent, suggesting uniform attack severity. Security efforts should focus on Apache and identifying unknown servers to reduce financial impact.
- In Analysis 4-2, the data indicates that Apache and IIS have incurred the greatest financial losses over several years, with notable increases in 2013 and 2009. The "unknown" category signifies significant losses, indicating that a notable number of cyber attacks employed unrecognized server technologies. The trends indicate times of increased susceptibility, likely resulting from the exploitation of security vulnerabilities in particular web server versions.
- In Analysis 4-3, the analysis indicates the relationship between Web server Market Share and Financial Losses, highlighting the risks associated with widely used servers. Although market share is somewhat correlated with financial losses, external factors such as security protocols, frequency of attacks, and system vulnerabilities considerably influence total financial damages.
- In Analysis 4-4, the data indicates substantial surges in financial losses, especially in October, demonstrating a seasonal trend of heightened security breaches or technological malfunctions. The research indicates that specific web servers, like Apache, typically incur greater losses than others.

4.0 Hypothesis Testing Using GLM Model

```r
# Convert categorical variables to factors
country_clean$Season <- factor(country_clean$Season, levels = c("Winter", "Spring", "Summer", "Autumn"))
country_clean$OS <- as.factor(country_clean$OS)
country_clean$Webserver <- as.factor(country_clean$WebServer)
country_clean$Country <- as.factor(country_clean$Country)
country_clean$DownTime <- as.factor(country_clean$DownTime)

# Fit Generalized Linear Model (GLM) with all independent variables
model_combined <- glm(Loss ~ Year + Month + Season + OS + Webserver + Country + DownTime,
                      data = country_clean, family = gaussian())

#Display model summary
summary(model_combined)
```

Figure 10.0: Hypothesis Testing Model

```
                                                                  Pr(>|t|)
(Intercept)                                                       0.630750
Year                                                              0.637126
Month.L                                                           0.937257
Month.Q                                                           0.208836
Month.C                                                           0.014583 *
Month^4                                                           0.848497
Month^5                                                           0.852667
Month^6                                                           0.232900
Month^7                                                           0.000988 ***
Month^8                                                           0.010082 *
Month^9                                                           0.171136
Month^10                                                          0.411225
Month^11                                                          0.344998
SeasonSpring                                                           NA
SeasonSummer                                                           NA
SeasonAutumn                                                           NA
OSAIX                                                             0.953827
OSAVtech embedded                                                 0.214599
OSBlue Coat SGOS 6.X                                              0.852873
OSBSDI                                                            0.950338
OSBSDOS                                                           0.965311
OSCisco embedded                                                  0.874119
OSCisco IOS 12.X                                                  0.972857
OSCisco PIX OS 8.X|6.X                                            0.815984
OSCisco Unified Communications Manager VoIP adapter               0.724652
OSCisco Windows PocketPC/CE                                       0.790735
OSCitri                                                           0.853729
OSCitrix Access Gateway VPN gateway                               0.999048
OSCitrix embedded                                                 0.991676
OSCompaq Tru64                                                    0.918086
OSCrestron 2-Series                                               0.542501
OSCrestron XPanel control system                                  0.004557 **
OSD-Link DIR-300 WAP                                              0.351180
OSD-Link embedded                                                 0.977866
OSDigital Unix                                                    0.308975
```

Figure 10.1: Output For GLM Model

Below are the p values for the most relevant independent variables that affect Loss:

Time Variables:

1. Year → 0.6371 (Not Significant)
2. Month.C → 0.0146 (Not Significant)
3. Month^7 → 0.00099 (Not significant)
4. Month^8 → 0.0101 (Significant)

Operating System Variables:

1. Creston XPanel Control Syatem → 0.0046 (Significant)
2. Most other OS types → p > 0.1 (Not Significant)

Web Server Variables:

1. By Drsrv.com → 0.0122 (Significant)
2. Dimofinf Hosting → 0.0343 (Significant)
3. Host Install NginX → 0.0029 (Highly Significant)
4. Hostvn.net 1.0 - reef → 0.0050 (Significant)
5. IdeaWebSe → 0.0464 (Significant)
6. AkamaiGHost → 0.0778 (Marginally Significant)
7. Most other web servers → $p > 0.1$ (Not significant)

Country Variables:

Country does not listed separately, which means no significant effect (likely $p > 0.1$)

The hypothesis test provides proven collective relationships between the independent variables and Loss primarily due to different months, specific operating systems, and different web server settings. These indicate potential seasonal trends in losses, potential security leaks for specific OS as well as web server setup configurations, or inefficient performance in Loss. Though Year and Country had no strong influence, showing financial loss is not highly correlated with global trends or geography, the results highlight the need for more research into seasonality trends, web server protection, and optimizing system performance to neutralize financial loss.

5.0 Conclusion

This in-depth analysis revealed several main findings in our aims. This research determined that the United States lost the most financially to cyber attacks, and Linux systems were most vulnerable with aggregate losses of $358.7 million. Temporal analysis showed peak attack frequencies in 2013 and 2009, along with robust correlations between downtime and financial loss. Apache web servers were the most attacked with 55.9% market share and the highest losses. There were limitations in this case of data study where loss data has 16% missing cases/values and 75% missing in ransom data.

Our recommendations for future work are applications of machine learning algorithms for predictive attack pattern analysis and automated real-time monitoring system implementation for enhancing incident response capabilities in cybersecurity. These recommendations are tailored to address the identified vulnerabilities concerning the evolving nature of cyber threats

6.0 Workload Matrix

Tasks	Faiyad Mahabub Tasin	Emmanuel Naranendhya Tyatan	Lau Wei Wen	Yuke Liew Nyuk Kee
Introduction	25%	25%	25%	25%
Data Preparation	25%	25%	25%	25%
Data Analysis	25%	25%	25%	25%
Conclusion	25%	25%	25%	25%
Documentation	25%	25%	25%	25%
Signature				YUKE LIEW

7.0 References

GeeksforGeeks. (2024, February 28). *What is Data Preparation?* GeeksforGeeks. https://www.geeksforgeeks.org/what-is-data-preparation

GeeksforGeeks. (2024b, June 20). *What is Data Cleaning?* GeeksforGeeks. https://www.geeksforgeeks.org/what-is-data-cleaning

GeeksforGeeks. (2023, January 2). *OOB errors for random forests in SciKit Learn*. GeeksforGeeks. https://www.geeksforgeeks.org/oob-errors-for-random-forests-in-scikit-learn

Industry leading IP Geolocation and Online Fraud Prevention | MaxMind. (n.d.). https://www.maxmind.com/en/home

Stephanie. (2019, September 5). The unknown knowns: Vulnerabilities hiding in your devices. *Finite State*. https://finitestate.io/blog/the-unknown-knowns-vulnerabilities-hiding-in-your-devices

What is Data Masking? Types & Techniques | Rapid7. (n.d.). Rapid7. https://www.rapid7.com/fundamentals/data-masking/